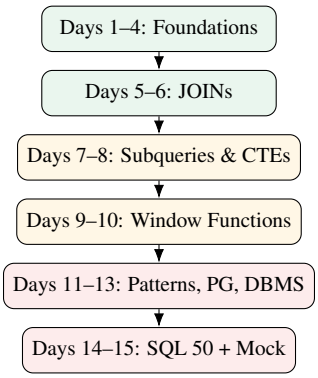
SQL INTERVIEW & PLACEMENT

From “I have never written a query”
to “I can solve LeetCode SQL 50 and explain why.”



PostgreSQL 14+ • 2–3 hours/day

Concept → Mental Model → Syntax → Example → Practice → LeetCode → Interview

Complete with practice database, answer key, and cheat sheets

Contents

Chapter 1

How to Use This Workbook

1.1 What this is

This is a *workbook*, not a reference manual. Every chapter follows the same twelve-part rhythm, and the parts are meant to be done in order:

1. **Learning Objectives** — what you should be able to do by tonight
2. **Concepts** — explained from zero, with the *why*
3. **Mental Model** — what the database is actually doing
4. **Syntax** — PostgreSQL, exactly as you will type it
5. **Worked Examples** — input tables → query → result
6. **Common Mistakes** — broken queries and why they break
7. **Guided Practice** — easy problems, with hints
8. **Independent Practice** — no hints
9. **LeetCode Practice** — mapped to today's concept
10. **Interview Questions** — verbal, conceptual
11. **Daily Quiz** — 5–10 quick checks
12. **Daily Revision** — a checklist to tick off

1.2 The single most important rule

Mental Model

Do not read the answer key until you have written a query and run it.
Reading SQL creates the illusion of understanding. Writing SQL — and watching PostgreSQL reject it with a syntax error — creates actual understanding. Every practice problem in this book has its solution in the Answer Key (Chapter 21), deliberately far away from the question.
Budget your day roughly **40% reading, 60% typing**.

1.3 Setting up in ten minutes

You need PostgreSQL and a way to type queries into it.

1. **Install PostgreSQL.** On Windows use the EDB installer; on macOS, `brew install postgresql@16` or Postgres.app; on Ubuntu, `sudo apt install postgresql`.
2. **Get an administrative prompt.** On Ubuntu, run:

```
sudo -u postgres psql
```

Not `psql -U postgres`. On Linux, the initial administrative connection depends on the local `pg_hba.conf` configuration. A common Ubuntu installation uses *peer* authentication for the `postgres` role, so a user logged into Linux under any other account will be rejected by `psql -U postgres` with `FATAL: Peer authentication failed`. Sections 1.4 to 1.26 explain exactly why, and it is worth understanding rather than memorising. On Windows and macOS the installers usually configure password authentication instead, so `psql -U postgres` works there — do not assume the three platforms behave alike.

3. **Create your practice database:**

```
CREATE DATABASE placement_prep;
\c placement_prep
```

4. **Run the schema script** from Chapter 2. Copy it into a file `setup.sql` and run `\i setup.sql` inside psql.
5. **Verify:** `SELECT COUNT(*) FROM employees;` should return 20.

Interview Insight

If installing locally is a barrier, use a browser-based Postgres sandbox (`pgexercises.com`, `db-fiddle.com` with the PostgreSQL engine selected, or `sqliteonline.com`'s Postgres mode). But by Day 12 you should have a local install — interviewers sometimes ask "have you actually used a database?" and "yes, locally, with psql" is a better answer than "in a web sandbox".

Four psql commands worth memorising

`\l`	list databases
`\dt`	list tables in the current database
`\d employees`	describe the `employees` table (columns, types, keys, indexes)
`\q`	quit

Mental Model

Read the next twenty-odd short sections before Day 1. They take about 45 minutes and they cover the layer *underneath* SQL: what the client is, what the server is, who you are to each of them, and how you are let in. Almost every beginner who gets stuck on "PostgreSQL doesn't work" is stuck here, not on SQL.

Each section is marked with how much of it matters for interviews:

● **Essential**	asked directly in SQL/DBMS interviews
● **Useful**	real PostgreSQL knowledge; shows you have actually used a database
● **Optional**	background; know it exists, do not memorise it

1.4 PostgreSQL client vs server

● **Essential**

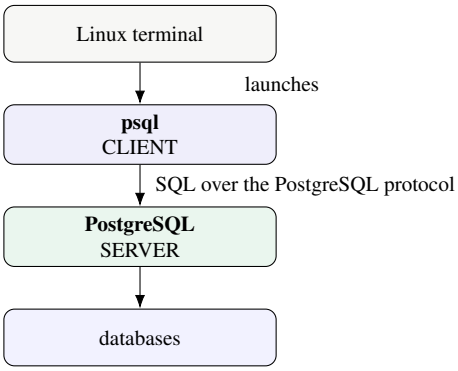
Concept

PostgreSQL is the database management system — the *server*. It is a program running in the background on your machine (or on a machine somewhere else) that:

- manages databases and stores the data on disk;
- executes SQL that clients send it;
- manages roles and connections;

- handles transactions, indexes, schemas and tables.

psql is the PostgreSQL command-line *client*. It is a completely separate program whose only job is to take what you type, send it to the server, and print what comes back. **psql is not PostgreSQL.**



Mental Model

"psql is the program I use to talk to PostgreSQL."
Other clients exist and talk to the same server in the same way: pgAdmin, DBeaver, a Python script using `psycopg2`, a Java application using JDBC. Everything you learn about connecting and authenticating with psql applies to all of them — which is why this is worth 45 minutes.

1.5 Linux commands vs psql commands vs SQL

● **Essential**

Three different things get typed into a terminal during a PostgreSQL session, and beginners mix them up constantly. They are interpreted by three different programs.

1.5.1 A. Linux shell commands

Typed at your normal shell prompt (usually ending in `\$`). Interpreted by the Linux shell.

```
ls          # list files
cd          # change directory
pwd         # print working directory
whoami      # which Linux user am I?
which psql  # where is the psql program?
sudo        # run something as another Linux user
psql        # launch the PostgreSQL client
```

`whoami` asks *Linux*: "who am I?" It might answer `saiyam-jain`. This has nothing to do with PostgreSQL.

1.5.2 B. psql meta-commands

Typed at the psql prompt (ending in `=\#` or `=>`). They all start with a backslash. Interpreted by **the psql client**, which never sends them to the server as SQL.

```
\l                  -- list databases
\dt                 -- list tables
\d employees        -- describe a table
\du                 -- list roles
\c placement_prep   -- connect to another database
\conninfo           -- how am I connected?
\q                  -- quit
```

These are **not SQL**. They will not work in pgAdmin, in a Python script, or on LeetCode. They are a convenience of this one client.

1.5.3 C. SQL commands

Also typed at the psql prompt, but sent to the server for execution. They end with a semicolon.

```
SELECT * FROM employees;
CREATE DATABASE placement_prep;
CREATE TABLE employees (...);
INSERT INTO employees (...) VALUES (...);
```

1.5.4 Which layer is which?

Command	Interpreted by
`whoami`	Linux shell
`which psql`	Linux shell
`sudo -u postgres psql`	Linux shell — launches psql *as another Linux user*
`psql -U postgres`	Linux shell — launches psql; `-U` names a *PostgreSQL role*
`\du`	psql client
`\dt`	psql client
`\d employees`	psql client
`SELECT ...`	PostgreSQL server
`CREATE TABLE ...`	PostgreSQL server
`INSERT ...`	PostgreSQL server

Common Mistake

Typing `\dt` into LeetCode, or into a Python `cursor.execute()`, produces a syntax error. Backslash commands exist only inside psql. Conversely, typing `SELECT 1;` at your Linux shell gives *command not found* — SQL only means something once you are inside a client connected to a server.

1.6 PostgreSQL roles vs Linux users

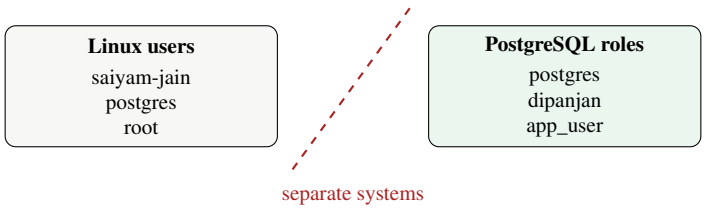
● **Essential**

Concept

These are **two completely separate identity systems** that happen to use similar names.

- A **Linux user** is an account on your operating system: `saiyam-jain`, `root`, and — because the PostgreSQL package creates it during installation — `postgres`.
- A **PostgreSQL role** is an account that exists *inside the database server*: `postgres`, `dipanjan`, `app_user`. PostgreSQL calls them "roles"; older documentation says "users". A role that is allowed to log in is effectively a user.

They are not automatically the same thing, and creating one does not create the other.



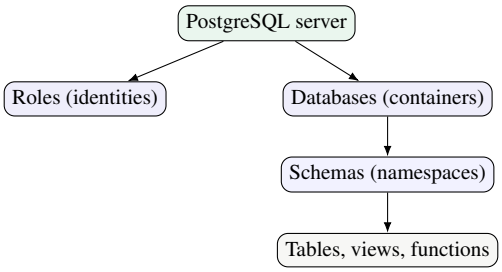
1.6.1 Asking each system who you are

Command	Asks	Typical answer
`whoami`	Linux	`saiyam-jain`
`SELECT current_user;`	PostgreSQL	`postgres`
`SELECT current_database();`	PostgreSQL	`placement_prep`
`\du`	PostgreSQL (via psql)	the full list of roles

Mental Model

`whoami` and `SELECT current_user` can and often *do* return different answers in the same session. That is not a bug — you are asking two different systems two different questions. Internalising this one fact prevents most of the confusion in the rest of this chapter.

1.6.2 The object hierarchy



Role	an identity that can connect and own things
Database	a container holding schemas; you connect *to* one at a time
Schema	a namespace inside a database; the default one is `public`
Table	the actual rows and columns

Roles live at the *server* level, not inside a database — which is why one role can own objects in several databases.

1.7 Understanding psql

● **Useful**

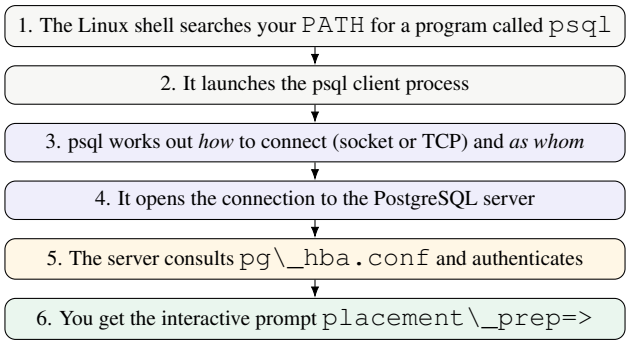
`psql` is a Linux command like any other. Prove it:

```
which psql        # -> /usr/bin/psql   (where the program lives)
psql --version    # -> psql (PostgreSQL) 16.2
```

Common Mistake

`psql -version` reports the **client** version, not the server version. They can differ — you might have a 16.x client talking to a 14.x server. For the server, run `SELECT version();` once you are connected. Confusing the two is a common support-ticket mistake.

What actually happens when you type `psql`:



Steps 3 to 5 are where beginners get stuck, and they are the subject of the next several sections.

1.8 psql connection options

● **Essential**

Four flags do almost everything.

Flag	Means	Default if omitted
`-U`	PostgreSQL **role**	your Linux username
`-h`	host	none — uses a local Unix socket
`-p`	port	5432
`-d`	database	a database named after the role

```
psql -h localhost -p 5432 -U saiyam -d placement_prep
```

Read that aloud, left to right: *"connect over TCP to the machine `localhost`, on port 5432, asking to be authenticated as PostgreSQL role `saiyam`, and put me in the database `placement_prep`."*

Common Mistake

`-U` does not change your Linux user. This is the single most important sentence in this chapter. `psql -U postgres` does **not** mean "become the Linux user postgres". It means *"I am still whoever I was, but please authenticate me as the PostgreSQL role postgres."* Whether PostgreSQL agrees to that is a separate question, decided by `pg_hba.conf` — and under peer authentication the answer is usually no.

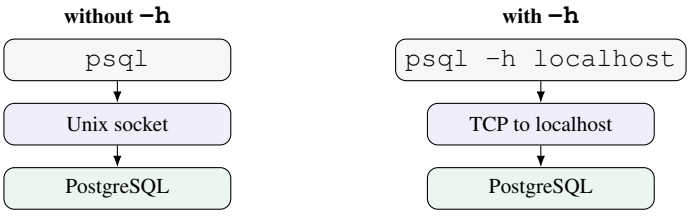
Mental Model

Cheat sheet: `-U` → PostgreSQL role `-h` → host `-p` → port `-d` → database.

1.9 Local Unix socket vs TCP

● **Useful**

There are two physical ways for a client to reach the server, and *which one you use changes how you are authenticated.*



Concept

A **Unix-domain socket** is a special file on the local filesystem that two programs on the *same machine* use to talk to each other. It never touches the network. On Debian and Ubuntu it is typically at

```
/var/run/postgresql/.s.PGSQL.5432
```

but the directory varies between distributions and installations — do not memorise the path. Ask PostgreSQL instead: `SHOW unix_socket_directories;`
TCP is an ordinary network connection, even when the destination is your own machine (`localhost`, address `127.0.0.1`). It is the only option when the server is on a different machine.

Mental Model

Why this matters more than it sounds like it should. `pg_hba.conf` has separate rules for socket connections and TCP connections. So the *same role, same password, same machine* can be accepted one way and rejected the other — purely because you added or omitted `-h localhost`. When a connection mysteriously starts or stops working, this flag is the first thing to check.

1.10 Port 5432

● **Useful**

A TCP connection needs a port number. PostgreSQL's default is **5432**.

```
psql -h localhost -p 5432 -U saiyam -d placement_prep
```

5432 is a convention, not a law. A machine running two PostgreSQL versions side by side will have the second on 5433; a managed cloud database may use something else entirely. Check with `SHOW port;` inside psql, or `sudo ss -lntp | grep postgres` at the shell.

Interview Insight

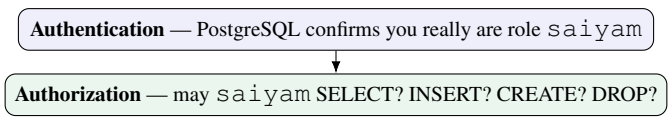
"What port does PostgreSQL use?" is a genuine (if shallow) interview question. Say *"5432 by default, though it is configurable"* — the second clause is what distinguishes a memorised answer from an understood one. For comparison: MySQL 3306, Oracle 1521, SQL Server 1433, MongoDB 27017, Redis 6379.

1.11 Authentication vs authorization

● **Essential**

Concept

Authentication answers: *"Who are you?"*
Authorization answers: *"What are you allowed to do?"*
They happen in that order, and they fail in different ways.



	Authentication failure	**Authorization failure**
When	at connection time	after you are connected
Typical message	`Peer authentication failed`	`permission denied for table employees`
Controlled by	`pg_hba.conf`	`GRANT` / `REVOKE` and object ownership

Mental Model

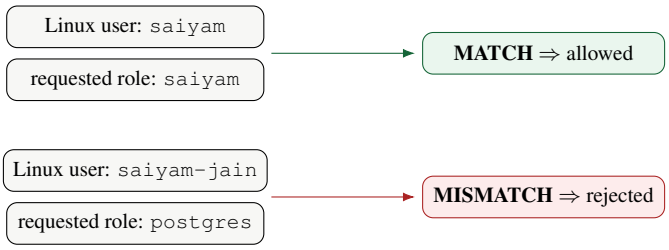
You can pass one and fail the other. Getting a psql prompt proves only that authentication succeeded. If you then run a query and see *permission denied*, nothing is wrong with your login — the role simply lacks a privilege on that object. Recognising which of the two failed tells you where to look, and it is exactly the distinction interviewers are probing when they ask "what is the difference between authentication and authorization?"

1.12 Peer authentication

● **Essential**

Concept

Peer authentication means: for a connection over a local Unix socket, PostgreSQL asks the operating system which Linux user opened the connection, and requires that name to **equal** the PostgreSQL role being requested. No password is involved at all.



Peer is a good default for a personal machine: nobody can reach the database without first having a shell as the right OS user, and there is no password to leak or forget. It applies only to local socket connections — it is meaningless over TCP, because the server on another machine has no way to inspect your OS identity.

Common Mistake

The comparison is a **literal string match**. If `whoami` says `saiyam-jain`, then a role called `saiyam` will not satisfy peer authentication. Run `whoami` and copy the answer exactly.
(PostgreSQL can map OS names to different role names via a `pg_ident.conf` user map. That is real but unnecessary here — know the term exists and move on.)

1.13 Password authentication: SCRAM

● **Essential**

Concept

`scram-sha-256` is PostgreSQL's modern password authentication method, and the default for new installations from version 14 onward. The role has a password stored in the server; the client proves it knows the password without ever sending the password itself across the connection.

```
CREATE USER saiyam WITH PASSWORD 'choose_something';
ALTER USER saiyam WITH PASSWORD 'a_new_one';        -- change it later
```

This is what applications use. A Python or Java service on another machine has no OS identity the database can inspect, so it authenticates with a role name and a password — supplied through environment variables or a secrets manager, never hard-coded in source.

Interview Insight

Say "SCRAM" rather than "password authentication" when you can. The full name is *Salted Challenge Response Authentication Mechanism*; the useful one-line summary is *"the password is never transmitted, and the stored form is salted, so intercepting the exchange does not give an attacker the password."*

1.14 MD5 authentication

● **Optional**

`md5` is the **older** password method, still seen in configurations written before PostgreSQL 10 and in systems upgraded from them. It also uses a role plus password, but the hashing scheme is weaker and it is vulnerable in ways SCRAM is not.

`scram-sha-256`	modern; use for anything new
`md5`	legacy; recognise it, migrate away from it

That is the whole of what you need. If an interviewer pushes further, the honest answer is *"I know MD5 is the legacy password method and SCRAM replaced it; I have not had to work with the cryptographic details."*

1.15 Trust authentication

● **Essential (as a warning)**

Concept

`trust` means PostgreSQL performs **no authentication at all**. Any connection matching the rule is accepted as whatever role it asks for, with no password and no identity check.

Common Mistake

Every "fix" you will find by searching your error message suggests changing `peer` to `trust`. **Do not.** It makes the error disappear by removing the security, not by solving the problem.
And this is genuinely dangerous:

```
host  all  all  0.0.0.0/0  trust     -- NEVER
```

That line means: *anyone on any network who can reach this port is now a database superuser.* No password, no logging in, nothing. Databases have been wiped by automated scanners within minutes of a line like that going live.
The only defensible uses of `trust` are a throwaway container in CI, or a recovery session when a superuser password has been lost — with the server bound to localhost only, and reverted immediately afterwards.

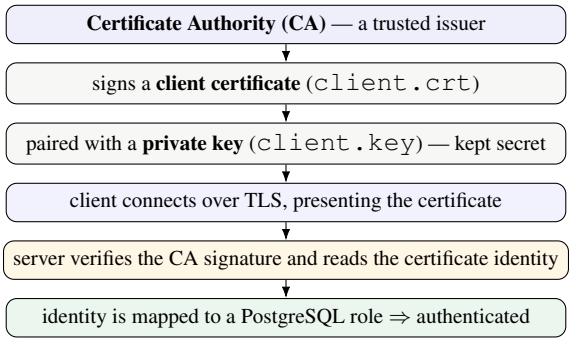
1.16 Certificate authentication

● **Optional (conceptual)**

Concept

`cert` authentication uses a **TLS client certificate** instead of a password. The client proves its identity by possessing a private key matching a certificate that the server trusts.

Password (SCRAM)	Certificate
client sends role name	client presents `client.crt`
proves knowledge of a password	proves possession of `client.key`
server verifies against stored password	server verifies the certificate was signed by a trusted CA



The vocabulary worth knowing: **TLS/SSL** (the encrypted transport), **CA** (who vouches for certificates), **client certificate** (the public credential), **private key** (the secret that must never be shared or committed to git), and **identity mapping** (how the name in the certificate becomes a PostgreSQL role).

A rule would conceptually look like:

```
hostssl  all  all  192.168.1.0/24  cert
```

`hostssl` rather than `host`, because certificates require TLS.

Common Mistake

Do not paste that line into your machine to try it. Certificate authentication needs a configured server certificate, a CA, and generated client key pairs — none of which you have on a fresh install, and all of which are outside what this workbook is for. Understand the concept; leave the PKI administration for when a job requires it.

1.16.1 Method comparison

Method	Identity source	Password?	Typical use
`peer`	OS/Linux identity	No	local Unix socket connections
`scram-sha-256`	role + password	Yes	applications, normal users
`md5`	role + password	Yes	legacy systems
`trust`	nothing checked	No	tightly controlled/throwaway only
`cert`	TLS client certificate	No	strong client identity, machine-to-machine

Mental Model

Four sentences to remember them by:

- **peer** — "Linux says who you are."
- **SCRAM** — "Prove you know the password."

- **cert** — "Prove you hold the private key for a trusted certificate."
- **trust** — "I explicitly trust this connection." (Rarely true.)

1.17 pg_hba.conf

● **Essential**

Concept

`pg_hba.conf` — **H**ost-**B**ased **A**uthentication — is the file that decides which authentication method applies to each incoming connection.
Each line answers five questions:

TYPE	how are you connecting? (socket or TCP)
DATABASE	to which database?
USER	as which role?
ADDRESS	from which network address? (TCP rules only)
METHOD	and therefore, how do we authenticate you?

Find yours — do not guess the path, it varies by version and distribution:

```
sudo -u postgres psql -c "SHOW hba_file;"
```

1.17.1 Reading the rules

```
# TYPE  DATABASE  USER      ADDRESS          METHOD

local   all       postgres                   peer
local   all       all                        scram-sha-256
host    all       all       127.0.0.1/32     scram-sha-256
host    all       all       ::1/128          scram-sha-256
```

Line by line:

`local all postgres peer`	A Unix-socket connection, to any database, as role `postgres`: use peer. So the OS user must be `postgres`.
`local all all scram-sha-256`	Any other socket connection: ask for a password.
`host all all 127.0.0.1/32 scram-sha-256`	A TCP connection from IPv4 localhost: ask for a password.
`host all all ::1/128 scram-sha-256`	The same for IPv6 localhost. Miss this line and `-h localhost` may fail on machines that resolve it to `::1`.

`local` means socket, `host` means TCP, `hostssl` means TCP requiring TLS. The `ADDRESS` column is blank for `local` rules — there is no network address involved.

Common Mistake

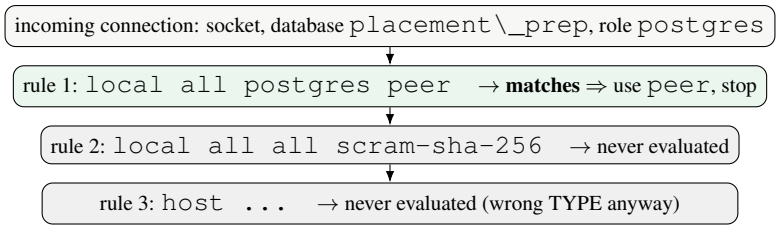
Resist widening the `ADDRESS` column. `0.0.0.0/0` means "the entire internet". Keep local development at `127.0.0.1/32` and `::1/128`.

1.18 How PostgreSQL chooses a rule

● **Essential**

Concept

PostgreSQL reads `pg_hba.conf` **from top to bottom** and uses the **first rule that matches** the connection on all of TYPE, DATABASE, USER and ADDRESS. It then applies that rule's METHOD — and **stops**. Later rules are never considered, even if one of them would have let you in.



Mental Model

First-match is the mechanism behind almost every "but I set up a password and it still won't let me in!" If a broad `peer` rule sits above your new `scram-sha-256` rule, the peer rule wins and your password is never requested. **Order is logic, not cosmetics** — put specific rules above general ones.

1.18.1 Reloading after a change

Editing the file changes nothing until the server re-reads it. A full restart is not needed:

```
sudo systemctl reload postgresql
# or, equivalently, from inside psql as a superuser:
sudo -u postgres psql -c "SELECT pg_reload_conf();"
```

Either one is sufficient. A reload does not drop existing connections; a restart does, so prefer reload.

1.19 `sudo -u postgres psql` VS `psql -U postgres`

● **Essential**

These two commands look almost identical and work completely differently. This is the heart of the chapter.

1.19.1 `sudo -u postgres psql`

Three separate pieces, all Linux:

`sudo`	a Linux program for running a command as a different Linux user
`-u postgres`	*sudo's* flag: "run it as the **Linux user** `postgres`"
`psql`	the command to run, with no options of its own

So: your effective OS identity becomes `postgres`, and *then* psql starts. Since no `-U` was given, psql defaults the role to the current OS username — `postgres`. Peer compares `postgres` with `postgres`, they match, and you are in.

Common Mistake

The `-u` in `sudo -u postgres` belongs to **sudo**, not psql, and it names a **Linux user**. The `-U` in `psql -U postgres` belongs to **psql** and names a **PostgreSQL role**. Same letter, different case, different

program, different meaning entirely.

1.19.2 `psql -U postgres`

Here you remain `saiyam-jain`. psql simply asks the server to authenticate you as the role `postgres`. Under peer, the server compares `saiyam-jain` with `postgres`, finds them different, and refuses.

1.19.3 Side by side

	`psql -U postgres`	`sudo -u postgres psql`
Linux user running psql	`saiyam-jain` (unchanged)	`postgres`
PostgreSQL role requested	`postgres`	`postgres` (defaulted from the OS user)
Peer comparison	`saiyam-jain` $\neq$ `postgres`	`postgres` $=$ `postgres`
Result	**authentication fails**	**authentication succeeds**

Command	**Linux identity**	**Role requested**
`psql`	current Linux user	current OS username (default)
`psql -U postgres`	current Linux user	`postgres`
`sudo -u postgres psql`	`postgres`	`postgres` (default)
`sudo -u postgres psql -U saiyam`	`postgres`	`saiyam`

The last row is worth a moment: both mechanisms at once. OS identity `postgres`, requested role `saiyam` — which peer would *reject*, because they differ.

1.20 Understanding the peer authentication error

● **Essential**

The scenario, exactly as it happens on a fresh Ubuntu install:

```
$ whoami
saiyam-jain

$ psql -U postgres
psql: error: connection to server on socket "/var/run/postgresql/.s.PGSQL.5432" failed
    :
FATAL:  Peer authentication failed for user "postgres"
```

Step by step:

1. The Linux shell runs the command as `saiyam-jain`.
2. `psql` launches — still as `saiyam-jain`, because no `sudo` was used.
3. `-U postgres` asks the server for the role `postgres`.
4. No `-h` was given, so the connection goes over the local Unix socket $\Rightarrow$ this is a `local` connection.
5. The server scans `pg_hba.conf` and the first matching rule is `local all postgres peer`.
6. Peer asks the OS: which user opened this socket? Answer: `saiyam-jain`.
7. `saiyam-jain` $\neq$ `postgres`.
8. The connection is rejected: *Peer authentication failed for user "postgres"*.

Now the same trace for the command that works:

1 `sudo` is a Linux command; it authorises you to act as another Linux user.
2 `-u postgres` makes the effective OS user `postgres`.
3 `psql` launches under that identity, with no `-U`, so the role defaults to `postgres`.
4 Still a socket connection, so the same `local all postgres peer` rule matches.
5 Peer asks the OS: `postgres`. Requested role: `postgres`.
6 They match ⇒ authenticated, and you get the `postgres=\#` prompt.

Mental Model

Notice that *nothing about PostgreSQL changed* between the two commands. The same server, the same rule, the same role. Only your **operating system identity** differed — and under peer authentication that is the entire test.

1.21 Recommended local development configuration

● **Useful**

For this workbook you have two good options. Option A needs no configuration change at all and is what most readers should do.

1.21.1 Option A — a role named after your Linux user (no editing required)

Get an admin prompt, then create a role matching `whoami` exactly, plus your database:

```
sudo -u postgres psql
```

```
-- inside psql, as postgres
CREATE USER dipanjan WITH PASSWORD 'choose_something';
CREATE DATABASE placement_prep OWNER dipanjan;
\q
```

Now, as your normal self:

```
psql -d placement_prep
```

No `-U`, no `-h`, no password: peer matches your OS user against the identically named role. This is the cleanest setup and the one the rest of the workbook assumes.

If your Linux username contains a hyphen (`saiyam-jain`), the role name must be quoted at creation — `CREATE USER "saiyam-jain" WITH PASSWORD '...';` — and quoted at every later reference. If that irritates you, use Option B.

1.21.2 Option B — password authentication over TCP

Create any role you like, then connect over TCP so a password rule applies:

```
psql -h localhost -p 5432 -U saiyam -d placement_prep
```

Nothing to edit here either, provided your `pg_hba.conf` already has the standard `host ... 127.0.0.1/32 scram-sha-256` line — most installations do.

1.21.3 If you do need to edit pg_hba.conf

Only if the above fails. Never overwrite the file wholesale.

```
# 1. find the real path (it varies by version and distro)
sudo -u postgres psql -c "SHOW hba_file;"

# 2. back it up first, using the path you just discovered
```

```
sudo cp /path/from/step/1/pg_hba.conf /path/from/step/1/pg_hba.conf.backup

# 3. edit it
sudo nano /path/from/step/1/pg_hba.conf

# 4. reload
sudo systemctl reload postgresql
```

A sound local configuration looks like this:

```
# TYPE  DATABASE  USER      ADDRESS         METHOD

local   all       postgres                  peer
local   all       all                       scram-sha-256
host    all       all       127.0.0.1/32    scram-sha-256
host    all       all       ::1/128         scram-sha-256
```

Line 1	the superuser stays on peer — reachable only by someone who can already become the `postgres` OS user
Line 2	everyone else connecting locally uses a password
Lines 3–4	TCP from this machine only (IPv4 and IPv6), password required

Common Mistake

Your existing file almost certainly contains additional distribution-specific lines — rules for replication, for `all` databases with different methods, sometimes comments that matter. **Do not delete rules you do not understand.** Add or adjust the specific line you need, keep it above more general rules (first match wins), and keep the backup.

1.22 Practical connection lab

● **Useful**

Do this at the terminal now. It takes ten minutes and makes everything above concrete. Answers are in the Answer Key, Section 21.16.

Practice

L1 Run `whoami`. Write the answer down — you will need it. Which system did you just ask?

L2 Run `which psql` and `psql -version`. What do these tell you, and which version is *not* reported here?

L3 Run `sudo -u postgres psql`. At the prompt, run each of these and say what each one asked and who answered:

```
SELECT current_user;
SELECT current_database();
SELECT version();
\du
\l
\conninfo
\q
```

L4 Now run `psql -U postgres`. If it fails — **do not look up the fix**. Write down, in your own words, the eight-step trace of exactly why, naming your Linux user, the requested role, the connection type, and the rule that matched.

L5 Create a role named after your Linux user and a database, as in Section 1.21. Then run `psql -d placement_prep` with no other flags. Why does this now work without a password?

L6 Connect again over TCP: `psql -h localhost -U <your role> -d placement_prep`. What changed? Which line of `pg_hba.conf` handled this connection, and why were you treated differently from L5 despite being the same person on the same machine?

L7 Inside psql, run `SHOW hba_file;`, `SHOW port;` and `SHOW unix_socket_directories;`. Compare the answers with the paths quoted in this chapter.

1.23 Troubleshooting connections

● **Useful**

Error	Meaning	What to investigate
`Peer authentication failed for user "X"`	Your OS user does not equal the requested role	`whoami`; the role name; the matching `local` rule. Use `sudo -u postgres psql`, or create a role matching your OS user
`role "X" does not exist`	You authenticated as far as naming a role that is not there	`\du` to list roles; `CREATE USER X ...`
`database "X" does not exist`	psql defaulted the database to your role name	`\l` to list databases; pass `-d placement_prep`
`password authentication failed for user "X"`	Wrong password, or the role has none	`ALTER USER X WITH PASSWORD '...';` as a superuser
`connection refused`	Nothing is listening on that host and port	`systemctl status postgresql`; check `-p`; check `listen_addresses`
`No such file or directory` (socket)	The server is not running, or the socket is elsewhere	`sudo systemctl start postgresql`; `SHOW unix_socket_directories;`
`permission denied for table employees`	**Not** an authentication problem — you are connected	`SELECT current_user;`; `GRANT`; or re-run the setup script as the role you actually use

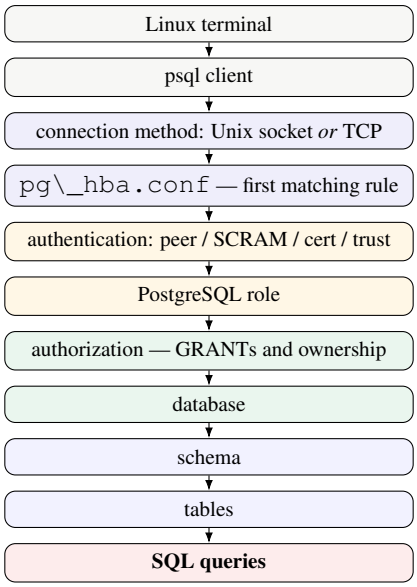
Common Mistake

Two habits to avoid when troubleshooting: switching everything to `trust` (removes the security instead of fixing the cause), and restarting PostgreSQL for a `pg_hba.conf` change (a `reload` suffices and does not drop connections).

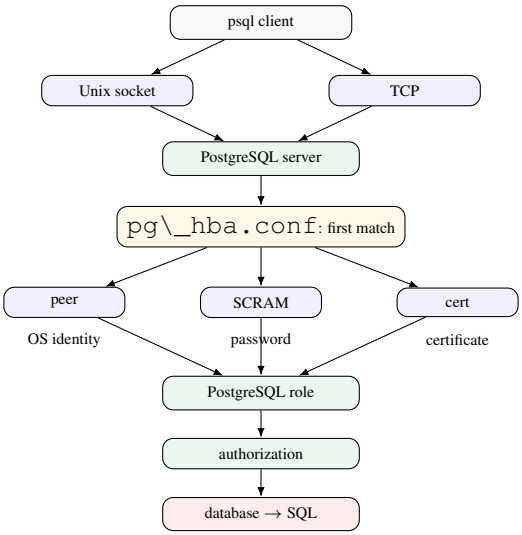
1.24 The mental model

● **Essential**

Everything in this chapter is one pipeline. **SQL is the last step, not the first.**



In prose: the operating system launches a **client**. The client chooses a **connection method** — socket if you gave no `-h`, TCP if you did. The server looks up `pg\_hba.conf` and takes the **first rule** matching the connection type, database, role and address. That rule names an **authentication method**, which decides *who you are*: your OS name (peer), a password (SCRAM/md5), a certificate (cert), or nothing at all (trust). Once you are a known **role**, **authorization** decides *what you may do*. Only then do you reach a **database**, its **schemas** and **tables** — and only then does your **SQL** run.



You should be able to redraw this from memory. If you can, you can diagnose almost any connection problem by asking, at each layer: *did we get past this one?*

1.25 Connection and authentication cheat sheet

POSTGRESQL CONNECTION & AUTHENTICATION

Linux commands

```
whoami                    # which Linux user am I?
which psql                # where is the client program?
psql --version            # CLIENT version (not the server's)
sudo systemctl status postgresql
sudo systemctl reload postgresql     # after editing pg_hba.conf
```

Connecting

```
psql                                  # socket, role = your OS username
psql -d placement_prep                # socket, choose the database
psql -U postgres                      # socket, request role postgres  (peer may reject)
sudo -u postgres psql                 # run psql AS the Linux user postgres
psql -h localhost -U saiyam           # TCP -> password rule applies
psql -h localhost -p 5432 -U saiyam -d placement_prep
```

Flags `-U` role · `-h` host · `-p` port · `-d` database

psql meta-commands `\l` databases · `\du` roles · `\dt` tables · `\d t` describe · `\c db` switch · `\conninfo` · `\q`

SQL for orientation

```
SELECT current_user;      SELECT current_database();     SELECT version();
SHOW hba_file;            SHOW port;      SHOW unix_socket_directories;
```

Authentication methods `peer` OS identity · `scram-sha-256` password (modern) · `md5` password (legacy) · `trust` no check (avoid) · `cert` TLS client certificate

pg_hba.conf columns: TYPE · DATABASE · USER · ADDRESS · METHOD. `local` = socket, `host` = TCP. **First matching rule wins.**

1.26 Master summary

`psql`	the PostgreSQL *client*
PostgreSQL	the database *server*
`-U`	the PostgreSQL role you are requesting
`sudo -u`	the Linux user the command runs as
`peer`	authenticate using the OS identity
`scram-sha-256`	authenticate with a password
`cert`	authenticate with a certificate and private key
`trust`	no authentication — avoid
`pg_hba.conf`	the rules deciding which method applies
`local`	a Unix-socket connection
`host`	a TCP connection
5432	the default PostgreSQL TCP port
Authentication	*Who are you?*
Authorization	*What are you allowed to do?*

Mental Model

Linux → psql → socket/TCP → `pg\_hba.conf` → authentication → PostgreSQL role → authorization → database → SQL

1.27 Interview questions

Answers are in the Answer Key, Section 21.17. Say each one aloud before you read the answer.

Practice

1. What is PostgreSQL?
2. What is psql?
3. Is psql the PostgreSQL server?
4. What happens, step by step, when you type `psql`?
5. What does `-U` mean?
6. What does `-h` mean?
7. What does `-p` mean?
8. What does `-d` mean?
9. What is a PostgreSQL role?
10. Is a PostgreSQL role the same as a Linux user?
11. What does `sudo -u postgres psql` do?
12. What is the difference between `sudo -u postgres psql` and `psql -U postgres`?
13. What is peer authentication?
14. Why can `psql -U postgres` fail under peer authentication?
15. What is SCRAM?
16. What is MD5 authentication?
17. What is trust authentication?
18. Why is trust potentially dangerous?
19. What is certificate authentication?
20. What is `pg_hba.conf`?
21. What does HBA stand for?
22. How does PostgreSQL choose which `pg_hba.conf` rule to apply?
23. Unix socket versus TCP — what is the practical difference?
24. Authentication versus authorization?
25. What is PostgreSQL's default port?

1.28 Quiz

Daily Quiz

Part 1 — recall

1. What is the difference between the PostgreSQL server and psql?
2. Is `psql` a Linux command, a psql meta-command, or SQL?
3. What does `-U` control?
4. What does `sudo -u` control?
5. What is the difference between a Linux user and a PostgreSQL role?
6. Why can `psql -U postgres` fail for a Linux user named `saiyam`?
7. Why does `sudo -u postgres psql` work?
8. What is peer authentication?
9. What is SCRAM?
10. What is certificate authentication?
11. What is `pg_hba.conf`?
12. What does "first matching rule" mean?
13. What is the difference between `local` and `host` in `pg_hba.conf`?
14. What is PostgreSQL's default TCP port?
15. What is the difference between a Unix socket and TCP?
16. What does authentication mean?
17. What does authorization mean?
18. Why should `trust` not be casually enabled?

19. What does `\du` do?
20. What does `SELECT current_user;` do?

Part 2 — scenarios

21. Linux user `alice`; requested role `postgres`; local Unix socket; the matching rule uses `peer`. Does authentication succeed? Why?
22. Linux user `alice`; requested role `postgres`; TCP to localhost; the matching rule uses `scram-sha-256`; she types the correct password. Does it succeed? Explain why this differs from the previous scenario.
23. Linux user `postgres`, running `sudo -u postgres psql`, with peer authentication. Trace the entire flow from shell to prompt.
24. `psql -h localhost -U saiyam -d placement_prep` — explain every option and say what kind of connection is being requested.
25. A user connects successfully, then sees `permission denied for table employees`. Is this an authentication problem or an authorization problem? How do you know?

Daily Revision Checklist

☐ I can explain the difference between psql and the PostgreSQL server
☐ I can classify any command as Linux / psql meta-command / SQL
☐ I know that `whoami` and `SELECT current_user` ask different systems
☐ I can explain `-U` vs `sudo -u` without hesitating
☐ I can trace the peer authentication error in eight steps
☐ I know the five authentication methods and one line about each
☐ I can explain first-match in `pg_hba.conf`
☐ I can redraw the master pipeline from memory
☐ I have completed the connection lab and can connect without `sudo`

1.29 Difficulty markers

Throughout the book:

● **Beginner**	You should be able to do this immediately after reading the day's concepts.
● **Intermediate**	Requires combining two ideas, or a bit of thought about order of operations.
● **Advanced**	Placement-interview level. Expect to struggle the first time — that is the point.

Problems are numbered `D3-P7` (Day 3, Practice 7) so the Answer Key is easy to search.

1.30 The 15-day calendar

Each day is designed for **2–3 hours**. If you only have 2 hours, cut the Independent Practice in half — never cut the Worked Examples or the LeetCode block.

Day	Topic	Time breakdown	Output
1	SQL Fundamentals	20 m setup · 40 m concepts · 50 m practice · 20 m LeetCode · 10 m quiz	10 queries written
2	Filtering & Sorting	35 m concepts · 60 m practice · 30 m LeetCode · 15 m interview Q	10 queries
3	Aggregates, GROUP BY	45 m concepts · 55 m practice · 30 m LeetCode · 20 m interview Q	10 queries

Day	Topic	Time breakdown	Output
4	GROUP BY mastery	30 m concepts · 75 m practice · 30 m LeetCode · 15 m quiz	13 queries
5	JOINs	60 m concepts · 55 m practice · 35 m LeetCode · 20 m interview Q	10 queries
6	Advanced JOINs	35 m concepts · 80 m practice · 35 m LeetCode	13 queries
7	Subqueries	50 m concepts · 60 m practice · 30 m LeetCode · 20 m interview Q	10 queries
8	CTEs	40 m concepts · 60 m practice · 30 m LeetCode · 20 m interview Q	10 queries
9	Window Functions I	60 m concepts · 60 m practice · 35 m LeetCode	10 queries
10	Window Functions II	45 m concepts · 65 m practice · 35 m LeetCode · 15 m interview Q	10 queries
11	Interview Patterns	45 m patterns · 80 m practice · 30 m LeetCode	14 queries
12	PostgreSQL Specifics	50 m concepts · 60 m practice · 25 m revision	10 queries
13	DBMS Theory	90 m theory · 45 m Q&A drill · 20 m self-test	spoken answers
14	SQL 50 Intensive	150 m problem solving · 30 m review of misses	15–20 problems
15	Mock Interview	120 m timed assessment · 45 m grading & diagnosis	score & weak-area plan

1.31 The 15-day roadmap at a glance

Day 1 SELECT, FROM, WHERE, NULL — the shape of a query

Day 2 ORDER BY, LIMIT, IN, BETWEEN, LIKE, CASE — shaping the output

Day 3 COUNT/SUM/AVG, GROUP BY, HAVING — collapsing rows

Day 4 GROUP BY mastery — multi-column, expressions, ordering of clauses

Day 5 INNER / LEFT / RIGHT / FULL / CROSS / self join — combining tables

Day 6 3+ tables, anti-joins, duplicates, accidental Cartesian products

Day 7 Subqueries: scalar, IN, EXISTS, correlated, derived tables

Day 8 CTEs — WITH, chaining, readability, recursive (intro)

Day 9 Window functions: OVER, PARTITION BY, ROW_NUMBER / RANK / DENSE_RANK

Day 10 LAG / LEAD, running totals, moving averages, Top-N per group, frames

Day 11 The 17 interview patterns, as reusable templates

Day 12 PostgreSQL: types, dates, strings, COALESCE, casting, STRING_AGG

Day 13 DBMS theory: keys, constraints, normalization, indexes, ACID

Day 14 LeetCode SQL 50 intensive, organised by concept

Day 15 Full mock placement interview + scoring rubric + diagnosis

1.32 If you fall behind

Situation	What to do
Only 1 hour today	Do Concepts + Worked Examples + 3 practice problems. Skip LeetCode; catch up at the weekend.
Stuck on a day	Do not skip forward. Days 5, 9 and 11 depend heavily on everything before them. Re-do the Worked Examples by typing them, not reading them.
Two weeks is too fast	Stretch to 20 days by splitting Days 5, 6, 9, 10 across two sessions each. The order matters far more than the pace.
Interview tomorrow	Read Chapter 18 (patterns), Chapter 19 (the 50 questions) and Chapter 20 (cheat sheets). That is a legitimate 3-hour crash plan.

Interview Insight

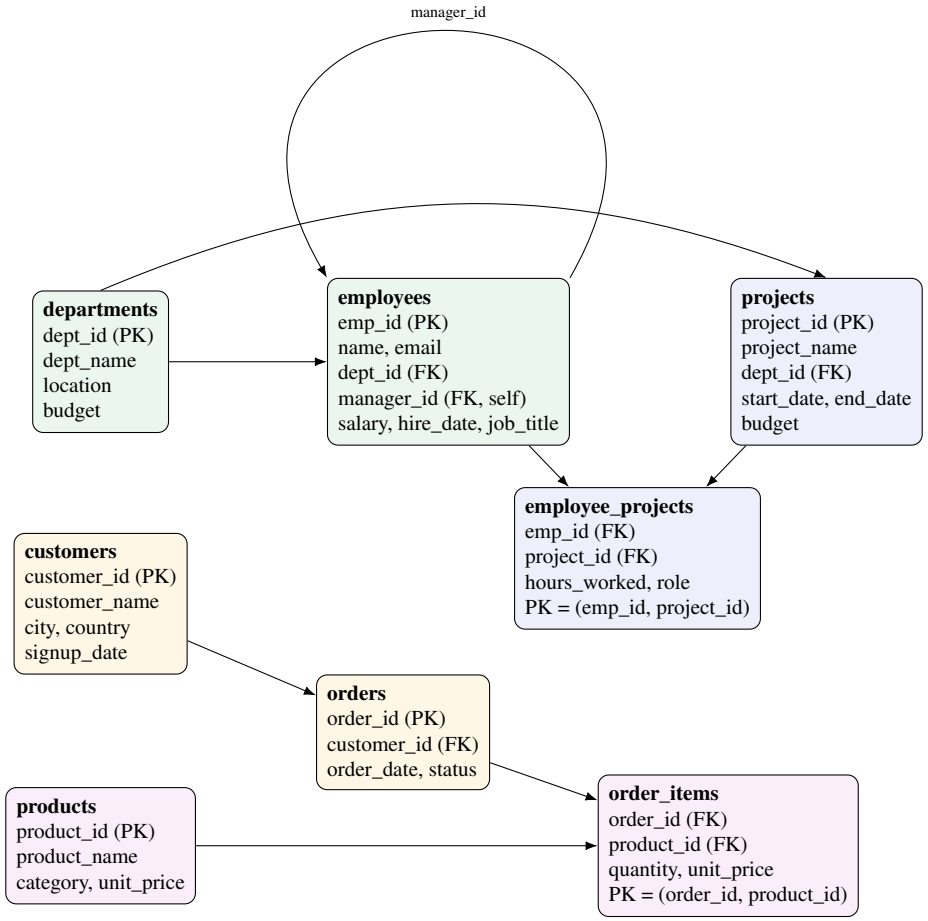
On honesty in interviews. If you have done this workbook properly, the true statement is: "I have written a few hundred queries against a PostgreSQL database, and I have completed the LeetCode SQL 50." That is a strong, specific, verifiable claim. Do not inflate it into "I have production database experience" — interviewers probe, and a specific modest claim beats a vague large one every time.

Chapter 2

The Practice Database

Everything in this workbook uses one database. Familiarity matters: by Day 9 you should know the column names without looking, so that your attention goes to the *logic* of the query rather than to remembering whether it is `dept_id` or `department_id`.

2.1 The schema



Mental Model

Read the arrows as "points at". A foreign key is a *pointer to another table's primary key*. `employees.dept_id` points at `departments.dept_id`, which is why joining them is natural and why you can never insert an employee into a department that does not exist.

Two tables here are **junction tables** (`employee_projects`, `order_items`): they exist because the relationship is many-to-many. An employee works on many projects; a project has many employees. You cannot express that with a single foreign key column, so you create a table whose whole job is to hold pairs.

2.2 Setup script — Part 1: tables

```sql
DROP TABLE IF EXISTS order_items, orders, products, customers,
                     employee_projects, projects, employees, departments CASCADE;

CREATE TABLE departments (
    dept_id     SERIAL PRIMARY KEY,
    dept_name   VARCHAR(50) NOT NULL UNIQUE,
    location    VARCHAR(50),
    budget      NUMERIC(12,2) CHECK (budget >= 0)
);

CREATE TABLE employees (
    emp_id      SERIAL PRIMARY KEY,
    name        VARCHAR(80) NOT NULL,
    email       VARCHAR(120) UNIQUE,
    dept_id     INT REFERENCES departments(dept_id),
    manager_id  INT REFERENCES employees(emp_id),   -- self-referencing FK
    salary      NUMERIC(10,2) CHECK (salary > 0),
    hire_date   DATE NOT NULL,
    job_title   VARCHAR(60)
);

CREATE TABLE projects (
    project_id   SERIAL PRIMARY KEY,
    project_name VARCHAR(80) NOT NULL,
    dept_id      INT REFERENCES departments(dept_id),
    start_date   DATE,
    end_date     DATE,
    budget       NUMERIC(12,2),
    CHECK (end_date IS NULL OR end_date >= start_date)
);

CREATE TABLE employee_projects (
    emp_id       INT REFERENCES employees(emp_id),
    project_id   INT REFERENCES projects(project_id),
    hours_worked INT DEFAULT 0 CHECK (hours_worked >= 0),
    role         VARCHAR(40),
    PRIMARY KEY (emp_id, project_id)                -- composite key
);

CREATE TABLE customers (
    customer_id   SERIAL PRIMARY KEY,
    customer_name VARCHAR(80) NOT NULL,
    city          VARCHAR(50),
    country       VARCHAR(50),
    signup_date   DATE
);

CREATE TABLE products (
    product_id   SERIAL PRIMARY KEY,
    product_name VARCHAR(80) NOT NULL,
    category     VARCHAR(40),
    unit_price   NUMERIC(10,2) CHECK (unit_price >= 0)
);

CREATE TABLE orders (
    order_id    SERIAL PRIMARY KEY,
    customer_id INT REFERENCES customers(customer_id),
    order_date  DATE NOT NULL,
    status      VARCHAR(20) DEFAULT 'PLACED'
                CHECK (status IN ('PLACED','SHIPPED','DELIVERED','CANCELLED'))
);

CREATE TABLE order_items (
```

```
    order_id   INT REFERENCES orders(order_id),
    product_id INT REFERENCES products(product_id),
    quantity   INT NOT NULL CHECK (quantity > 0),
    unit_price NUMERIC(10,2) NOT NULL,
    PRIMARY KEY (order_id, product_id)
);
```

Interview Insight

This script is itself an interview artefact. It demonstrates: `SERIAL` (auto-increment), `PRIMARY KEY`, `UNIQUE`, `NOT NULL`, `CHECK`, `REFERENCES` (foreign keys), a *self-referencing* foreign key, two *composite* primary keys, and a table-level `CHECK` spanning two columns. Day 13 explains each of these; being able to point at the line that does it is worth a lot.

2.3 Setup script — Part 2: data

departments (5 rows)

```
INSERT INTO departments (dept_name, location, budget) VALUES
('Engineering', 'Bangalore', 5000000),
('Sales',       'Mumbai',    3000000),
('HR',          'Bangalore',  800000),
('Marketing',   'Delhi',     1500000),
('Research',    'Hyderabad', 2500000);
```

employees (20 rows)

Note the deliberate imperfections: one employee has a `NULL` department, two have `NULL` manager (they are the top of the hierarchy), and one has a `NULL` email. These NULLs are not sloppiness — they are what makes Day 1 and Day 5 interesting.

```
INSERT INTO employees
  (name, email, dept_id, manager_id, salary, hire_date, job_title) VALUES
('Asha Nair',      'asha@corp.com',    1, NULL, 180000, '2015-03-01', 'CTO'),
('Rohit Verma',    'rohit@corp.com',   1,    1, 120000, '2016-07-15', 'Engineering
    Manager'),
('Priya Sharma',   'priya@corp.com',   1,    2,  95000, '2018-01-10', 'Senior Engineer
    '),
('Karan Mehta',    'karan@corp.com',   1,    2,  88000, '2019-05-20', 'Senior Engineer
    '),
('Sneha Iyer',     'sneha@corp.com',   1,    2,  72000, '2021-02-01', 'Engineer'),
('Aditya Rao',     'aditya@corp.com',  1,    2,  68000, '2021-08-11', 'Engineer'),
('Meera Joshi',    'meera@corp.com',   1,    2,  61000, '2022-11-05', 'Junior Engineer
    '),
('Vikram Singh',   'vikram@corp.com',  2, NULL, 150000, '2014-06-01', 'VP Sales'),
('Neha Gupta',     'neha@corp.com',    2,    8,  92000, '2017-09-12', 'Sales Manager')
    ,
('Arjun Desai',    'arjun@corp.com',   2,    9,  67000, '2020-03-23', 'Sales Executive
    '),
('Divya Menon',    'divya@corp.com',   2,    9,  67000, '2020-04-01', 'Sales Executive
    '),
('Sameer Khan',    'sameer@corp.com',  2,    9,  54000, '2022-01-17', 'Sales Associate
    '),
('Ananya Bose',    'ananya@corp.com',  3,    1,  85000, '2017-02-20', 'HR Manager'),
('Rahul Pillai',   'rahul@corp.com',   3,   13,  52000, '2021-06-30', 'HR Executive'),
('Ishita Kapoor',  'ishita@corp.com',  4,    1,  98000, '2016-11-11', 'Marketing Head'
    ),
('Nikhil Jain',    'nikhil@corp.com',  4,   15,  63000, '2020-08-08', 'Marketing
    Analyst'),
```

```
('Farah Sheikh',   'farah@corp.com',   4,   15,  59000, '2022-05-19', 'Content
    Strategist'),
('Deepak Reddy',   'deepak@corp.com',  5,    1, 110000, '2018-04-02', 'Principal
    Scientist'),
('Tanvi Shah',     'tanvi@corp.com',   5,   18,  79000, '2021-09-27', 'Research
    Scientist'),
('Omar Ali',       NULL,            NULL, NULL,  57000, '2023-01-09', 'Contractor');
```

projects (6 rows)

```
INSERT INTO projects (project_name, dept_id, start_date, end_date, budget) VALUES
('Payments Platform', 1, '2022-01-01', '2023-06-30', 1200000),
('Mobile App Revamp', 1, '2023-02-15', NULL,          900000),
('CRM Rollout',       2, '2022-05-01', '2023-01-31',  400000),
('Employee Portal',   3, '2023-03-01', NULL,          150000),
('Brand Campaign',    4, '2023-01-10', '2023-09-30',  600000),
('LLM Prototype',     5, '2023-06-01', NULL,          750000);
```

employee_projects (18 rows)

```
INSERT INTO employee_projects (emp_id, project_id, hours_worked, role) VALUES
(2,1,320,'Lead'),   (3,1,480,'Developer'), (4,1,410,'Developer'),
(5,1,260,'Developer'),(3,2,300,'Lead'),    (5,2,340,'Developer'),
(6,2,290,'Developer'),(7,2,180,'Intern'),
(9,3,220,'Lead'),   (10,3,260,'Analyst'),  (11,3,240,'Analyst'),
(13,4,150,'Lead'),  (14,4,200,'Coordinator'),
(15,5,180,'Lead'),  (16,5,320,'Analyst'),  (17,5,280,'Writer'),
(18,6,400,'Lead'),  (19,6,360,'Researcher');
```

customers (10 rows)

```
INSERT INTO customers (customer_name, city, country, signup_date) VALUES
('Zenith Retail',    'Mumbai',    'India',     '2021-01-15'),
('Apex Traders',     'Delhi',     'India',     '2021-06-02'),
('Nova Systems',     'Bangalore', 'India',     '2022-03-19'),
('Orion Logistics',  'Chennai',   'India',     '2022-07-25'),
('Vertex Foods',     'Pune',      'India',     '2022-11-30'),
('Helios Media',     'Singapore', 'Singapore', '2023-01-08'),
('Quantum Labs',     'Dubai',     'UAE',       '2023-02-14'),
('Summit Apparel',   'Kolkata',   'India',     '2023-05-21'),
('Lyra Interiors',   'Hyderabad', 'India',     '2023-08-03'),
('Cobalt Motors',    'London',    'UK',        '2023-09-17');
```

Common Mistake

`Lyra Interiors` and `Cobalt Motors` deliberately have **no orders**. Every "customers with no orders" problem in this book depends on them. If you ever get an empty result for an anti-join exercise, check that you inserted all ten customers.

products (10 rows)

```
INSERT INTO products (product_name, category, unit_price) VALUES
('Laptop Pro 14',   'Electronics', 125000),
('Wireless Mouse',  'Electronics',   1500),
('Standing Desk',   'Furniture',    28000),
('Ergonomic Chair', 'Furniture',    18000),
```

```
('Monitor 27in',    'Electronics',   22000),
('Notebook Pack',   'Stationery',      450),
('Whiteboard',      'Stationery',     3200),
('Server Rack',     'Hardware',      95000),
('Network Switch',  'Hardware',      42000),
('Desk Lamp',       'Furniture',      2400);
```

orders (15 rows)

```
INSERT INTO orders (customer_id, order_date, status) VALUES
(1,'2023-01-12','DELIVERED'), (1,'2023-04-05','DELIVERED'),
(2,'2023-02-20','SHIPPED'),   (3,'2023-03-11','DELIVERED'),
(3,'2023-06-18','CANCELLED'), (3,'2023-09-02','PLACED'),
(4,'2023-04-28','DELIVERED'), (5,'2023-05-09','SHIPPED'),
(5,'2023-08-14','DELIVERED'), (6,'2023-06-30','DELIVERED'),
(7,'2023-07-07','PLACED'),    (8,'2023-08-22','DELIVERED'),
(1,'2023-10-01','PLACED'),    (2,'2023-10-15','SHIPPED'),
(4,'2023-11-03','DELIVERED');
```

order_items (30 rows)

```
INSERT INTO order_items (order_id, product_id, quantity, unit_price) VALUES
(1,1,2,125000),(1,2,4,1500),(1,5,2,22000),
(2,3,1,28000),(2,10,2,2400),
(3,1,1,125000),(3,4,3,18000),
(4,5,4,22000),(4,2,10,1500),(4,6,20,450),
(5,8,1,95000),
(6,9,2,42000),(6,7,3,3200),
(7,4,5,18000),(7,3,2,28000),
(8,1,1,125000),(8,5,1,22000),
(9,6,50,450),(9,2,6,1500),
(10,8,2,95000),(10,9,1,42000),
(11,7,4,3200),(11,10,3,2400),
(12,1,3,125000),(12,4,2,18000),(12,2,5,1500),
(13,5,2,22000),
(14,3,1,28000),(14,10,1,2400),
(15,9,3,42000);
```

2.4 Verify your setup

Run these three checks before Day 1. If any number is wrong, re-run the script.

```
SELECT COUNT(*) FROM employees;       -- expect 20
SELECT COUNT(*) FROM order_items;     -- expect 30
SELECT COUNT(*) FROM employees WHERE dept_id IS NULL;  -- expect 1
```

2.5 Smaller teaching tables

Some concepts are clearer on four rows than on twenty. When a chapter needs one, it will build it inline like this:

```
-- a tiny table used to demonstrate NULL behaviour
CREATE TABLE demo_scores (name TEXT, score INT);
INSERT INTO demo_scores VALUES
('A', 10), ('B', NULL), ('C', 30), ('D', NULL);
```

You can create and drop these freely — they are never referenced by the main schema.

Mental Model

A habit worth forming today. Before writing any query, say out loud: *"Which tables hold the data I need? What connects them? What is one row of my answer?"*
That last question is the one beginners skip, and it is the most useful. If the answer is "one row per employee", your query will look different than if the answer is "one row per department". Deciding the *grain* of the result first prevents most SQL mistakes.

Chapter 3

Day 1 — SQL Fundamentals

Learning Objectives

By the end of today you can:

- explain what a relational database is, and what a table, row and column are;
- write `SELECT ... FROM ... WHERE ...` from scratch;
- combine conditions with `AND`, `OR`, `NOT`;
- use `DISTINCT` correctly and say what it actually does;
- explain why `= NULL` never works, and use `IS NULL` / `IS NOT NULL`;
- answer the interview question "what is NULL in SQL?" in two sentences.

3.1 Concepts

3.1.1 What is a relational database?

A relational database stores data in **tables**. A table is a grid:

emp_id	name	salary
1	Asha Nair	180000
2	Rohit Verma	120000
3	Priya Sharma	95000

← a **row** = one employee

↑ a **column** = one attribute, one data type

Three properties matter:

- **Every column has a fixed type.** `salary` holds numbers; you cannot put the word "high" in it. The database enforces this.
- **Rows have no inherent order.** A table is a *set* of rows. If you want an order you must ask for one with `ORDER BY`. This surprises beginners constantly.
- **Tables relate to each other** through shared values — that is the "relational" part, and it is what Day 5 is about.

3.1.2 SQL vs PostgreSQL

SQL	A *language standard* (ANSI SQL). It defines `SELECT`, `JOIN`, `GROUP BY` and so on. Nobody implements it perfectly.
PostgreSQL	A *database system* that implements SQL, plus its own extensions. Also: MySQL, SQL Server, Oracle, SQLite.
Why it matters	About 90 percent of what you learn transfers between systems. The rest — string functions, date arithmetic, `LIMIT` vs `TOP` — differs, and interviewers usually do not care which dialect you use as long as you name it.

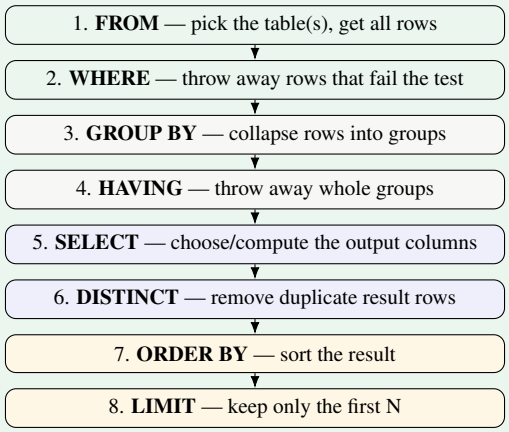
Interview Insight

Good answer to "do you know SQL or PostgreSQL?": *"SQL is the language; PostgreSQL is the engine I have practised on. The core query language is the same everywhere — the differences are mostly built-in functions and some syntax like `LIMIT` versus `TOP`."*

3.2 Mental Model: the shape of a query

Mental Model

A query has a physical writing order and a *logical execution order*, and they are not the same. Today you only need the first two steps, but learn the whole picture now — it explains most of the confusing errors you will hit on Days 3 and 4.



You *write* `SELECT` first. The database *runs* it fifth. That single fact explains why you cannot use a `SELECT` alias in `WHERE`, and why `WHERE` cannot see aggregate results.

3.3 Syntax

```sql
SELECT column1, column2          -- which columns you want
FROM   table_name                -- where they live
WHERE  condition;                -- which rows qualify
```

`SELECT *` means "every column". Convenient when exploring, poor style in real code (and in interviews) because the result silently changes if someone adds a column.

3.3.1 Comparison operators

=	equal	<> or !=	not equal
<	less than	<=	less than or equal
>	greater than	>=	greater than or equal

PostgreSQL accepts both <> and !=; <> is the standard one, and using it is a small signal of care.

3.3.2 Logical operators

`AND` (both must be true), `OR` (at least one), `NOT` (invert). Precedence: `NOT` binds tightest, then `AND`, then `OR`. When in doubt use parentheses — and in an interview use them even when not in doubt.

3.4 Worked Examples

Example 1 — the simplest useful query

```
SELECT name, salary
FROM   employees
WHERE  salary > 100000;
```

```
name          | salary
--------------+---------
Asha Nair     | 180000
Rohit Verma   | 120000
Vikram Singh  | 150000
Deepak Reddy  | 110000
```

Trace it in logical order: `FROM employees` produces 20 rows; `WHERE` keeps the 4 that pass; `SELECT` narrows 8 columns down to 2.

Example 2 — AND vs OR, and why parentheses matter

```
SELECT name, salary
FROM   employees
WHERE  dept_id = 1 AND salary > 80000;
```

```
name         | salary
-------------+---------
Asha Nair    | 180000
Rohit Verma  | 120000
Priya Sharma |  95000
Karan Mehta  |  88000
```

Now compare these two, which look similar and mean different things:

```
-- (A) dept 1 or 2, AND earning over 90k
SELECT name FROM employees
WHERE (dept_id = 1 OR dept_id = 2) AND salary > 90000;

-- (B) dept 1, OR anyone in dept 2 earning over 90k
SELECT name FROM employees
WHERE dept_id = 1 OR dept_id = 2 AND salary > 90000;
```

Query (B) is parsed as `dept_id = 1 OR (dept_id = 2 AND salary > 90000)` because `AND` binds tighter. It returns all seven engineers plus two salespeople — nine rows instead of four.

Example 3 — DISTINCT

```
SELECT DISTINCT job_title
FROM   employees
WHERE  dept_id = 1;
```

```
job_title
--------------------
CTO
Engineering Manager
Senior Engineer          <- appeared twice in the table, once here
Engineer
Junior Engineer
```

Mental Model

`DISTINCT` deduplicates the *entire output row*, not one column. `SELECT DISTINCT dept_id, job_title` gives distinct *pairs* — so a job title can still appear several times if it exists in several departments. This is the number one `DISTINCT` misunderstanding.

Example 4 — NULL

`NULL` does not mean zero and does not mean empty string. It means **unknown**.

```
SELECT name, email FROM employees WHERE email = NULL;      -- returns 0 rows!
SELECT name, email FROM employees WHERE email IS NULL;     -- correct
```

```
name       | email
-----------+-------
Omar Ali   | (null)
```

Why does `= NULL` fail? Because comparing anything to an unknown value produces `UNKNOWN`, not `TRUE`. And `WHERE` keeps only rows where the condition is `TRUE`. So the row is discarded.

Expression	Result	Row kept by WHERE?
`5 = 5`	TRUE	yes
`5 = 3`	FALSE	no
`5 = NULL`	UNKNOWN	no
`NULL = NULL`	UNKNOWN	no
`NULL IS NULL`	TRUE	yes
`NOT (5 = NULL)`	UNKNOWN	no

Interview Insight

Two-sentence answer to "what is NULL?": *"NULL represents an unknown or missing value, not zero and not an empty string. Because any comparison with an unknown yields UNKNOWN rather than TRUE, you test it with `IS NULL` or `IS NOT NULL` rather than =."* This is asked in a very large fraction of fresher interviews.

3.5 Common Mistakes

Common Mistake

1. `WHERE email = NULL` — silently returns zero rows. Use `IS NULL`.
2. Double quotes for string literals. `WHERE name = "Asha Nair"` fails in PostgreSQL with *column "Asha Nair" does not exist*, because double quotes mean *identifier*. Strings use single quotes: `'Asha Nair'`.
3. Forgetting the semicolon — psql just waits, showing a `->` prompt. It is not frozen; it wants a `;`.
4. `NOT` placed wrongly. `WHERE NOT salary > 90000` works, but `WHERE salary NOT > 90000` is a syntax error. Prefer `<=` where you can.
5. Assuming row order. Without `ORDER BY`, PostgreSQL may return rows in any order, and that order can change after updates.

3.6 Guided Practice

Practice

Hints are given. Write the query, run it, then check the Answer Key.

D1-P1 ● Select every column of the `departments` table. *Hint: two keywords.*

D1-P2 ● Show only the `name` and `job_title` of all employees.

D1-P3 ● List employees hired before 2018. *Hint: dates compare like numbers; write the literal as* `'2018-01-01'`.

D1-P4 ● List employees whose salary is between 60000 and 90000, using only `>=` and `<=`.

D1-P5 ● List the distinct `location` values in `departments`.

3.7 Independent Practice

Practice

D1-P6 ● Find all employees in department 2 who earn more than 60000.

D1-P7 ● Find all employees who are *not* in department 1 — and make sure Omar Ali (whose `dept_id` is NULL) appears. Explain in one line why the obvious query misses him.

D1-P8 ● List the names of employees who have no manager.

D1-P9 ● List distinct `category` values from `products` where `unit_price` is above 20000.

D1-P10 ● Find employees who earn more than 90000 *or* were hired before 2016, but who are not in department 4. Use parentheses deliberately and be ready to explain where they go.

3.8 LeetCode Practice

Problem	Name	Diff.	Concept
1757	Recyclable and Low Fat Products	● Easy	`WHERE` with `AND`
584	Find Customer Referee	● Easy	**NULL handling** — the classic trap
595	Big Countries	● Easy	`WHERE` with `OR`
1148	Article Views I	● Easy	`DISTINCT` + `ORDER BY`

Interview Insight

Do 584 slowly. It asks for customers whose referee id is not 2. The naive `WHERE referee_id <> 2` misses everyone with a NULL referee. Getting this wrong once, today, is worth more than reading about NULL ten times.

3.9 Interview Questions

1. What is the difference between SQL and PostgreSQL?
2. What is NULL? How do you test for it, and why not with `=`?
3. Does `DISTINCT` operate on a column or on the whole row?
4. Is `SELECT *` good practice? When is it acceptable?
5. In what order does the database logically process `SELECT`, `FROM` and `WHERE`?

Daily Quiz

1. What does `WHERE salary <> NULL` return?
2. How many rows does `SELECT DISTINCT dept_id, job_title FROM employees` return for department 1?
3. Which runs first logically: `WHERE` or `SELECT`?
4. Are `'abc'` and `"abc"` the same thing in PostgreSQL?
5. True or false: a table guarantees the order of its rows.
6. Write the condition "dept is 1 or 3, and salary at least 70000".
7. What is the result of `NULL = NULL`?

Daily Revision Checklist

☐ I can write `SELECT / FROM / WHERE` without looking anything up
☐ I know the logical processing order (FROM → WHERE → SELECT)
☐ I use single quotes for strings
☐ I never write `= NULL`
☐ I can explain `DISTINCT` on multiple columns
☐ I completed 4 LeetCode problems

Chapter 4

Day 2 — Filtering, Sorting and Conditional Logic

Learning Objectives

By the end of today you can:

- sort results with `ORDER BY`, including multi-column and NULL placement;
- page through results with `LIMIT` and `OFFSET`;
- write compact filters with `IN` and `BETWEEN`;
- pattern-match text with `LIKE` and PostgreSQL's `ILIKE`;
- build conditional columns with `CASE WHEN`;
- explain why an alias is unavailable in `WHERE` but available in `ORDER BY`.

4.1 Concepts and Syntax

4.1.1 ORDER BY

```
SELECT name, salary
FROM   employees
ORDER  BY salary DESC;          -- ASC is the default
```

Sort by several columns — the second breaks ties in the first:

```
SELECT name, dept_id, salary
FROM   employees
ORDER  BY dept_id ASC, salary DESC;
```

This reads: group the display by department, and within each department show the highest-paid first.

4.1.2 NULLs in ORDER BY

PostgreSQL puts NULLs **last** in `ASC` and **first** in `DESC` by default. You can override this:

```
SELECT name, manager_id
FROM   employees
ORDER  BY manager_id ASC NULLS FIRST;
```

4.1.3 LIMIT and OFFSET

```
SELECT name, salary FROM employees
ORDER BY salary DESC
LIMIT 5;                        -- top 5

SELECT name, salary FROM employees
ORDER BY salary DESC
LIMIT 5 OFFSET 5;               -- rows 6-10: "page 2"
```

Common Mistake

`LIMIT` without `ORDER BY` is meaningless — "any 5 rows" is not a specification, and the answer can change between runs. In an interview, always pair them.
Also: `LIMIT` is PostgreSQL/MySQL syntax. SQL Server uses `TOP`; Oracle uses `FETCH FIRST n ROWS ONLY`. Mentioning this shows dialect awareness.

4.1.4 IN and BETWEEN

```
-- equivalent
WHERE dept_id = 1 OR dept_id = 3 OR dept_id = 5
WHERE dept_id IN (1, 3, 5)

-- equivalent (BETWEEN is INCLUSIVE on both ends)
WHERE salary >= 60000 AND salary <= 90000
WHERE salary BETWEEN 60000 AND 90000
```

Common Mistake

`BETWEEN` is inclusive — it includes both endpoints. Interviewers ask this constantly.
`NOT IN` with NULLs is a trap. If the list contains a NULL, then `x NOT IN (1, 2, NULL)` is never TRUE for any `x`, so you get zero rows. This bites hard when the list comes from a subquery (Day 7). `NOT EXISTS` is the safe alternative.

4.1.5 LIKE and pattern matching

Wildcard	Means	Example
percent sign	any sequence of characters, including none	`'A%'` matches `'Asha'`, `'Arjun'`
underscore	exactly one character	`'_riya'` matches `'Priya'`

```
SELECT name FROM employees WHERE name LIKE 'A%';          -- starts with A
SELECT name FROM employees WHERE name LIKE '%Sharma';     -- ends with Sharma
SELECT name FROM employees WHERE name LIKE '%a%';         -- contains an 'a'
SELECT name FROM employees WHERE name ILIKE 'a%';         -- PostgreSQL: case-
    insensitive
```

Interview Insight

`ILIKE` is PostgreSQL-specific and genuinely useful. The portable equivalent is `WHERE LOWER(name) LIKE 'a%'`. Knowing both — and knowing that wrapping a column in a function usually prevents the database from using an index on it (Day 13) — is a strong answer.

4.1.6 CASE WHEN

`CASE` is SQL's if/else. It produces a *value*, so it lives in `SELECT`, `ORDER BY`, and (from Day 4) inside aggregates.

```
SELECT name,
       salary,
       CASE
           WHEN salary >= 120000 THEN 'Executive'
           WHEN salary >=  80000 THEN 'Senior'
           WHEN salary >=  60000 THEN 'Mid'
           ELSE 'Junior'
       END AS salary_band
FROM employees
ORDER BY salary DESC;
```

```
name          | salary | salary_band
--------------+--------+------------
Asha Nair     | 180000 | Executive
Vikram Singh  | 150000 | Executive
Rohit Verma   | 120000 | Executive
Deepak Reddy  | 110000 | Senior
Ishita Kapoor |  98000 | Senior
...
Rahul Pillai  |  52000 | Junior
```

Mental Model

`CASE` evaluates its `WHEN` branches **top to bottom and stops at the first match**. That is why the bands above are written from highest to lowest — reverse the order and everyone above 60000 would be labelled "Mid". Order is part of the logic, not cosmetic.
If no branch matches and there is no `ELSE`, the result is `NULL`.

4.1.7 Aliases

```sql
SELECT e.name        AS employee_name,
       e.salary / 12 AS monthly_salary
FROM   employees AS e
WHERE  e.salary > 90000;
```

`AS` is optional for both columns and tables, but writing it is clearer. Table aliases become essential from Day 5 onward.

Common Mistake

You cannot use a `SELECT` alias in `WHERE`.

```sql
-- FAILS: column "monthly_salary" does not exist
SELECT salary / 12 AS monthly_salary
FROM employees
WHERE monthly_salary > 8000;
```

Why? Logical processing order: `WHERE` runs at step 2, `SELECT` at step 5. When `WHERE` runs, the alias does not exist yet. Repeat the expression, or wrap the query in a subquery (Day 7).
You *can* use the alias in `ORDER BY`, because `ORDER BY` runs *after* `SELECT`. This asymmetry is a favourite interview question.

4.2 Worked Example — combining everything

"Show the three most recently hired employees in Engineering or Sales, with a tenure label."

```sql
SELECT name,
       hire_date,
       CASE WHEN hire_date >= '2021-01-01' THEN 'New'
            ELSE 'Experienced'
       END AS tenure
FROM   employees
WHERE  dept_id IN (1, 2)
ORDER  BY hire_date DESC
LIMIT  3;
```

```
name         | hire_date  | tenure
-------------+------------+--------
Meera Joshi  | 2022-11-05 | New
```

```
Sameer Khan   | 2022-01-17 | New
Aditya Rao    | 2021-08-11 | New
```

Follow the pipeline: `FROM` gives 20 rows → `WHERE` keeps 12 → `SELECT` builds three columns → `ORDER BY` sorts → `LIMIT` keeps 3.

4.3 Guided Practice

Practice

D2-P1 ● List all employees sorted by salary, highest first.

D2-P2 ● List the five cheapest products. *Hint:* `ORDER BY` *then* `LIMIT`.

D2-P3 ● Find all employees in departments 1, 3 or 5 using `IN`.

D2-P4 ● Find products priced between 2000 and 30000 inclusive.

D2-P5 ● Find employees whose name starts with `'A'`.

4.4 Independent Practice

Practice

D2-P6 ● Show employees ordered by department ascending, then salary descending.

D2-P7 ● Show orders 6 through 10 when sorted by `order_date` ascending.

D2-P8 ● Find all customers whose city contains the letter sequence `'ba'`, case-insensitively.

D2-P9 ● Add a column classifying products as `'Premium'` (above 50000), `'Standard'` (5000–50000) or `'Budget'` (below 5000). Sort by price descending.

D2-P10 ● List employees with a column `has_manager` showing `'Yes'` or `'No'`, sorted so that employees without a manager appear first, then by salary descending.

4.5 LeetCode Practice

Problem	Name	Diff.	Concept
1683	Invalid Tweets	● Easy	`WHERE` + a string function
1873	Calculate Special Bonus	● Easy	**`CASE WHEN`** + string functions
627	Swap Salary	● Easy	`CASE` inside `UPDATE`
1667	Fix Names in a Table	● Easy	string functions + concatenation
196	Delete Duplicate Emails	● Easy	`DELETE` with a self-referencing condition

4.6 Interview Questions

1. Is `BETWEEN` inclusive or exclusive?
2. Why can you use a column alias in `ORDER BY` but not in `WHERE`?
3. What is the difference between `LIKE` and `ILIKE`?
4. How would you implement pagination for a page showing 20 rows at a time?
5. Where do NULLs sort by default in PostgreSQL, and how do you change it?

Daily Quiz

1. Write "salary is 50000, 70000 or 90000" using `IN`.
2. Does `BETWEEN 10 AND 20` include 20?
3. What does the pattern `'_a%'` match?
4. What does a `CASE` return when nothing matches and there is no `ELSE`?
5. Which clause must come before `LIMIT` for the result to be deterministic?
6. Write the `OFFSET` for page 4 with 25 rows per page.
7. Why does `WHERE monthly_salary > 8000` fail when `monthly_salary` is a `SELECT` alias?

Daily Revision Checklist

☐ `ORDER BY` with multiple columns and directions
☐ `LIMIT`/`OFFSET` pagination
☐ `IN` and `BETWEEN` (inclusive!)
☐ `LIKE` wildcards, plus `ILIKE`
☐ `CASE WHEN` evaluates top-down and stops at the first match
☐ I can explain the alias-in-WHERE restriction using processing order

Chapter 5

Day 3 — Aggregate Functions, GROUP BY and HAVING

Learning Objectives

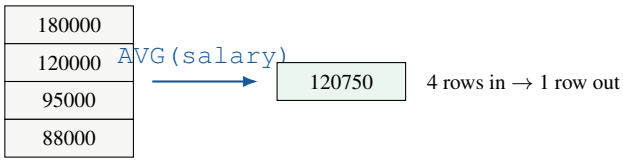
By the end of today you can:

- use `COUNT`, `SUM`, `AVG`, `MIN`, `MAX`;
- explain the difference between `COUNT(*)` and `COUNT(column)`;
- say exactly how NULLs affect each aggregate;
- write `GROUP BY` queries and predict their output shape;
- state the difference between `WHERE` and `HAVING` without hesitating.

5.1 Concepts

5.1.1 What an aggregate does

Every function so far worked on *one row at a time*. An aggregate function is different: it takes **many rows in and produces one value out**.



`COUNT(*)`	Counts **rows**. Never ignores anything. Always returns an integer, possibly 0.
`COUNT(col)`	Counts **non-NULL values** in that column.
`COUNT(DISTINCT col)`	Counts distinct non-NULL values.
`SUM(col)`	Adds non-NULL values. Returns NULL if every value is NULL (not 0!).
`AVG(col)`	`SUM(col) / COUNT(col)` — NULLs are excluded from both.
`MIN` / `MAX`	Smallest / largest non-NULL value. Works on numbers, dates and text.

5.1.2 How NULL affects aggregates — the demo that makes it stick

```sql
CREATE TABLE demo_scores (name TEXT, score INT);
INSERT INTO demo_scores VALUES ('A',10), ('B',NULL), ('C',30), ('D',NULL);

SELECT COUNT(*)     AS rows_total,
       COUNT(score) AS scores_present,
       SUM(score)   AS total,
       AVG(score)   AS average,
       MIN(score)   AS lowest,
       MAX(score)   AS highest
FROM demo_scores;
```

```
rows_total | scores_present | total | average | lowest | highest
-----------+----------------+-------+---------+--------+---------
         4 |              2 |    40 |      20 |     10 |      30
```

Mental Model

Read the two count columns side by side: **4 rows exist, but only 2 have a score.**
Now the crucial one: the average is **20**, not 10. SQL computed $40/2$, not $40/4$ — it divided by the number of *non-NULL* values. If you wanted NULLs treated as zero you must say so explicitly:

```
SELECT AVG(COALESCE(score, 0)) FROM demo_scores;    -- 10
```

Neither answer is "right" — they answer different questions ("average of the scores we have" vs "average score where a missing score counts as zero"). Knowing that SQL silently picks the first is what matters.

Interview Insight

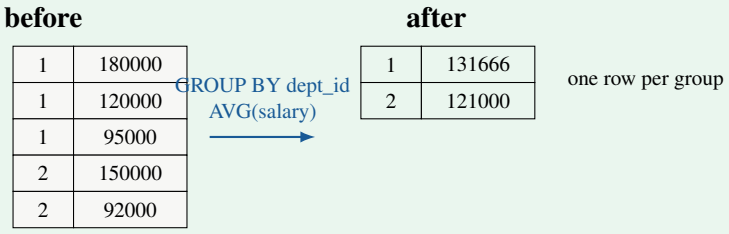
"What is the difference between `COUNT(*)` and `COUNT(column)`?" — asked in a huge share of fresher interviews. Answer: *"`COUNT(*)` counts rows; `COUNT(column)` counts rows where that column is not NULL. They differ exactly when the column contains NULLs."* Then offer the example above.

5.2 GROUP BY

5.2.1 The mental model

Mental Model

`GROUP BY` **collapses many rows into one row per group**. Picture it as sorting the rows into buckets, then running the aggregate separately inside each bucket.



The golden rule: after `GROUP BY`, every column in `SELECT` must either (a) appear in the `GROUP BY` list, or (b) be wrapped in an aggregate function. Anything else is ambiguous — if a department has seven employees, which of the seven names should appear in the single output row? PostgreSQL refuses to guess and raises an error.

5.2.2 Syntax and worked example

```
SELECT dept_id,
       COUNT(*)     AS headcount,
       AVG(salary)  AS avg_salary,
       MAX(salary)  AS top_salary
FROM   employees
GROUP  BY dept_id
ORDER  BY dept_id;
```

```
dept_id | headcount | avg_salary | top_salary
--------+-----------+------------+------------
      1 |         7 |   97714.29 |     180000
      2 |         5 |   86000.00 |     150000
      3 |         2 |   68500.00 |      85000
      4 |         3 |   73333.33 |      98000
      5 |         2 |   94500.00 |     110000
 (null) |         1 |   57000.00 |      57000     <- Omar Ali
```

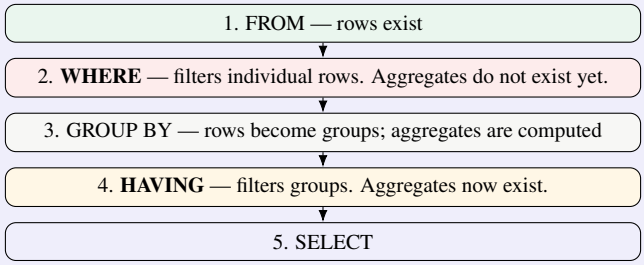
Common Mistake

Notice the NULL group. `GROUP BY` treats all NULLs as **one group** — which is inconsistent with `NULL = NULL` being UNKNOWN elsewhere, but it is the defined behaviour and interviewers like asking about it. If you do not want that row, filter it out with `WHERE dept_id IS NOT NULL`.

5.3 HAVING — and the difference from WHERE

Concept

`WHERE` filters rows. `HAVING` filters groups.
That one line is the answer, but you must be able to explain *why* it has to be that way. Look back at the processing order:



`WHERE` runs *before* the aggregates are computed, so it physically cannot see `COUNT(*)`. `HAVING` runs after, so it can. This is not an arbitrary rule — it falls out of the pipeline.

Side by side

```sql
-- WHERE: keep only high earners, THEN group them
SELECT dept_id, COUNT(*) AS senior_count
FROM   employees
WHERE  salary > 80000            -- filters ROWS, before grouping
GROUP  BY dept_id;
```

```
dept_id | senior_count
--------+--------------
      1 |            4
      2 |            2
      3 |            1
      4 |            1
      5 |            2
```

```sql
-- HAVING: group everyone, THEN keep only large departments
SELECT dept_id, COUNT(*) AS headcount
FROM   employees
GROUP  BY dept_id
HAVING COUNT(*) >= 3;            -- filters GROUPS, after grouping
```

```
dept_id | headcount
--------+-----------
      1 |         7
      2 |         5
      4 |         3
```

Both together — the pattern interviewers want

"For departments with more than 2 employees earning above 60000, show the average salary of those employees."

```
SELECT dept_id,
       COUNT(*)     AS qualifying_employees,
       ROUND(AVG(salary), 2) AS avg_salary
FROM   employees
WHERE  salary > 60000            -- 1. drop low earners (rows)
GROUP  BY dept_id                -- 2. bucket by department
HAVING COUNT(*) > 2              -- 3. drop small buckets (groups)
ORDER  BY avg_salary DESC;
```

```
dept_id | qualifying_employees | avg_salary
--------+----------------------+------------
      1 |                    6 |  102333.33
      2 |                    4 |   94000.00
```

Mental Model

Read that query as a sentence in *execution* order, not writing order:
"Take employees, keep those earning over 60000, bucket them by department, throw away buckets with 2 or fewer people, then report the count and average for each surviving bucket, sorted by average."
If you can narrate a query like that, you can debug it. If you can only read it top to bottom, you cannot.

5.4 Common Mistakes

Common Mistake

1. Aggregate in WHERE.

```
SELECT dept_id FROM employees
WHERE COUNT(*) > 3 GROUP BY dept_id;       -- ERROR: aggregate functions are not
    allowed in WHERE
```

Use `HAVING COUNT(*) > 3`.

2. Bare column in SELECT.

```
SELECT dept_id, name, AVG(salary) FROM employees GROUP BY dept_id;
-- ERROR: column "employees.name" must appear in the GROUP BY clause
--        or be used in an aggregate function
```

Which of the seven engineers' names should the single row show? Either group by `name` too, or aggregate it (`MAX(name)`), or — usually what you actually wanted — use a window function (Day 9).

3. Expecting `SUM` of all-NULLs to be 0. It returns NULL. Wrap it: `COALESCE(SUM(x), 0)`.

4. Using `COUNT(column)` when you meant rows. If the column has NULLs you will undercount, silently.

5. Filtering with HAVING when WHERE would do. `HAVING salary > 60000` on a non-grouped column may work in some engines but is wasteful and confusing — it filters after grouping instead of before. Filter early.

5.5 Guided Practice

Practice

D3-P1 🟢 Count the total number of employees.
D3-P2 🟢 Find the highest and lowest salary in the company.
D3-P3 🟢 Count how many employees have an email address. *Hint: this is different from counting rows.*
D3-P4 🟢 Find the average product price per category.
D3-P5 🟢 Count how many employees are in each department.

5.6 Independent Practice

Practice

D3-P6 🟢 Find the total salary bill per department, sorted highest first.
D3-P7 🟤 Count the number of distinct job titles in the company.
D3-P8 🟤 Find departments where the average salary exceeds 80000.
D3-P9 🟤 For each order status, count the orders and show the earliest order date.
D3-P10 🔴 For each product category, show the number of products and the average price — but only for categories with at least 2 products and an average price above 10000.

5.7 LeetCode Practice

Problem	Name	Diff.	Concept
620	Not Boring Movies	🟢 Easy	`WHERE` + modulo + `ORDER BY`
1741	Find Total Time Spent by Each Employee	🟢 Easy	`GROUP BY` + `SUM`
1211	Queries Quality and Percentage	🟤 Easy	`AVG` + conditional aggregation
1633	Percentage of Users Attended a Contest	🟤 Easy	`COUNT` + ratio + `ROUND`
596	Classes More Than 5 Students	🟤 Easy	**`GROUP BY`** + **`HAVING`** — the canonical one

5.8 Interview Questions

1. `WHERE` vs `HAVING` — explain with an example and with the processing order.
2. `COUNT(*)` vs `COUNT(col)` vs `COUNT(DISTINCT col)`.
3. How does `AVG` treat NULLs? How would you make NULLs count as zero?
4. Why does `SELECT dept_id, name, AVG(salary) ... GROUP BY dept_id` fail?
5. Can `HAVING` be used without `GROUP BY`? What happens?

Daily Quiz

1. `SUM` over a column that is entirely NULL returns what?
2. Which runs first: `WHERE` or `GROUP BY`?
3. Does `GROUP BY` put all NULLs in one group or one group each?
4. Write a filter for "departments with more than 5 employees".
5. `AVG` of `(10, NULL, 30)` is what?

6. Can you put an aggregate in `ORDER BY`?
7. How many rows does a `GROUP BY dept_id` query return on our data?

Daily Revision Checklist

☐ I can name all six aggregates and their NULL behaviour
☐ `COUNT(*)` vs `COUNT(col)` — I can give the example
☐ The golden rule of `GROUP BY` (group it or aggregate it)
☐ `WHERE` filters rows, `HAVING` filters groups — and I can explain why using the pipeline
☐ I completed 5 LeetCode problems

Chapter 6

Day 4 — GROUP BY Mastery

Learning Objectives

By the end of today you can:

- group by several columns and predict the number of output rows;
- group by an *expression*, not just a column;
- combine `WHERE`, `GROUP BY`, `HAVING` and `ORDER BY` correctly;
- use **conditional aggregation** — the single most useful intermediate SQL trick;
- trace a grouped query's intermediate results on paper.

6.1 Multi-column grouping

Concept

`GROUP BY a, b` makes one group for every *distinct combination* of `a` and `b` that actually occurs in the data. Not every possible combination — only the ones present.

```
SELECT dept_id, job_title, COUNT(*) AS n
FROM   employees
WHERE  dept_id IS NOT NULL
GROUP  BY dept_id, job_title
ORDER  BY dept_id, n DESC;
```

```
dept_id | job_title           | n
--------+---------------------+---
      1 | Senior Engineer     | 2
      1 | Engineer            | 2
      1 | CTO                 | 1
      1 | Engineering Manager | 1
      1 | Junior Engineer     | 1
      2 | Sales Executive     | 2
      2 | VP Sales            | 1
      ...
```

Mental Model

Predicting the row count. With `GROUP BY a`, you get one row per distinct `a`. With `GROUP BY a, b`, you get one row per distinct *pair* — always at least as many rows, usually more, and at most the number of rows in the table.

Adding a column to `GROUP BY` makes the result *finer grained*. Removing one makes it *coarser*. When a grouped query returns more rows than you expected, the first thing to check is whether you accidentally grouped by something unique — like an id.

6.2 Grouping by an expression

You can group by anything that produces a value, not only a bare column.

```
-- how many employees were hired each year?
SELECT EXTRACT(YEAR FROM hire_date) AS hire_year,
       COUNT(*) AS hires
```

```
FROM   employees
GROUP  BY EXTRACT(YEAR FROM hire_date)
ORDER  BY hire_year;
```

```
hire_year | hires
----------+-------
     2014 |     1
     2015 |     1
     2016 |     2
     2017 |     2
     2018 |     2
     2019 |     1
     2020 |     3
     2021 |     4
     2022 |     3
     2023 |     1
```

PostgreSQL also lets you group by the *output column position* or by the alias:

```
GROUP BY 1;            -- group by the first SELECT column; concise but fragile
GROUP BY hire_year;    -- PostgreSQL allows the alias here (unlike WHERE)
```

Interview Insight

Grouping by ordinal position (`GROUP BY 1`) works and is common in analytics code, but in an interview prefer the explicit expression or alias — it survives someone reordering the `SELECT` list. Being able to say *why* you avoid it is worth more than the keystrokes saved.

6.3 Conditional aggregation — learn this cold

Concept

Put a `CASE` *inside* an aggregate and you can compute several different filtered metrics in a single pass over the table. This is how you turn rows into a report.

```
SUM(CASE WHEN condition THEN 1 ELSE 0 END)      -- count rows meeting a condition
COUNT(CASE WHEN condition THEN 1 END)           -- same, using COUNT's NULL-
    skipping
AVG(CASE WHEN condition THEN value END)         -- average over a subset
```

PostgreSQL also has the cleaner `FILTER` clause, which does the same thing:

```
COUNT(*) FILTER (WHERE condition)
```

Worked example

"For each department, show the headcount, how many earn above 80000, how many below, and the average salary of only the high earners."

```
SELECT dept_id,
       COUNT(*)                                              AS headcount,
       SUM(CASE WHEN salary > 80000 THEN 1 ELSE 0 END)   AS high_earners,
       SUM(CASE WHEN salary <= 80000 THEN 1 ELSE 0 END)  AS others,
       ROUND(AVG(CASE WHEN salary > 80000 THEN salary END), 0) AS avg_high
FROM   employees
WHERE  dept_id IS NOT NULL
GROUP  BY dept_id
ORDER  BY dept_id;
```

```
dept_id | headcount | high_earners | others | avg_high
--------+-----------+--------------+--------+----------
      1 |         7 |            4 |      3 |   120750
      2 |         5 |            2 |      3 |   121000
      3 |         2 |            1 |      1 |    85000
      4 |         3 |            1 |      2 |    98000
      5 |         2 |            2 |      0 |    94500
```

Mental Model

Why does `AVG(CASE WHEN salary > 80000 THEN salary END)` work? Because the `CASE` has no `ELSE`, so rows that fail the condition produce NULL — and `AVG` ignores NULLs. You are exploiting Day 3's NULL rule deliberately.
Compare with `WHERE salary > 80000`, which would have removed those rows from *every* column, destroying the `headcount`. Conditional aggregation lets different columns have different filters. That is exactly why it exists.

6.4 Intermediate results — trace it on paper

Take this query and follow the data through each stage:

```
SELECT dept_id, COUNT(*) AS n, ROUND(AVG(salary)) AS avg_sal
FROM   employees
WHERE  hire_date < '2021-01-01'
GROUP  BY dept_id
HAVING COUNT(*) >= 2
ORDER  BY avg_sal DESC;
```

Stage 1 — after `FROM`: all 20 rows.
Stage 2 — after `WHERE hire_date < '2021-01-01'`: 12 rows remain (the eight hired in 2021 or later are gone, including Omar Ali).

```
dept_id: 1,1,1,1  2,2,2,2  3,3  4,4     <- the surviving rows' departments
```

Stage 3 — after `GROUP BY dept_id`: 4 groups.

```
dept 1 -> 4 rows -> avg 120750
dept 2 -> 4 rows -> avg 94000       (dept 5 had only 1 pre-2021 hire)
dept 3 -> 2 rows -> avg 68500
dept 4 -> 2 rows -> avg 80500
dept 5 -> 1 row  -> avg 110000
```

Stage 4 — after `HAVING COUNT(*) >= 2`: department 5 is dropped (only one qualifying employee).
Stage 5 — `SELECT` then `ORDER BY avg_sal DESC`:

```
dept_id | n | avg_sal
--------+---+---------
      1 | 4 |  120750
      2 | 4 |   94000
      4 | 2 |   80500
      3 | 2 |   68500
```

Mental Model

Notice how department 5 survived `WHERE` (Deepak Reddy was hired in 2018) but died at `HAVING`. A row-level filter and a group-level filter remove different things. Being able to say which stage removed a row is the whole skill of debugging grouped queries.

6.5 Common Mistakes

Common Mistake

1. Grouping by a unique column. `GROUP BY emp_id` gives one group per employee — 20 groups of 1 row. Technically valid, completely pointless. If your grouped query returns as many rows as the table, this is why.

2. Forgetting that `WHERE` shrinks what the aggregate sees. In the trace above, `AVG(salary)` for department 1 is 120750, not 97714 — because `WHERE` already removed the three newer hires. Aggregates only see surviving rows.

3. `HAVING` without `GROUP BY`. It is legal: the whole table becomes one group. `SELECT COUNT(*) FROM employees HAVING COUNT(*) > 100` returns zero rows. Rare, but a good interview curiosity.

4. Counting with `COUNT(CASE WHEN c THEN 1 ELSE 0 END)`. This counts *everything*, because 0 is not NULL and `COUNT` counts non-NULLs. Either use `SUM(CASE ... THEN 1 ELSE 0 END)` or `COUNT(CASE WHEN c THEN 1 END)` with no `ELSE`. Mixing the two is one of the most common silent bugs in interview answers.

5. Ordering by a column that is not grouped or aggregated — same error as in `SELECT`.

6.6 Guided Practice

Practice

D4-P1 ● Count employees per department per job title.

D4-P2 ● Count how many orders each customer placed.

D4-P3 ● Count how many employees were hired in each year.

D4-P4 ● For each product category, show count, min price and max price.

D4-P5 ● For each customer, count orders and count only the DELIVERED ones. *Hint: conditional aggregation.*

6.7 Independent Practice

Practice

D4-P6 ● For each department, show headcount and total salary, only for departments whose total salary exceeds 300000.

D4-P7 ● For each order, compute the order total from `order_items` (`quantity * unit_price` summed).

D4-P8 ● Count employees per department, splitting into those hired before 2020 and those hired later, in a single row per department.

D4-P9 ● For each city in `customers`, count customers — but only show cities with more than one customer.

D4-P10 ● For each department show the headcount, the number of employees earning above the value 90000, and the percentage of the department that represents, rounded to one decimal place.

D4-P11 ● For each project, show total hours worked and the number of distinct employees on it.

D4-P12 ● For each year in `orders`, show the number of orders and how many were cancelled.

D4-P13 ● Find job titles held by more than one employee, showing the title, the count, and the salary range (max minus min).

6.8 LeetCode Practice

Problem	Name	Diff.	Concept
1729	Find Followers Count	Easy	`GROUP BY` + `COUNT`
1050	Actors and Directors Who Cooperated	Easy	multi-column `GROUP BY` + `HAVING`
619	Biggest Single Number	Easy	`GROUP BY` + `HAVING COUNT = 1`
1070	Product Sales Analysis III	Medium	group-wise minimum
1075	Project Employees I	Easy	`GROUP BY` + `AVG` + `ROUND`

6.9 Interview Questions

1. How many rows does `GROUP BY a, b` produce, compared with `GROUP BY a`?
2. Explain conditional aggregation and give a use case.
3. Why does `COUNT(CASE WHEN x THEN 1 ELSE 0 END)` count everything?
4. Can you `GROUP BY` an expression? Give an example.
5. Walk me through what happens to the rows in a query with `WHERE`, `GROUP BY`, `HAVING` and `ORDER BY`.

Daily Quiz

1. Does adding a column to `GROUP BY` increase or decrease the row count?
2. Write conditional aggregation counting rows where `status = 'CANCELLED'`.
3. What is the PostgreSQL `FILTER` equivalent of that?
4. Which stage removes a row: `WHERE` or `HAVING`, for the condition "salary above 50000"?
5. Can `GROUP BY` use a `SELECT` alias in PostgreSQL?
6. What does `AVG(CASE WHEN c THEN x END)` average over?
7. If `GROUP BY emp_id` returns 20 rows on a 20-row table, what went wrong?

Daily Revision Checklist

☐ Multi-column grouping and predicting the row count
☐ Grouping by expressions such as `EXTRACT(YEAR FROM ...)`
☐ Conditional aggregation with `SUM(CASE ...)` and `FILTER`
☐ I can trace a query stage by stage and say which stage removed a row
☐ I know why `COUNT(CASE ... ELSE 0 END)` is wrong

Chapter 7

Day 5 — JOINs

Learning Objectives

By the end of today you can:

- explain why joins exist, and what a primary key / foreign key relationship is;
- write `INNER`, `LEFT`, `RIGHT`, `FULL OUTER` and `CROSS` joins;
- predict where NULLs appear in an outer join result;
- write a self join and say what it is for;
- choose the right join type from the wording of a question.

7.1 Why joins exist

Our `employees` table stores `dept_id = 1`, not `'Engineering'`. Why not just store the name?

If we stored the name in every row	Storing an id and joining
Renaming a department means updating 7 rows	Update 1 row
Someone types "Enginering" — now you have two departments	Impossible: the FK must point at a real row
Department budget and location repeated 7 times	Stored once

That is **normalization** (Day 13): store each fact exactly once. The price you pay is that answering "which department does Priya work in?" requires putting two tables back together — and that is a **join**.

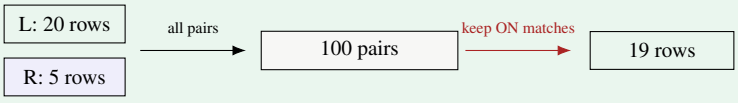
Concept

Primary key (PK): a column (or set of columns) that uniquely identifies a row. `departments.dept_id`.
Foreign key (FK): a column that holds a value of some other table's primary key. `employees.dept_id`. The database enforces that the referenced row exists — this is *referential integrity*.
A join condition is almost always FK = PK. If you can find the FK, you have found the `ON` clause.

7.2 Mental Model: what a join actually does

Mental Model

Conceptually, a join takes every row of the left table and every row of the right table, forms all possible pairs, and then keeps the pairs that satisfy the `ON` condition.



The database does not literally build 100 rows (it uses indexes and hash tables), but *thinking* that way predicts the answer correctly every time — especially when the join condition is wrong and you get an explosion of rows.

7.3 The five joins, one at a time

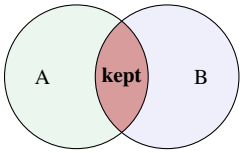
Throughout, use these two tiny tables so you can check every row by hand:

```
      A (left)                  B (right)
 id | name                  id | label
----+-------               ----+--------
  1 | Alpha                  1 | One
  2 | Beta                   2 | Two
  3 | Gamma                  4 | Four
```

7.3.1 INNER JOIN — only matches

```
SELECT A.id, A.name, B.label
FROM A INNER JOIN B ON A.id = B.id;
```

```
 id | name  | label
----+-------+-------
  1 | Alpha | One
  2 | Beta  | Two
```



rows 3 (Gamma) and 4 (Four) are dropped

Gamma has no match in B, Four has no match in A — both vanish. **No NULLs are introduced.**

7.3.2 LEFT JOIN — all of the left, matches from the right

```
SELECT A.id, A.name, B.label
FROM A LEFT JOIN B ON A.id = B.id;
```

```
 id | name  | label
----+-------+--------
  1 | Alpha | One
  2 | Beta  | Two
  3 | Gamma | (null)      <- kept, padded with NULL
```

Mental Model

This is the join you will use most, and the NULL is the point. A `LEFT JOIN` says: *"keep every left row no matter what; where there is no partner, fill the right columns with NULL."*
That NULL is not a nuisance — it is a signal. "Which left rows have no partner?" becomes `LEFT JOIN ... WHERE B.id IS NULL`. That is the **anti-join** pattern, and it answers a huge family of interview questions: customers with no orders, employees on no project, students who took no exam.

7.3.3 RIGHT JOIN — the mirror image

```
SELECT A.id, A.name, B.label
FROM A RIGHT JOIN B ON A.id = B.id;
```

```
 id     | name    | label
--------+---------+-------
      1 | Alpha   | One
      2 | Beta    | Two
 (null) | (null)  | Four
```

Interview Insight

`A RIGHT JOIN B` is identical to `B LEFT JOIN A` with the columns reordered. In practice almost nobody writes `RIGHT JOIN` — it is harder to read, because the table you care about is not the one you named first. Know it exists, know the equivalence, and write `LEFT JOIN` in interviews.

7.3.4 FULL OUTER JOIN — everything from both sides

```
SELECT A.id, A.name, B.label
FROM A FULL OUTER JOIN B ON A.id = B.id;
```

```
 id     | name    | label
--------+---------+--------
      1 | Alpha   | One
      2 | Beta    | Two
      3 | Gamma   | (null)
 (null) | (null)  | Four
```

Used for reconciliation: "show me everything in either system, and flag what is missing from each". Note that MySQL does not support it; PostgreSQL does.

7.3.5 CROSS JOIN — every pair

```
SELECT A.name, B.label FROM A CROSS JOIN B;     -- 3 x 3 = 9 rows
```

No `ON` clause. Legitimate uses are rare but real: generating all date × product combinations for a report, or building a calendar. Its main significance is as a **bug**: forget the `ON` condition and you get one accidentally.

7.3.6 SELF JOIN — a table joined to itself

Nothing special about the syntax; the trick is the aliases. Our `employees` table has `manager_id` pointing at `emp_id` *in the same table*.

```
SELECT e.name AS employee,
       m.name AS manager
FROM   employees e
LEFT   JOIN employees m ON e.manager_id = m.emp_id
ORDER  BY manager NULLS FIRST, employee;
```

```
employee      | manager
--------------+--------------
Asha Nair     | (null)
Omar Ali      | (null)
Vikram Singh  | (null)
Ananya Bose   | Asha Nair
Deepak Reddy  | Asha Nair
...
Priya Sharma  | Rohit Verma
```

Mental Model

Pretend there are two copies of the table on the desk in front of you. One copy you call `e` and read as "the employee"; the other you call `m` and read as "the manager". Then the join condition is just the sentence: *"the employee's manager_id equals the manager's emp_id"*.
The `LEFT` matters: with `INNER JOIN` the three people who have no manager disappear from the result. Whether you want them is part of the question, and interviewers often check whether you noticed.

7.4 The real thing — joining our tables

```
SELECT e.name, e.salary, d.dept_name, d.location
FROM   employees e
JOIN   departments d ON e.dept_id = d.dept_id      -- JOIN means INNER JOIN
WHERE  d.location = 'Bangalore'
ORDER  BY e.salary DESC;
```

```
name          | salary | dept_name   | location
--------------+--------+-------------+----------
Asha Nair     | 180000 | Engineering | Bangalore
Rohit Verma   | 120000 | Engineering | Bangalore
Priya Sharma  |  95000 | Engineering | Bangalore
Karan Mehta   |  88000 | Engineering | Bangalore
Ananya Bose   |  85000 | HR          | Bangalore
...
```

Omar Ali is absent — his `dept_id` is NULL, so he matches no department. Switch to `LEFT JOIN` and he reappears with NULL department columns.

7.5 Common Mistakes

Common Mistake

1. Filtering the outer table in `WHERE` instead of `ON`.

```
-- BROKEN: this silently becomes an INNER JOIN
SELECT c.customer_name, o.order_id
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
WHERE o.status = 'DELIVERED';
```

For a customer with no orders, `o.status` is NULL, and `NULL = 'DELIVERED'` is UNKNOWN, so `WHERE` throws the row away — undoing the whole point of the `LEFT JOIN`. Put the condition in `ON` instead:

```
LEFT JOIN orders o ON c.customer_id = o.customer_id AND o.status = 'DELIVERED'
```

Rule: conditions on the *optional* table go in `ON`; conditions on the *required* table go in `WHERE`. This is one of the highest-value facts on this page.
2. Ambiguous column names. `SELECT dept_id FROM employees JOIN departments ...` fails with *column reference "dept_id" is ambiguous*. Qualify it: `e.dept_id`.
3. Forgetting `ON`. `FROM a, b` with no `WHERE` is a Cartesian product. On 20 × 5 rows that is 100 rows; on two million-row tables it is a catastrophe.
4. Joining on the wrong columns. `ON e.emp_id = d.dept_id` compiles fine and returns nonsense. Always check the FK direction.
5. Assuming `JOIN` means `LEFT JOIN`. Bare `JOIN` is `INNER JOIN`. If rows are mysteriously missing, this is usually why.

7.6 Guided Practice

Practice

D5-P1 🟢 List each employee's name with their department name.

D5-P2 🟢 List every employee with their department name, including employees who have no department. *Hint: which table must be preserved?*

D5-P3 🟢 List each order with the customer's name.

D5-P4 🟢 List every department with its employees, including departments that have none. (All five of ours have employees — try inserting an empty department first.)

D5-P5 🟡 List each employee with their manager's name.

7.7 Independent Practice

Practice

D5-P6 🟡 List employees in Bangalore-based departments earning above 80000.

D5-P7 🟡 List each order with the customer name and the customer's country, only for delivered orders.

D5-P8 🟡 List every product together with the total quantity ever ordered, including products never ordered.

D5-P9 🔴 Find customers who have never placed an order. *Hint: LEFT JOIN, then look for the NULL.*

D5-P10 🔴 List each employee, their manager, and their department name — keeping employees who have no manager.

7.8 LeetCode Practice

Problem	Name	Diff.	Concept
1378	Replace Employee ID With The Unique Identifier	🟢 Easy	`LEFT JOIN` basics
1068	Product Sales Analysis I	🟢 Easy	simple `INNER JOIN`
577	Employee Bonus	🟢 Easy	`LEFT JOIN` + `IS NULL`
1581	Customer Who Visited but Did Not Make Any Transactions	🟡 Easy	**anti-join**
175	Combine Two Tables	🟢 Easy	`LEFT JOIN` — the canonical one

7.9 Interview Questions

1. `INNER JOIN` vs `LEFT JOIN` — when does the result differ?
2. Where do NULLs come from in a `LEFT JOIN` result?
3. What is the difference between putting a condition in `ON` versus `WHERE` for an outer join?
4. What is a self join and when would you use one?
5. What is a Cartesian product and how do you create one by accident?

Daily Quiz

1. Does bare `JOIN` mean inner or outer?
2. `A LEFT JOIN B` is equivalent to which right join?
3. How many rows does `CROSS JOIN` of 4 and 6 rows produce?

4. Which side gets NULL-padded in a `LEFT JOIN`?
5. Write the anti-join skeleton for "customers with no orders".
6. Why is `WHERE o.status = 'X'` dangerous after a `LEFT JOIN`?
7. In a self join, how many aliases do you need?

Daily Revision Checklist

☐ I can draw the Venn diagram for each join type
☐ I can say where NULLs appear and why
☐ `ON` vs `WHERE` for outer joins
☐ I can write a self join with two aliases
☐ I know the anti-join pattern

Chapter 8

Day 6 — Advanced JOIN Problems

Learning Objectives

By the end of today you can:

- join three or more tables, including through a junction table;
- combine joins with aggregation without double-counting;
- find unmatched records (anti-join) and duplicates using joins;
- recognise and avoid accidental Cartesian products and fan-out.

8.1 Joining three or more tables

Joins chain left to right: the result of the first join becomes the left input of the second.

"Which employees work on which projects, and in which department is the project?"

```
SELECT e.name       AS employee,
       p.project_name,
       ep.role,
       ep.hours_worked,
       d.dept_name
FROM   employees e
JOIN   employee_projects ep ON e.emp_id     = ep.emp_id
JOIN   projects p           ON ep.project_id = p.project_id
JOIN   departments d        ON p.dept_id     = d.dept_id
ORDER  BY p.project_name, ep.hours_worked DESC;
```

```
employee      | project_name      | role      | hours_worked | dept_name
--------------+-------------------+-----------+--------------+------------
Nikhil Jain   | Brand Campaign    | Analyst   |          320 | Marketing
Farah Sheikh  | Brand Campaign    | Writer    |          280 | Marketing
Ishita Kapoor | Brand Campaign    | Lead      |          180 | Marketing
Divya Menon   | CRM Rollout       | Analyst   |          260 | Sales
...
```

Mental Model

The junction table is the hinge. `employee_projects` exists because the relationship is many-to-many, and you cannot get from `employees` to `projects` without passing through it. Whenever a question involves "which X are involved with which Y" and both sides can have many of the other, look for — or design — a junction table.

Write multi-table joins in *path order*: start at the table you are reporting on, then follow the foreign keys outward. Do not jump around.

8.2 Joins with aggregation — and the fan-out trap

"How many employees and how many projects does each department have?" The obvious query is wrong:

```
-- BROKEN
SELECT d.dept_name, COUNT(e.emp_id) AS employees, COUNT(p.project_id) AS projects
FROM   departments d
LEFT JOIN employees e ON d.dept_id = e.dept_id
LEFT JOIN projects  p ON d.dept_id = p.dept_id
GROUP  BY d.dept_name;
```

```
dept_name   | employees | projects
------------+-----------+----------
Engineering |        14 |       14      <- WRONG: 7 employees x 2 projects
Sales       |         5 |        5
```

Common Mistake

Fan-out. Engineering has 7 employees and 2 projects. Joining both produces $7 \times 2 = 14$ rows, so both counts are inflated. The join multiplied the rows before the aggregate ever ran.

Fixes, in order of preference:

1. `COUNT(DISTINCT e.emp_id)` and `COUNT(DISTINCT p.project_id)` — quick, correct, slightly slow.
2. Aggregate each table *separately* in subqueries or CTEs and then join the results (Days 7–8). This is the clean solution.

```sql
-- FIXED (quick version)
SELECT d.dept_name,
       COUNT(DISTINCT e.emp_id)     AS employees,
       COUNT(DISTINCT p.project_id) AS projects
FROM   departments d
LEFT JOIN employees e ON d.dept_id = e.dept_id
LEFT JOIN projects  p ON d.dept_id = p.dept_id
GROUP  BY d.dept_name
ORDER  BY employees DESC;
```

```
dept_name   | employees | projects
------------+-----------+----------
Engineering |         7 |        2
Sales       |         5 |        1
Marketing   |         3 |        1
HR          |         2 |        1
Research    |         2 |        1
```

A safe aggregation join

Aggregating exactly one child table is always safe:

```sql
-- total revenue per customer
SELECT c.customer_name,
       COUNT(DISTINCT o.order_id)              AS orders,
       COALESCE(SUM(oi.quantity * oi.unit_price), 0) AS revenue
FROM   customers c
LEFT JOIN orders o      ON c.customer_id = o.customer_id
LEFT JOIN order_items oi ON o.order_id    = oi.order_id
GROUP  BY c.customer_id, c.customer_name
ORDER  BY revenue DESC;
```

Here `SUM` is correct even though rows fan out, because every fanned row contributes a real line item. `COUNT(DISTINCT o.order_id)` is needed because *order* rows are duplicated by the item join. Knowing which measures survive fan-out and which do not is a genuinely senior skill.

Mental Model

The fan-out test: after all your joins, ask *"what is one row of this intermediate result?"* Above, it is "one order line". Then: `SUM` of a line-level amount is fine; `COUNT` of orders is not (each order appears once per line); `AVG` of an order-level value is not.

8.3 Finding unmatched records (anti-join)

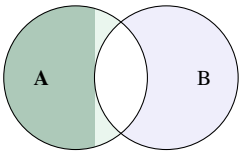
Three ways to answer "customers with no orders". Learn all three; they come up constantly.

```
-- 1. LEFT JOIN + IS NULL    (most common in interviews)
SELECT c.customer_name
FROM   customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
WHERE  o.order_id IS NULL;

-- 2. NOT EXISTS             (usually the best performer, NULL-safe)
SELECT c.customer_name
FROM   customers c
WHERE  NOT EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id);

-- 3. NOT IN                 (DANGEROUS if the subquery can return NULL)
SELECT c.customer_name
FROM   customers c
WHERE  c.customer_id NOT IN (SELECT customer_id FROM orders);
```

```
customer_name
----------------
Lyra Interiors
Cobalt Motors
```

Common Mistake

Option 3 returns **zero rows** if any `orders.customer_id` is NULL, because `x NOT IN (1, 2, NULL)` is never TRUE. Our data has no such NULL, so it works here — but in an interview, say the caveat out loud and prefer `NOT EXISTS`. Interviewers specifically probe for this.



anti-join: left rows with *no* partner

8.4 Finding duplicates

```
-- 1. Which values are duplicated?  (GROUP BY + HAVING)
SELECT salary, COUNT(*) AS n
FROM   employees
GROUP  BY salary
HAVING COUNT(*) > 1;
```

```
salary | n
-------+---
 67000 | 2      <- Arjun Desai and Divya Menon
```

```
-- 2. Which ROWS are the duplicates?  (self join)
SELECT a.name, b.name, a.salary
FROM   employees a
JOIN   employees b ON a.salary = b.salary
                  AND a.emp_id < b.emp_id;       -- '<' avoids self-pairs and mirror
    pairs
```

```
name          | name          | salary
--------------+---------------+--------
Arjun Desai   | Divya Menon   |  67000
```

Mental Model

The condition `a.emp_id < b.emp_id` does two jobs at once: it stops a row from pairing with itself (`a.emp_id = b.emp_id`), and it stops each pair appearing twice in both orders. Using `<>` instead would give you both (Arjun, Divya) and (Divya, Arjun). This "strictly less than" trick appears in almost every self-join problem — memorise it.

8.5 Common JOIN mistakes recap

Common Mistake

1. Accidental Cartesian product — missing or wrong `ON`. Symptom: row count is a suspiciously round multiple of what you expected.
2. Fan-out inflating aggregates — joining two child tables and counting both. Symptom: counts that are products of each other.
3. `LEFT JOIN` silently downgraded to `INNER` by a `WHERE` on the right table.
4. Duplicate rows after joining because the join key is not unique on the right side. Check with `SELECT key, COUNT(*) ... HAVING COUNT(*) > 1`.
5. Joining on a non-key column — `ON e.name = c.customer_name` may "work" on sample data and fail in production.
Diagnostic habit: run `SELECT COUNT(*)` before and after adding a join. If the number goes up unexpectedly, you have fan-out.

8.6 Guided Practice

Practice

D6-P1 ● List employee name, project name and hours for every assignment.
D6-P2 ● Show each order with the customer name and the product names in it.
D6-P3 ● Count how many projects each department has.
D6-P4 ● For each project, show the project name and the number of employees on it.
D6-P5 ● Find employees not assigned to any project.

8.7 Independent Practice

Practice

D6-P6 ● For each customer, show the number of orders and total revenue, including customers with none (revenue should be 0, not NULL).
D6-P7 ● Find products that have never been ordered.
D6-P8 ● For each department show the number of employees and the number of projects, correctly (no fan-out).
D6-P9 ● Find pairs of employees in the same department with the same job title, without duplicate mirrored pairs.
D6-P10 ● For each project, list the lead's name (`role = 'Lead'`) and the total hours logged by all members.
D6-P11 ● Find the top 3 customers by revenue, showing name, country and revenue.
D6-P12 ● List employees who earn more than their own manager.
D6-P13 ● For each department, show the department name, the manager count (employees who

manage at least one person) and the total headcount.

8.8 LeetCode Practice

Problem	Name	Diff.	Concept
1661	Average Time of Process per Machine	Easy	self join + conditional aggregation
1934	Confirmation Rate	Medium	`LEFT JOIN` + `AVG` of a `CASE`
570	Managers with at Least 5 Direct Reports	Medium	self join + `HAVING`
1174	Immediate Food Delivery II	Medium	join + group-wise minimum
1193	Monthly Transactions I	Medium	date grouping + conditional aggregation
197	Rising Temperature	Easy	self join on **date arithmetic**

8.9 Interview Questions

1. What is fan-out, and how do you detect and fix it?
2. Give three ways to find rows in A with no match in B. Which is safest and why?
3. Why does `a.id < b.id` appear in self joins?
4. How would you find duplicate rows in a table?
5. How do you join two tables that have no direct foreign key between them?

Daily Quiz

1. Joining a 7-row and a 2-row child of the same parent gives how many rows?
2. Why is `NOT IN` risky?
3. Which aggregate survives fan-out: `SUM` of a line amount, or `COUNT` of orders?
4. How many join conditions do you need to chain 4 tables?
5. What does `COALESCE(SUM(x), 0)` protect against?
6. Name the three anti-join formulations.
7. How do you check whether a join key is unique?

Daily Revision Checklist

☐ I can chain 3–4 tables through a junction table
☐ I can spot fan-out and fix it with `DISTINCT` or pre-aggregation
☐ I know all three anti-join forms and the `NOT IN` caveat
☐ I can find duplicates two ways
☐ I always ask "what is one row of this result?"

Chapter 9

Day 7 — Subqueries

Learning Objectives

By the end of today you can:

- write scalar subqueries, `IN` subqueries and `EXISTS` subqueries;
- explain what makes a subquery *correlated* and what that costs;
- use a subquery in `FROM` (a derived table) and in `SELECT`;
- choose between a subquery and a join, and justify the choice.

9.1 What is a subquery?

Concept

A subquery is a `SELECT` inside another statement, wrapped in parentheses. The database runs the inner query and feeds its result to the outer one.
Subqueries come in three shapes, and the shape determines where you can use them:

Shape	Returns	Usable in
Scalar	one row, one column	anywhere a value goes: `SELECT`, `WHERE` comparisons
Column / list	many rows, one column	`IN`, `ANY`, `ALL`
Table (derived)	many rows, many columns	`FROM`, `JOIN`

9.2 Scalar subqueries

"Who earns more than the company average?"

```
SELECT name, salary
FROM   employees
WHERE  salary > (SELECT AVG(salary) FROM employees)
ORDER  BY salary DESC;
```

```
name          | salary
--------------+---------
Asha Nair     | 180000
Vikram Singh  | 150000
Rohit Verma   | 120000
Deepak Reddy  | 110000
Ishita Kapoor |  98000
Priya Sharma  |  95000
Neha Gupta    |  92000
Karan Mehta   |  88000
Ananya Bose   |  85000
```

Mental Model

This is the answer to a question you could not solve on Day 3. `WHERE` cannot contain an aggregate — but it *can* contain a subquery whose result is a single number. The inner query runs once, produces `89400`, and the outer query then reads as `WHERE salary > 89400`.
Whenever a question compares each row against a whole-table summary, you need a scalar subquery. "Above average", "more than the maximum in department 3", "more than twice the minimum" — all the same shape.

Scalar subqueries also work in `SELECT`:

```
SELECT name,
       salary,
       (SELECT ROUND(AVG(salary)) FROM employees) AS company_avg,
       salary - (SELECT AVG(salary) FROM employees) AS diff_from_avg
FROM employees
ORDER BY diff_from_avg DESC
LIMIT 5;
```

Common Mistake

A scalar subquery that returns more than one row raises *more than one row returned by a subquery used as an expression*. If you are not certain it returns one row, use `IN` instead of =, or add `LIMIT 1` with an explicit `ORDER BY`.

9.3 Subqueries with IN

"Who works in a Bangalore department?"

```
SELECT name, dept_id
FROM   employees
WHERE  dept_id IN (SELECT dept_id FROM departments WHERE location = 'Bangalore');
```

The inner query returns `\{1, 3\}`; the outer keeps employees in those departments.

Common Mistake

`NOT IN` plus NULL, once more. If the subquery can return NULL, `NOT IN` returns nothing at all:

```
-- returns 0 rows because employees.manager_id contains NULLs
SELECT name FROM employees
WHERE emp_id NOT IN (SELECT manager_id FROM employees);
```

Fix it with `WHERE manager_id IS NOT NULL` inside the subquery, or use `NOT EXISTS`. This exact query — "employees who manage nobody" — is a classic interview trap.

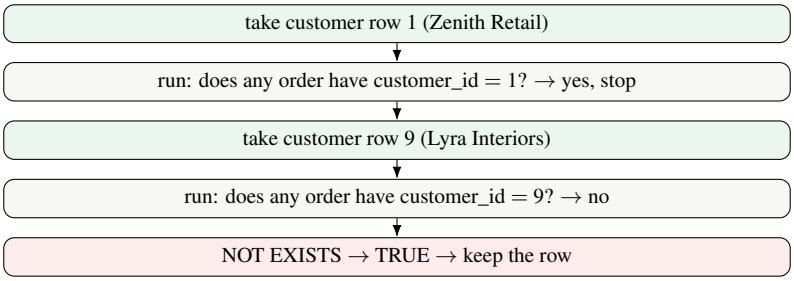
9.4 EXISTS and correlated subqueries

Concept

A **correlated** subquery references a column from the outer query. It therefore cannot be run once — conceptually it re-runs for every outer row.
`EXISTS` asks a yes/no question: *"does at least one matching row exist?"* It stops at the first match and does not care what the subquery selects, which is why everyone writes `SELECT 1`.

```
-- customers who HAVE placed an order
SELECT c.customer_name
```

```sql
FROM   customers c
WHERE  EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id);
                                        -- ^^^^^^^^^^^^^^ this is the correlation

-- customers who have NOT
SELECT c.customer_name
FROM   customers c
WHERE  NOT EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id);
```



A correlated subquery that is not EXISTS

"Employees earning more than their department's average."

```sql
SELECT e.name, e.dept_id, e.salary
FROM   employees e
WHERE  e.salary > (SELECT AVG(e2.salary)
                   FROM   employees e2
                   WHERE  e2.dept_id = e.dept_id)    -- correlated
ORDER  BY e.dept_id, e.salary DESC;
```

```
name          | dept_id | salary
--------------+---------+---------
Asha Nair     |       1 | 180000
Rohit Verma   |       1 | 120000
Priya Sharma  |       1 |  95000
Vikram Singh  |       2 | 150000
Neha Gupta    |       2 |  92000
Ananya Bose   |       3 |  85000
Ishita Kapoor |       4 |  98000
Deepak Reddy  |       5 | 110000
```

Mental Model

Read it as: *"for this employee, compute the average salary of people in the same department, and keep the employee if they beat it."* The subquery's answer changes from row to row — that is what "correlated" means, and it is the difference from the whole-company version at the top of this chapter.
On Day 9 you will write this same query with a window function, and it will be shorter and faster. But you should be able to write both.

9.5 Subquery in FROM — derived tables

A subquery in `FROM` is a temporary table. It **must** have an alias in PostgreSQL.

"Which departments have an average salary above 80000, and how does that compare with their headcount?"

```sql
SELECT dept_stats.dept_id, dept_stats.headcount, dept_stats.avg_salary
FROM (
        SELECT dept_id,
```

```
            COUNT(*)                AS headcount,
            ROUND(AVG(salary), 2)   AS avg_salary
        FROM   employees
        WHERE  dept_id IS NOT NULL
        GROUP  BY dept_id
    ) AS dept_stats                              -- alias is mandatory
WHERE dept_stats.avg_salary > 80000
ORDER BY dept_stats.avg_salary DESC;
```

```
dept_id | headcount | avg_salary
--------+-----------+------------
      1 |         7 |   97714.29
      5 |         2 |   94500.00
      2 |         5 |   86000.00
```

Mental Model

This is how you **filter on an alias** that `WHERE` could not see (Day 2's restriction). Compute it in an inner query; the outer query sees it as a real column.
It is also how you *aggregate twice* — for example "the average of the department averages". You cannot nest aggregates directly (`AVG(AVG(x))` is an error), so you aggregate in a derived table and aggregate again outside.

9.6 Subquery vs JOIN — how to choose

You want…	Use	Why
Columns from both tables	**JOIN**	A subquery cannot give you the other table's columns
Just to test membership	`EXISTS` / `IN`	No risk of duplicating rows
To exclude matched rows	`NOT EXISTS`	Safe with NULLs; clear intent
A whole-table summary to compare against	scalar subquery	A join cannot produce a single number to compare per row
A per-group summary compared per row	correlated subquery, or a window function (Day 9)	Window is usually clearer and faster
Multi-step logic	derived table / CTE (Day 8)	Readability

Interview Insight

"IN vs EXISTS — which is faster?" The honest, senior answer: *"In PostgreSQL the planner usually rewrites both into a semi-join, so performance is often identical. The real differences are semantic: `NOT IN` breaks if the subquery returns NULL, while `NOT EXISTS` does not; and `EXISTS` can short-circuit on the first match. I default to `EXISTS` for correctness reasons."*
A candidate who says "EXISTS is always faster" is repeating folklore. A candidate who mentions the NULL semantics is demonstrating understanding.

Common Mistake

1. Missing alias on a derived table — *subquery in FROM must have an alias.*
2. Correlated subquery where a join would do — correct but potentially slow, since it may re-execute per row.
3. `= (subquery)` when the subquery returns many rows — runtime error.
4. Referring to an outer alias inside a non-correlated subquery — if you forget the correlation condition, `EXISTS` becomes "does this table have any rows at all", which is true for every outer row. Symptom: your filter does nothing.

9.7 Guided Practice

Practice

D7-P1 🟢 Find employees earning more than the company average.
D7-P2 🟢 Find employees in departments located in Bangalore, using `IN`.
D7-P3 🟠 Find products priced above the average product price.
D7-P4 🟠 Find customers who have placed at least one order, using `EXISTS`.
D7-P5 🟠 Show each employee's salary alongside the company average and the difference.

9.8 Independent Practice

Practice

D7-P6 🟠 Find employees earning more than the average salary *of their own department*.
D7-P7 🟠 Find customers with no orders, using `NOT EXISTS`.
D7-P8 🟠 Find the department with the highest total salary bill. *Hint: derived table plus* `ORDER BY ... LIMIT 1`.
D7-P9 🔴 Find the second highest salary in the company, without using window functions.
D7-P10 🔴 Find employees who work on at least one project with more than 300 hours logged by them.
D7-P11 🔴 Compute the average of the department average salaries. (Two levels of aggregation.)
D7-P12 🔴 Find employees who manage nobody — and make sure your query is not broken by NULLs.

9.9 LeetCode Practice

Problem	Name	Diff.	Concept
1978	Employees Whose Manager Left the Company	🟠 Easy	`NOT IN` + NULL care
1341	Movie Rating	🔴 Medium	two aggregations + `UNION ALL` + tie-breakin
602	Friend Requests II	🟠 Medium	`UNION ALL` + `GROUP BY`
585	Investments in 2016	🔴 Medium	multiple correlated conditions
176	Second Highest Salary	🟠 Medium	subquery + `OFFSET`, NULL when absent

9.10 Interview Questions

1. What is a correlated subquery? How does it differ in execution from a normal one?
2. `IN` vs `EXISTS` — semantics and performance.
3. When would you use a subquery instead of a join?
4. Why must a subquery in `FROM` have an alias?
5. How do you compute an aggregate of an aggregate?

Daily Quiz

1. What error do you get if a scalar subquery returns 3 rows?
2. Why does `NOT IN` with a NULL-containing list return nothing?
3. What does `SELECT 1` inside `EXISTS` mean?
4. Where can a scalar subquery appear?

5. Is `EXISTS` correlated by definition?
6. Write the skeleton for "rows above their group's average".
7. Can you nest `AVG(AVG(x))`?

Daily Revision Checklist

☐ Three subquery shapes: scalar, list, table
☐ Correlated vs non-correlated
☐ `EXISTS` / `NOT EXISTS` and the `NOT IN` NULL trap
☐ Derived tables need an alias
☐ I can state when a join beats a subquery and vice versa

Chapter 10

Day 8 — Common Table Expressions (CTEs)

Learning Objectives

By the end of today you can:

- write `WITH` clauses, including several chained CTEs;
- rewrite a nested subquery as a readable CTE pipeline;
- combine CTEs with joins and aggregation;
- explain what a recursive CTE does at an interview-appropriate level.

10.1 What a CTE is

Concept

A CTE is a named temporary result set defined at the top of a query with `WITH`. It exists only for the duration of that one statement.

```
WITH cte_name AS (
    SELECT ...
)
SELECT ... FROM cte_name;
```

Functionally, a CTE is a derived table with a name in front instead of an alias behind. That small change has a large effect on readability.

The same query, two ways

"Departments whose average salary is above 80000, with the department name."

```
-- With a nested subquery: read inside-out
SELECT d.dept_name, s.avg_salary
FROM   (SELECT dept_id, AVG(salary) AS avg_salary
        FROM employees WHERE dept_id IS NOT NULL
        GROUP BY dept_id) s
JOIN   departments d ON d.dept_id = s.dept_id
WHERE  s.avg_salary > 80000;

-- With a CTE: read top-down
WITH dept_avg AS (
    SELECT dept_id, AVG(salary) AS avg_salary
    FROM   employees
    WHERE  dept_id IS NOT NULL
    GROUP  BY dept_id
)
SELECT d.dept_name, ROUND(a.avg_salary, 2) AS avg_salary
FROM   dept_avg a
JOIN   departments d ON d.dept_id = a.dept_id
WHERE  a.avg_salary > 80000
ORDER  BY a.avg_salary DESC;
```

Mental Model

Why CTEs improve readability. A nested subquery forces you to read from the inside out — you must find the innermost parentheses first, and the logic runs backwards from how you thought about the problem.
A CTE lets you write the query in the order you *reasoned*: "first compute department averages; then join them to names; then filter." Each step gets a name, and a named step is a step you can explain out loud. In an interview that is worth more than the query being one line shorter.

10.2 Multiple CTEs

Separate them with commas. Later CTEs may reference earlier ones.

"Which customers spent above the average customer spend?"

```
WITH order_totals AS (            -- step 1: value of each order
    SELECT o.order_id,
           o.customer_id,
           SUM(oi.quantity * oi.unit_price) AS order_value
    FROM   orders o
    JOIN   order_items oi ON o.order_id = oi.order_id
    GROUP  BY o.order_id, o.customer_id
),
customer_spend AS (               -- step 2: total per customer
    SELECT customer_id,
           SUM(order_value) AS total_spend,
           COUNT(*)         AS order_count
    FROM   order_totals
    GROUP  BY customer_id
),
overall AS (                      -- step 3: the benchmark
    SELECT AVG(total_spend) AS avg_spend FROM customer_spend
)
SELECT c.customer_name,
       cs.order_count,
       cs.total_spend,
       ROUND(o.avg_spend, 2) AS avg_customer_spend
FROM   customer_spend cs
JOIN   customers c ON c.customer_id = cs.customer_id
CROSS  JOIN overall o                     -- one row, so CROSS JOIN is safe
WHERE  cs.total_spend > o.avg_spend
ORDER  BY cs.total_spend DESC;
```

```
customer_name  | order_count | total_spend | avg_customer_spend
---------------+-------------+-------------+--------------------
Zenith Retail  |           3 |    488000.00|          230637.50
Nova Systems   |           3 |    322600.00|          230637.50
Summit Apparel |           1 |    412500.00|          230637.50
...
```

Mental Model

Read the CTE names top to bottom and you get the algorithm in English: *order totals* → *customer spend* → *overall average* → *compare*. This is exactly how you should *narrate* a complex query in an interview, and writing it as CTEs means your code and your narration have the same structure.
Note the `CROSS JOIN overall`: because `overall` has exactly one row, cross joining attaches that benchmark to every customer row. This is a clean idiom for "compare each row against a global number".

10.3 CTE plus everything else

CTE + JOIN + GROUP BY

```
WITH project_hours AS (
    SELECT project_id,
           SUM(hours_worked)   AS total_hours,
           COUNT(DISTINCT emp_id) AS team_size
    FROM   employee_projects
    GROUP  BY project_id
)
SELECT p.project_name,
       d.dept_name,
       ph.team_size,
       ph.total_hours,
       ROUND(ph.total_hours::numeric / ph.team_size, 1) AS hours_per_person
FROM   project_hours ph
JOIN   projects p    ON p.project_id = ph.project_id
JOIN   departments d ON d.dept_id    = p.dept_id
ORDER  BY ph.total_hours DESC;
```

```
project_name      | dept_name   | team_size | total_hours | hours_per_person
------------------+-------------+-----------+-------------+------------------
Payments Platform | Engineering |         4 |        1470 |             367.5
Mobile App Revamp | Engineering |         4 |        1110 |             277.5
Brand Campaign    | Marketing   |         3 |         780 |             260.0
LLM Prototype     | Research    |         2 |         760 |             380.0
CRM Rollout       | Sales       |         3 |         720 |             240.0
Employee Portal   | HR          |         2 |         350 |             175.0
```

Interview Insight

Notice `::numeric`. In PostgreSQL, integer divided by integer is *integer division*: `1470 / 4` is 367, not 367.5. Casting one side forces real division. This bites people in interviews when computing percentages and ratios — Day 12 covers it properly.

10.4 Recursive CTEs — the interview-level version

Concept

A recursive CTE repeatedly feeds its own output back into itself until nothing new is produced. The shape is always:

```
WITH RECURSIVE name AS (
    <anchor query>            -- the starting rows
    UNION ALL
    <recursive query>         -- refers to 'name', produces the next level
)
SELECT * FROM name;
```

The canonical use is walking a hierarchy — which our `employees` table has, via `manager_id`.

```
WITH RECURSIVE org_chart AS (
    -- anchor: the top of the tree
    SELECT emp_id, name, manager_id, 1 AS level, name::text AS path
    FROM   employees
    WHERE  manager_id IS NULL
```

```
    UNION ALL

    -- recursive: everyone reporting to someone already in org_chart
    SELECT e.emp_id, e.name, e.manager_id, oc.level + 1,
           oc.path || ' > ' || e.name
    FROM   employees e
    JOIN   org_chart oc ON e.manager_id = oc.emp_id
)
SELECT level, name, path
FROM   org_chart
ORDER  BY path;
```

```
level | name          | path
------+---------------+---------------------------------
    1 | Asha Nair     | Asha Nair
    2 | Ananya Bose   | Asha Nair > Ananya Bose
    3 | Rahul Pillai  | Asha Nair > Ananya Bose > Rahul Pillai
    2 | Deepak Reddy  | Asha Nair > Deepak Reddy
    3 | Tanvi Shah    | Asha Nair > Deepak Reddy > Tanvi Shah
    2 | Rohit Verma   | Asha Nair > Rohit Verma
    3 | Priya Sharma  | Asha Nair > Rohit Verma > Priya Sharma
    ...
```

Interview Insight

How much do you need for a fresher interview? You should be able to (a) say what a recursive CTE is for — hierarchies, graph traversal, generating sequences — (b) name the two parts (anchor and recursive term joined by `UNION ALL`), and (c) note that it needs a termination condition or it runs forever.
You are unlikely to be asked to write one from scratch. If you are, the org-chart query above is the pattern for essentially all of them. Do not spend Day 8 memorising exotic recursive queries at the expense of window functions — those are far more likely to appear.

Common Mistake

1. `WITH` placed after `SELECT` — it must come first.
2. Expecting a CTE to persist — it vanishes after the statement. For reuse, create a `VIEW`.
3. Referencing a later CTE from an earlier one — only backwards references work.
4. Forgetting `RECURSIVE` — PostgreSQL requires the keyword even though the recursion is visible in the query.
5. Assuming CTEs are always optimisation fences — they were in PostgreSQL before version 12; since 12 they are inlined when possible. If you need the old behaviour, write `AS MATERIALIZED`. Mentioning this version detail is a strong signal.

10.5 Guided Practice

Practice

D8-P1 ● Rewrite "employees earning above the company average" using a CTE for the average.
D8-P2 ● Using a CTE, compute headcount per department and then show only those with more than 2 employees.
D8-P3 ● Use a CTE to compute each order's total value, then list the five largest orders with the customer name.
D8-P4 ● Use two CTEs to find each department's total salary and then the department with the largest.
D8-P5 ● Using a CTE, find products whose total ordered quantity exceeds 10.

10.6 Independent Practice

Practice

D8-P6 🟠 Using CTEs, compute revenue per customer and show each customer's share of total revenue as a percentage.

D8-P7 🟠 Find departments where the average salary is above the company-wide average, using two CTEs.

D8-P8 🔴 For each project, compute total hours; then list projects with more hours than the average project.

D8-P9 🔴 Build a monthly order summary for 2023: month, order count, revenue.

D8-P10 🔴 Write a recursive CTE that lists every employee under Asha Nair (`emp_id = 1`) with their depth in the hierarchy.

10.7 LeetCode Practice

Problem	Name	Diff.	Concept
1204	Last Person to Fit in the Bus	🔴 Medium	CTE + running total
1907	Count Salary Categories	🟠 Medium	CTE + `UNION ALL` for missing groups
602	Friend Requests II	🟠 Medium	CTE + `UNION ALL` + ranking
1113	Reported Posts	🟢 Easy	simple CTE + `GROUP BY`

10.8 Interview Questions

1. What is a CTE and how does it differ from a subquery? From a view?
2. Why do CTEs improve readability? Give an example.
3. What is a recursive CTE and what are its two parts?
4. Does a CTE improve performance?
5. Can a CTE reference itself? Can it reference a CTE defined after it?

Daily Quiz

1. Where must `WITH` appear in the statement?
2. How do you define three CTEs in one query?
3. What keyword makes a CTE recursive in PostgreSQL?
4. Does a CTE survive after the statement finishes?
5. Which operator joins the anchor and recursive parts?
6. What is the difference between a CTE and a view?
7. Why does `1470 / 4` give 367 in PostgreSQL?

Daily Revision Checklist

☐ `WITH name AS (...)` syntax and comma-chaining
☐ I can rewrite a nested subquery as a CTE pipeline
☐ CTE + JOIN + GROUP BY in one query
☐ Recursive CTE: anchor + `UNION ALL` + recursive term
☐ CTE vs view vs subquery — I can state all three differences

Chapter 11

Day 9 — Window Functions I

Learning Objectives

By the end of today you can:

- explain what a window function is and how it differs from an aggregate;
- use `OVER()`, `PARTITION BY` and `ORDER BY` inside `OVER`;
- use `ROW_NUMBER`, `RANK` and `DENSE_RANK`, and state exactly how they differ;
- solve "top N per group" and "Nth highest" with confidence.

Interview Insight

This is the highest-value day in the workbook. Window functions separate candidates who can write basic SQL from candidates who can write *useful* SQL, and they appear in a large share of medium/hard interview questions. If you only have time to master one topic, master this one.

11.1 The problem window functions solve

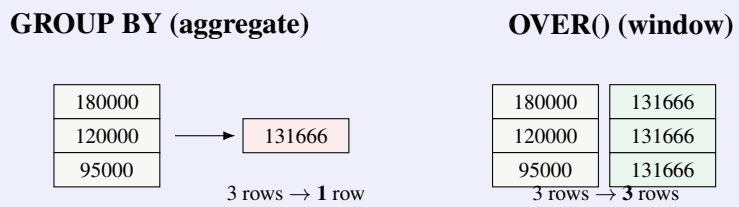
On Day 3 you hit a wall. This fails:

```
SELECT dept_id, name, MAX(salary) FROM employees GROUP BY dept_id;
-- ERROR: column "employees.name" must appear in the GROUP BY clause
```

You wanted *"for each department, the highest-paid employee"*, and `GROUP BY` could give you the number but not the person — because grouping **destroys the rows**.

Concept

A window function computes a value *across a set of related rows* while **keeping every row**. No collapsing.




```
SELECT name, dept_id, salary,
       AVG(salary) OVER (PARTITION BY dept_id) AS dept_avg
FROM employees
WHERE dept_id = 1;
```

```
name         | dept_id | salary | dept_avg
-------------+---------+--------+-----------
Asha Nair    |       1 | 180000 | 97714.29
Rohit Verma  |       1 | 120000 | 97714.29
Priya Sharma |       1 |  95000 | 97714.29
Karan Mehta  |       1 |  88000 | 97714.29
Sneha Iyer   |       1 |  72000 | 97714.29
Aditya Rao   |       1 |  68000 | 97714.29
Meera Joshi  |       1 |  61000 | 97714.29
```

Every employee is still there *and* each one carries the departmental average. Now comparing a row to its group is trivial.

11.2 Anatomy of the OVER clause

```
function() OVER (
    PARTITION BY col      -- optional: split rows into independent groups
    ORDER BY col          -- optional: order rows WITHIN each partition
    [frame]               -- optional: which rows around the current one (Day 10)
)
```

`OVER ()`	The window is the **whole result set**. `AVG(salary) OVER ()` is the company average, repeated on every row.
`PARTITION BY d`	Like `GROUP BY` for the window only: restart the calculation for each distinct `d`. Rows are *not* merged.
`ORDER BY x`	Two effects: it defines the sequence for ranking functions, and — for aggregates — it turns them into **running** totals (Day 10).

Mental Model

Say the clause out loud in English:
`RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC)`
→ *"rank, restarting for each department, ordering by salary highest first"*.
Every window function reads this way. If you can translate the English of the question into that sentence, you have written the query.

11.3 ROW_NUMBER vs RANK vs DENSE_RANK

All three number rows. They differ **only in how they handle ties.** We use the Sales department, which conveniently contains a genuine tie (Arjun and Divya both earn 67000).

```
SELECT name, salary,
       ROW_NUMBER() OVER (ORDER BY salary DESC) AS row_num,
       RANK()       OVER (ORDER BY salary DESC) AS rnk,
       DENSE_RANK() OVER (ORDER BY salary DESC) AS dense_rnk
FROM   employees
WHERE  dept_id = 2
ORDER  BY salary DESC;
```

```
name         | salary | row_num | rnk | dense_rnk
-------------+--------+---------+-----+-----------
Vikram Singh | 150000 |       1 |   1 |         1
Neha Gupta   |  92000 |       2 |   2 |         2
Arjun Desai  |  67000 |       3 |   3 |         3     <- tie
Divya Menon  |  67000 |       4 |   3 |         3     <- tie
Sameer Khan  |  54000 |       5 |   5 |         4
```

ROW_NUMBER
1 2 3 4 5 — always distinct; the tie is broken arbitrarily

RANK
1 2 3 3 5 — ties share a rank; the next rank **skips** (no 4)

DENSE_RANK
1 2 3 3 4 — ties share a rank; no gaps

Mental Model

How to choose, in one line each:

- `ROW_NUMBER` — "give me exactly one row per group" (deduplication, pick the latest record, top-1 per group).
- `RANK` — "Olympic medals": two silvers means no bronze. Use when the gap is meaningful.
- `DENSE_RANK` — "distinct salary levels". Use for "the 3rd highest *salary*" as opposed to "the 3rd highest-paid *employee*".

That last distinction is a real interview question. "Third highest salary" with values `100, 90, 90, 80`: `DENSE_RANK` says 80; `RANK` says nothing has rank 3. Ask the interviewer which they mean — asking is itself a good signal.

Common Mistake

`ROW_NUMBER` with ties is **non-deterministic**: which of Arjun and Divya gets 3 can change between runs. If determinism matters, add a tie-breaker: `ORDER BY salary DESC, emp_id ASC`. Interviewers notice when you do this unprompted.

11.4 PARTITION BY — ranking within groups

```sql
SELECT d.dept_name, e.name, e.salary,
       RANK() OVER (PARTITION BY e.dept_id ORDER BY e.salary DESC) AS dept_rank
FROM   employees e
JOIN   departments d ON d.dept_id = e.dept_id
ORDER  BY d.dept_name, dept_rank;
```

```
dept_name   | name           | salary | dept_rank
------------+----------------+--------+-----------
Engineering | Asha Nair      | 180000 |         1
Engineering | Rohit Verma    | 120000 |         2
Engineering | Priya Sharma   |  95000 |         3
...
HR          | Ananya Bose    |  85000 |         1
HR          | Rahul Pillai   |  52000 |         2
Marketing   | Ishita Kapoor  |  98000 |         1
...
```

The counter restarts at 1 for every department. That single behaviour solves the entire "top N per group" family of problems.

11.5 The Top-N-per-group pattern

Concept

You cannot filter on a window function in `WHERE` — window functions are computed *after* `WHERE` (they run at step 5.5, between `SELECT` and `ORDER BY`). So the pattern is always two-step: **rank in a CTE, filter outside**.

```sql
WITH ranked AS (
    SELECT ..., ROW_NUMBER() OVER (PARTITION BY g ORDER BY x DESC) AS rn
    FROM t
)
SELECT * FROM ranked WHERE rn <= N;
```

"The two highest-paid employees in each department."

```sql
WITH ranked AS (
    SELECT e.name, e.salary, d.dept_name,
           DENSE_RANK() OVER (PARTITION BY e.dept_id ORDER BY e.salary DESC) AS r
    FROM   employees e
    JOIN   departments d ON d.dept_id = e.dept_id
)
SELECT dept_name, name, salary, r
FROM   ranked
WHERE  r <= 2
ORDER  BY dept_name, r;
```

```
dept_name   | name          | salary | r
------------+---------------+--------+---
Engineering | Asha Nair     | 180000 | 1
Engineering | Rohit Verma   | 120000 | 2
HR          | Ananya Bose   |  85000 | 1
HR          | Rahul Pillai  |  52000 | 2
Marketing   | Ishita Kapoor |  98000 | 1
Marketing   | Nikhil Jain   |  63000 | 2
Research    | Deepak Reddy  | 110000 | 1
Research    | Tanvi Shah    |  79000 | 2
Sales       | Vikram Singh  | 150000 | 1
Sales       | Neha Gupta    |  92000 | 2
```

11.6 Nth highest salary

```sql
-- 3rd highest DISTINCT salary
WITH r AS (
    SELECT DISTINCT salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS rk
    FROM employees
)
SELECT salary FROM r WHERE rk = 3;
```

Interview Insight

Three ways to get the Nth highest, and you should know all three:

1. `DENSE_RANK` in a CTE (above) — clearest, handles ties correctly.
2. `ORDER BY salary DESC LIMIT 1 OFFSET N-1` — simple, but counts *rows*, so duplicates shift the answer.
3. Correlated subquery counting how many salaries are strictly greater — classic, but slow and harder to read.

Leading with the `DENSE_RANK` version and *mentioning* the `OFFSET` shortcut is the strongest answer. LeetCode 177 (Nth Highest Salary) is exactly this.

11.7 Common Mistakes

Common Mistake

1. Filtering a window function in `WHERE`.

```sql
SELECT name, RANK() OVER (ORDER BY salary DESC) AS r
FROM employees WHERE r <= 3;          -- ERROR: column "r" does not exist
```

Wrap it in a CTE or subquery.

2. Confusing `PARTITION BY` with `GROUP BY`. `PARTITION BY` does not reduce rows. You can use both in one query, and they need not match.

3. Omitting `ORDER BY` inside `OVER` for a ranking function — ranking without an order is meaningless, and PostgreSQL will reject it.
4. Using `RANK` when you needed `ROW_NUMBER` — if a department has two people tied at the top, `WHERE r = 1` returns both. Sometimes that is what you want; often it is not.
5. Assuming `DISTINCT` plays nicely with window functions. `DISTINCT` runs *after* the window function, so `SELECT DISTINCT x, ROW_NUMBER() OVER (...)` usually does nothing useful — every row already has a distinct number.

11.8 Guided Practice

Practice

D9-P1 🟢 Show every employee with the company average salary alongside.
D9-P2 🟢 Show every employee with their department's average salary alongside.
D9-P3 🟢 Rank all employees by salary descending using all three ranking functions in one query, and eyeball the differences.
D9-P4 🟡 Rank employees within each department by salary.
D9-P5 🟡 Show each employee's salary as a difference from their department's average.

11.9 Independent Practice

Practice

D9-P6 🟡 Find the highest-paid employee in each department (exactly one per department).
D9-P7 🟡 Find the top 2 highest-paid employees per department, including ties.
D9-P8 🟡 Find the 3rd highest distinct salary in the company.
D9-P9 🔴 For each customer, rank their orders by value and show only their largest order.
D9-P10 🔴 For each project, rank members by hours worked and show the top contributor with their role.
D9-P11 🔴 Show each employee with their salary, their department average, and their percentile position within the department (rank divided by department size).

11.10 LeetCode Practice

Problem	Name	Diff.	Concept
185	Department Top Three Salaries	🔴 Hard	`DENSE_RANK` + `PARTITION BY` — the flagship
184	Department Highest Salary	🟡 Medium	top-1 per group
177	Nth Highest Salary	🟡 Medium	`DENSE_RANK` or `OFFSET`
176	Second Highest Salary	🟡 Medium	the N=2 special case
178	Rank Scores	🟡 Medium	`DENSE_RANK` — read the tie rules carefully

11.11 Interview Questions

1. What is a window function? How does it differ from an aggregate?
2. `RANK` vs `DENSE_RANK` vs `ROW_NUMBER` — with a tie example.

3. Why can't you filter on a window function in `WHERE`? What do you do instead?
4. What does `PARTITION BY` do, and how is it different from `GROUP BY`?
5. How would you find the top 3 earners per department?

Daily Quiz

1. Does `OVER ()` with no `PARTITION BY` span the whole result?
2. With salaries 100, 90, 90, 80: what does `RANK` give the 80?
3. And `DENSE_RANK`?
4. Which ranking function is non-deterministic on ties?
5. Where in the processing order do window functions run?
6. Write the skeleton for top-N-per-group.
7. Can a query use both `GROUP BY` and a window function?

Daily Revision Checklist

☐ I can explain window vs aggregate in one sentence with the row-count difference
☐ I can read an `OVER` clause aloud in English
☐ I know the three ranking functions' tie behaviour cold
☐ I can write top-N-per-group without thinking
☐ I know why `WHERE` cannot see a window function

Chapter 12

Day 10 — Window Functions II

Learning Objectives

By the end of today you can:

- use `LAG` and `LEAD` to compare a row with its neighbours;
- build running totals and moving averages;
- read and write a window `frame` clause;
- use `FIRST_VALUE` / `LAST_VALUE` and know the `LAST_VALUE` trap.

12.1 LAG and LEAD — looking at the neighbours

Concept

`LAG(col, n, default)` returns `col` from `n` rows *before* the current row in the window's order. `LEAD` looks forward. Both default to `n = 1` and to `NULL` when there is no such row.
This is how SQL answers "compare each row with the previous one" — month-over-month growth, consecutive dates, price changes, streaks.

```
SELECT name, hire_date, salary,
       LAG(salary)  OVER (ORDER BY hire_date) AS prev_hire_salary,
       LEAD(salary) OVER (ORDER BY hire_date) AS next_hire_salary,
       salary - LAG(salary) OVER (ORDER BY hire_date) AS diff
FROM   employees
WHERE  dept_id = 1
ORDER  BY hire_date;
```

```
name         | hire_date  | salary | prev_hire_salary | next_hire_salary | diff
-------------+------------+--------+------------------+------------------+--------
Asha Nair    | 2015-03-01 | 180000 |           (null) |           120000 | (null)
Rohit Verma  | 2016-07-15 | 120000 |           180000 |            95000 | -60000
Priya Sharma | 2018-01-10 |  95000 |           120000 |            88000 | -25000
Karan Mehta  | 2019-05-20 |  88000 |            95000 |            72000 |  -7000
Sneha Iyer   | 2021-02-01 |  72000 |            88000 |            68000 | -16000
Aditya Rao   | 2021-08-11 |  68000 |            72000 |            61000 |  -4000
Meera Joshi  | 2022-11-05 |  61000 |            68000 |           (null) |  -7000
```

Mental Model

The NULLs at the ends are structural, not errors — the first row has no predecessor. Handle them with the third argument or `COALESCE`:

```
LAG(salary, 1, 0) OVER (ORDER BY hire_date)      -- 0 instead of NULL
```

The recognition rule: any question containing "previous", "next", "compared with last month", "change from", "consecutive" or "gap" is a `LAG`/`LEAD` question.

Consecutive records — the classic

"Find customers who placed orders on two consecutive days."

```
WITH gaps AS (
```

```
    SELECT customer_id, order_date,
           LAG(order_date) OVER (PARTITION BY customer_id ORDER BY order_date) AS
    prev_date
    FROM orders
)
SELECT customer_id, prev_date, order_date
FROM   gaps
WHERE  order_date - prev_date = 1;      -- date subtraction gives an integer in
    PostgreSQL
```

Interview Insight

LeetCode 180 (Consecutive Numbers) is the same idea: use two `LAG`s (or a self join) and check that three consecutive values are equal. `LAG(num,1)` and `LAG(num,2)` compared with `num` is the neatest solution and worth practising.

12.2 Running totals

Concept

Add `ORDER BY` inside `OVER` and an aggregate becomes **cumulative**. This is the single most surprising and useful behaviour in window functions.

`SUM(x) OVER (PARTITION BY g)`	the group total, same on every row
`SUM(x) OVER (PARTITION BY g ORDER BY d)`	the **running** total up to this row

Adding `ORDER BY` silently changes the meaning. Interviewers love asking why.

```
SELECT o.order_date,
       SUM(oi.quantity * oi.unit_price) AS order_value,
       SUM(SUM(oi.quantity * oi.unit_price)) OVER (ORDER BY o.order_date) AS
    running_total
FROM   orders o
JOIN   order_items oi ON o.order_id = oi.order_id
GROUP  BY o.order_date
ORDER  BY o.order_date;
```

```
order_date | order_value | running_total
-----------+-------------+---------------
2023-01-12 |    300000   |        300000
2023-02-20 |    179000   |        479000
2023-03-11 |    121000   |        600000
2023-04-05 |    106000   |        706000
...
```

Mental Model

The double `SUM(SUM(...))` looks bizarre but is exactly right, and understanding it proves you understand the processing order:

- the **inner** `SUM` is an aggregate; it runs at the `GROUP BY` stage and produces one value per date;
- the **outer** `SUM ... OVER` is a window function; it runs *after* grouping, over the already-aggregated rows.

Window functions always operate on the post-`GROUP BY` result. That is why this is legal while `AVG(AVG(x))` is not.

12.3 Window frames

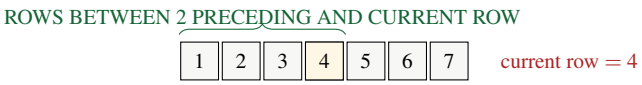
Concept

The frame says which rows around the current one the function sees. Default when you write `ORDER BY` inside `OVER`:

```
RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW     -- start of partition .. here
```

Common explicit frames:

`ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW`	running total (the default)
`ROWS BETWEEN 2 PRECEDING AND CURRENT ROW`	3-row moving window
`ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING`	centred 3-row window
`ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING`	the whole partition



Moving average

```
SELECT o.order_date,
       SUM(oi.quantity * oi.unit_price) AS daily_value,
       ROUND(AVG(SUM(oi.quantity * oi.unit_price)) OVER (
                 ORDER BY o.order_date
                 ROWS BETWEEN 2 PRECEDING AND CURRENT ROW), 2) AS moving_avg_3
FROM   orders o
JOIN   order_items oi ON o.order_id = oi.order_id
GROUP  BY o.order_date
ORDER  BY o.order_date;
```

Common Mistake

`ROWS` vs `RANGE`. `ROWS` counts physical rows; `RANGE` groups rows with the *same ORDER BY value* together. With duplicate dates, `RANGE ... CURRENT ROW` includes *all* rows sharing that date, while `ROWS` stops at this one. For moving averages you almost always want `ROWS`. This is a favourite "gotcha" question.

12.4 FIRST_VALUE, LAST_VALUE and the trap

```
SELECT name, dept_id, salary,
       FIRST_VALUE(name) OVER (PARTITION BY dept_id ORDER BY salary DESC) AS
   top_earner,
       LAST_VALUE(name)  OVER (PARTITION BY dept_id ORDER BY salary DESC
                               ROWS BETWEEN UNBOUNDED PRECEDING
                                        AND UNBOUNDED FOLLOWING) AS lowest_earner
FROM employees
WHERE dept_id IS NOT NULL;
```

Common Mistake

The `LAST_VALUE` trap. Without the explicit frame, `LAST_VALUE` uses the default frame ending at the *current row* — so "the last value" is just the current row's value, which is useless. You must widen the frame to `UNBOUNDED FOLLOWING`.
This catches almost everybody the first time. `FIRST_VALUE` works without the frame because the frame already starts at the partition beginning.

12.5 Guided Practice

Practice

D10-P1 🟢 Show each employee with the salary of the previously hired employee.
D10-P2 🟢 Show each order date with the next order date for the same customer.
D10-P3 🟡 Compute a running total of order values ordered by date.
D10-P4 🟡 For each employee show their salary and the difference from the previous hire's salary.
D10-P5 🟡 For each department, show each employee alongside the top earner's name.

12.6 Independent Practice

Practice

D10-P6 🟡 Compute a running total of salary per department, ordered by hire date.
D10-P7 🟡 Compute a 3-order moving average of order values.
D10-P8 🔴 For each customer, compute the number of days between consecutive orders.
D10-P9 🔴 Find customers whose latest order was smaller than their previous one.
D10-P10 🔴 For each department, show the top earner and the lowest earner on every employee row.
D10-P11 🔴 Compute each month's revenue in 2023 alongside the previous month's and the percentage change.

12.7 LeetCode Practice

Problem	Name	Diff.	Concept
1321	Restaurant Growth	🔴 Medium	7-day moving window — frames in practice
180	Consecutive Numbers	🟡 Medium	`LAG` twice, or a self join
1204	Last Person to Fit in the Bus	🔴 Medium	running total + filter
601	Human Traffic of Stadium	🔴 Hard	consecutive-run detection
626	Exchange Seats	🟡 Medium	`LAG`/`LEAD` + `CASE`

12.8 Interview Questions

1. What do `LAG` and `LEAD` do? Give a use case for each.
2. How do you turn `SUM` into a running total?
3. What is a window frame? What is the default?
4. `ROWS` vs `RANGE` — when does the difference matter?
5. Why does `LAST_VALUE` usually return the wrong answer?

Daily Quiz

1. What does `LAG(x)` return for the first row of a partition?
2. Which clause turns a window aggregate into a running total?
3. Write a frame for a centred 5-row moving average.
4. Why is `SUM(SUM(x)) OVER (...)` legal?
5. Which of `ROWS` / `RANGE` counts physical rows?
6. How do you make `LAST_VALUE` see the whole partition?
7. What does `LEAD(x, 2)` do?

Daily Revision Checklist

☐ `LAG`/`LEAD` with offset and default
☐ Running total = aggregate + `ORDER BY` inside `OVER`
☐ Frame syntax, and `ROWS` vs `RANGE`
☐ The `LAST_VALUE` frame trap
☐ I recognise "previous/next/consecutive/change" as `LAG` keywords

Chapter 13

Day 11 — Intermediate SQL Interview Patterns

Learning Objectives

Today you convert everything from Days 1–10 into *reusable templates*. By tonight you should recognise a problem type from its wording and write the skeleton before thinking about the details.

Mental Model

The translation table. Interview SQL questions are written in English, and the English contains keywords that map directly to SQL constructs. Learn to translate first, code second.

Phrase in the question	What it means in SQL
"for each . . . "	`GROUP BY` or `PARTITION BY`
"highest / largest / most"	`MAX`, or `ORDER BY ... DESC LIMIT`, or a ranking function
". . . and show the employee's name"	you need the *row*, not just the number ⇒ window function
"more than / at least N" after a count	`HAVING`
"who have never / with no"	anti-join: `NOT EXISTS` or `LEFT JOIN ... IS NULL`
"second / third / Nth"	`DENSE_RANK` or `OFFSET`
"compared with previous / consecutive"	`LAG` / `LEAD`
"running / cumulative"	aggregate + `ORDER BY` inside `OVER`
"duplicate"	`GROUP BY ... HAVING COUNT(*) > 1`
"percentage of"	conditional aggregation + a cast to avoid integer division

13.1 Pattern 1 — Find duplicates

```sql
-- which values are duplicated, and how often
SELECT email, COUNT(*) AS n
FROM   employees
GROUP  BY email
HAVING COUNT(*) > 1;
```

For *whole-row* duplicates, group by every column that defines identity. To see the duplicate rows themselves, rank them:

```sql
WITH d AS (
  SELECT emp_id, email,
         ROW_NUMBER() OVER (PARTITION BY email ORDER BY emp_id) AS rn
  FROM employees
)
SELECT * FROM d WHERE rn > 1;       -- every copy except the first
```

13.2 Pattern 2 — Remove duplicates (conceptually)

```
-- keep the lowest emp_id for each email, delete the rest
DELETE FROM employees a
USING  employees b
WHERE  a.email = b.email
  AND  a.emp_id > b.emp_id;
```

Interview Insight

LeetCode 196 is exactly this. The mental model: *"delete row A if there exists another row B with the same key and a smaller id."* The > is what makes it keep exactly one — the same "strictly less than" trick as in self joins.
Always say out loud that you would run the equivalent `SELECT` first to see what would be deleted. Interviewers like candidates who are careful with `DELETE`.

13.3 Pattern 3 — Second / Nth highest

```
-- Nth highest DISTINCT value (N = 2 here)
WITH r AS (
  SELECT DISTINCT salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS rk
  FROM employees
)
SELECT salary FROM r WHERE rk = 2;

-- shortcut, but counts rows not distinct values
SELECT DISTINCT salary FROM employees ORDER BY salary DESC LIMIT 1 OFFSET 1;
```

13.4 Pattern 4 — Top N per group

```
WITH ranked AS (
  SELECT e.*, DENSE_RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS r
  FROM employees e
)
SELECT * FROM ranked WHERE r <= 3;
```

Use `ROW_NUMBER` for "exactly N rows", `DENSE_RANK` for "N distinct levels, ties included".

13.5 Pattern 5 — Employees earning more than their manager

```
SELECT e.name AS employee, e.salary, m.name AS manager, m.salary AS manager_salary
FROM   employees e
JOIN   employees m ON e.manager_id = m.emp_id
WHERE  e.salary > m.salary;
```

The self join with two role-named aliases. LeetCode 181 is this exact query.

13.6 Pattern 6 — Customers with no orders (anti-join)

```
SELECT c.customer_name
FROM   customers c
WHERE  NOT EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id);
```

13.7 Pattern 7 — Consecutive records

```sql
-- three consecutive equal values
WITH t AS (
  SELECT id, num,
         LAG(num,1) OVER (ORDER BY id) AS p1,
         LAG(num,2) OVER (ORDER BY id) AS p2
  FROM logs
)
SELECT DISTINCT num FROM t WHERE num = p1 AND num = p2;
```

Mental Model

The gaps-and-islands trick. For consecutive *dates* or *integers*, the standard technique is to subtract a row number from the value: consecutive values produce a constant difference, so grouping by that difference isolates each run.

```sql
SELECT customer_id,
       order_date - (ROW_NUMBER() OVER (PARTITION BY customer_id
                                        ORDER BY order_date))::int AS grp
FROM orders;
-- rows in the same 'grp' form one consecutive streak
```

This is advanced, but it is the answer to a whole family of hard problems (LeetCode 601, 1225). Understand the idea even if you cannot write it from memory.

13.8 Pattern 8 — Missing records

```sql
-- dates with no orders: generate the calendar, then anti-join
SELECT d::date AS missing_day
FROM   generate_series('2023-01-01'::date, '2023-12-31'::date, '1 day') d
WHERE  NOT EXISTS (SELECT 1 FROM orders o WHERE o.order_date = d::date);
```

`generate_series` is PostgreSQL-specific and extremely handy: you cannot find missing rows by looking only at rows that exist, so you must manufacture the complete set first.

13.9 Pattern 9 — Maximum/minimum per group, with details

Three ways — know at least two:

```sql
-- (a) window function: clearest
WITH r AS (SELECT e.*, RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) rk
           FROM employees e)
SELECT * FROM r WHERE rk = 1;

-- (b) correlated subquery
SELECT * FROM employees e
WHERE salary = (SELECT MAX(salary) FROM employees e2 WHERE e2.dept_id = e.dept_id);

-- (c) PostgreSQL DISTINCT ON: shortest, dialect-specific
SELECT DISTINCT ON (dept_id) dept_id, name, salary
FROM employees ORDER BY dept_id, salary DESC;
```

Interview Insight

`DISTINCT ON` is a PostgreSQL exclusive and worth showing off *after* you have given the portable answer. It keeps the first row of each group as defined by `ORDER BY` — so the `ORDER BY` is not cosmetic, it selects the winner.

13.10 Pattern 10 — Comparing with the previous row

`LAG` (Day 10). Recognition keywords: “growth”, “change”, “since last”, “month-over-month”.

13.11 Pattern 11 — Date-based filtering

```
WHERE order_date >= '2023-01-01' AND order_date < '2024-01-01'      -- good: sargable
WHERE EXTRACT(YEAR FROM order_date) = 2023                          -- works, but no
    index use
WHERE order_date >= CURRENT_DATE - INTERVAL '30 days'               -- last 30 days
WHERE DATE_TRUNC('month', order_date) = '2023-06-01'                -- a whole month
```

Common Mistake

Wrapping a column in a function (`EXTRACT(YEAR FROM order_date) = 2023`) usually prevents the database from using an index on that column. Prefer a range condition. Saying this in an interview marks you as someone who has thought about performance.

13.12 Pattern 12 — Conditional aggregation and percentages

```
SELECT dept_id,
       COUNT(*) AS headcount,
       COUNT(*) FILTER (WHERE salary > 90000) AS high_earners,
       ROUND(100.0 * COUNT(*) FILTER (WHERE salary > 90000) / COUNT(*), 1) AS pct
FROM   employees
WHERE  dept_id IS NOT NULL
GROUP  BY dept_id;
```

Note `100.0` rather than `100` — that single decimal point forces floating-point division and is the difference between `33.3` and `0`.

13.13 Independent Practice

Practice

D11-P1 ● Find duplicate salaries and who holds them.
D11-P2 ● Find the second highest salary in each department.
D11-P3 ● Find employees earning more than their manager.
D11-P4 ● Find products never ordered.
D11-P5 ● Find the top 3 customers by revenue.
D11-P6 ● Find customers who ordered in consecutive months of 2023.
D11-P7 ● For each department, show the highest-paid employee’s name and salary.
D11-P8 ● Show monthly revenue for 2023 with month-over-month percentage change.
D11-P9 ● Find months in 2023 with no orders at all.
D11-P10 ● For each department, show the percentage of employees hired after 2020.
D11-P11 ● Find employees whose salary is in the top 25 percent of their department.
D11-P12 ● Find the customer with the largest single order, showing the order value and date.
D11-P13 ● For each project, show whether it is over or under the average project budget for its department.
D11-P14 ● List employees who work on more projects than the company average.

13.14 LeetCode Practice

Problem	Name	Diff.	Pattern
181	Employees Earning More Than Their Managers	Easy	self join
183	Customers Who Never Order	Easy	anti-join
182	Duplicate Emails	Easy	duplicate detection
596	Classes More Than 5 Students	Easy	`HAVING`
1084	Sales Analysis III	Easy	"all rows satisfy" via `HAVING`
1045	Customers Who Bought All Products	Medium	relational division

Daily Revision Checklist

- ☐ I can translate question-English into SQL constructs
- ☐ I have written all 12 patterns at least once
- ☐ I know two ways to do max-per-group
- ☐ I know why `100.0` matters
- ☐ I know the gaps-and-islands idea

Chapter 14

Day 12 — PostgreSQL-Specific SQL

Learning Objectives

By the end of today you can use PostgreSQL's data types, date/time handling, string functions, NULL helpers and casting — the subset that actually appears in backend and data interviews.

14.1 Data types worth knowing

`INTEGER` / `BIGINT`	Whole numbers. `INT` is 4 bytes (about ±2.1 billion); use `BIGINT` for ids that might exceed it.
`NUMERIC(p,s)`	Exact decimal. **Use this for money.** `NUMERIC(10,2)` means 10 digits total, 2 after the point.
`REAL` / `DOUBLE PRECISION`	Floating point. Fast, *inexact* — never use for currency.
`VARCHAR(n)`	Text with a length limit.
`TEXT`	Text with no limit. In PostgreSQL, `TEXT` and `VARCHAR` perform identically — unlike some other databases. Use `VARCHAR(n)` only when the limit is a real business rule.
`BOOLEAN`	`TRUE` / `FALSE` / `NULL`. Accepts `'t'`, `'yes'`, `1` on input.
`DATE`	Calendar date, no time.
`TIMESTAMP`	Date + time. `TIMESTAMPTZ` additionally stores a time zone — prefer it for anything real.
`INTERVAL`	A duration: `INTERVAL '3 days'`, `INTERVAL '2 hours 30 minutes'`.
`SERIAL`	Auto-incrementing integer (shorthand for `INTEGER` + a sequence).

Interview Insight

"Why not use FLOAT for money?" Because binary floating point cannot represent 0.1 exactly, so sums drift. `NUMERIC` stores decimal digits exactly. This is a very common interview question and a one-line answer earns real credit.

`SERIAL` vs `GENERATED AS IDENTITY`: `SERIAL` is the older PostgreSQL-specific form; `GENERATED ALWAYS AS IDENTITY` is the SQL-standard replacement and is preferred in new code. Knowing both exist is enough.

14.2 NULL helpers: COALESCE and NULLIF

```
COALESCE(a, b, c)    -- returns the first non-NULL argument
NULLIF(a, b)         -- returns NULL if a = b, otherwise a
```

```
SELECT name,
       COALESCE(email, 'no-email@corp.com') AS contact,
       COALESCE(manager_id, 0)              AS mgr
FROM employees WHERE emp_id = 20;
```

```
name     | contact              | mgr
---------+----------------------+-----
Omar Ali | no-email@corp.com    |   0
```

`NULLIF`'s main job is preventing division by zero:

```
SELECT total / NULLIF(count_value, 0) AS avg_value FROM t;
-- if count_value is 0, NULLIF makes it NULL, and x / NULL = NULL instead of an error
```

14.3 Casting

```
CAST(expr AS type)      -- SQL standard, portable
expr::type              -- PostgreSQL shorthand
```

```
SELECT 7 / 2               AS integer_division,    -- 3
       7::numeric / 2      AS real_division,       -- 3.5
       CAST('2023-05-01' AS DATE) AS d,
       '42'::int + 8       AS n;                   -- 50
```

Common Mistake

Integer division is the single most common PostgreSQL surprise. `5/2` is 2, not 2.5. Any percentage or ratio calculation needs a cast or a decimal literal:

```
ROUND(100.0 * a / b, 2)          -- the '100.0' does the work
ROUND(a::numeric / b, 2)         -- explicit cast
```

14.4 Date and time

`CURRENT_DATE`, `NOW()`	today; current timestamp
`AGE(d1, d2)`	the interval between two dates
`EXTRACT(YEAR FROM d)`	pull out a field: `YEAR`, `MONTH`, `DAY`, `DOW`, `QUARTER`
`DATE_TRUNC('month', ts)`	round down to the start of the month — the standard way to group by month
`d + INTERVAL '7 days'`	date arithmetic
`d2 - d1`	difference between two `DATE`s, in *days* (an integer)
`TO_CHAR(d, 'YYYY-MM')`	format a date as text

```
SELECT name,
       hire_date,
       EXTRACT(YEAR FROM hire_date)             AS hire_year,
       DATE_TRUNC('month', hire_date)::date     AS hire_month,
       AGE(CURRENT_DATE, hire_date)             AS tenure,
       CURRENT_DATE - hire_date                 AS days_employed,
       TO_CHAR(hire_date, 'Mon YYYY')           AS pretty
FROM employees
ORDER BY hire_date
LIMIT 3;
```

```
name          | hire_date  | hire_year | hire_month | tenure              |
    days_employed | pretty
--
    ----------+------------+-----------+------------+---------------------+---------------+------

Vikram Singh | 2014-06-01 |      2014 | 2014-06-01 | 11 years 2 mons ... |
    4089 | Jun 2014
Asha Nair    | 2015-03-01 |      2015 | 2015-03-01 | 10 years 5 mons ... |
    3816 | Mar 2015
Rohit Verma  | 2016-07-15 |      2016 | 2016-07-01 | 9 years 0 mons ...  |
    3314 | Jul 2016
```

14.5 String functions

`LENGTH(s)`		number of characters
`UPPER(s)` / `LOWER(s)` / `INITCAP(s)`		case conversion
`TRIM(s)`, `LTRIM`, `RTRIM`		strip whitespace
`SUBSTRING(s FROM 2 FOR 3)`	or	extract a slice
`SUBSTR(s,2,3)`		
`POSITION('a' IN s)`		index of a substring, 1-based, 0 if absent
`s1 || s2`		concatenation (`CONCAT()` also works and ignores NULLs)
`REPLACE(s, 'a', 'b')`		substitution
`SPLIT_PART(s, '@', 2)`		the nth field after splitting
`LEFT(s,n)` / `RIGHT(s,n)`		first/last n characters

```
SELECT name,
       UPPER(LEFT(name, 1)) || LOWER(SUBSTRING(name FROM 2)) AS proper,
       SPLIT_PART(email, '@', 1)  AS username,
       SPLIT_PART(email, '@', 2)  AS domain,
       LENGTH(name)               AS name_length
FROM employees WHERE email IS NOT NULL LIMIT 3;
```

Common Mistake

`'a' || NULL` is `NULL` — concatenation with `||` propagates NULL, which can silently blank out a whole computed column. `CONCAT('a', NULL)` returns `'a'` instead. When building display strings from nullable columns, use `CONCAT` or wrap in `COALESCE`.

14.6 Aggregating into lists: STRING_AGG and ARRAY_AGG

```
SELECT d.dept_name,
       COUNT(*) AS headcount,
       STRING_AGG(e.name, ', ' ORDER BY e.salary DESC) AS members,
       ARRAY_AGG(e.salary ORDER BY e.salary DESC)      AS salaries
FROM   employees e
JOIN   departments d ON d.dept_id = e.dept_id
GROUP  BY d.dept_name
ORDER  BY headcount DESC;
```

```
dept_name   | headcount | members                                          | salaries
--
    ----------+-----------+--------------------------------------------------+--------------------

Engineering |         7 | Asha Nair, Rohit Verma, Priya Sharma, ...  |
    {180000,120000,...}
Sales       |         5 | Vikram Singh, Neha Gupta, Arjun Desai, ... |
    {150000,92000,...}
HR          |         2 | Ananya Bose, Rahul Pillai                  | {85000,52000}
```

Interview Insight

`STRING_AGG` is the PostgreSQL answer to "list all the X for each Y in one row". MySQL calls it `GROUP_CONCAT`; SQL Server uses `STRING_AGG` too. The `ORDER BY` *inside* the aggregate is a nice touch that many candidates do not know exists.

14.7 Two more worth a mention

```
-- generate a series of dates (used for finding gaps)
SELECT generate_series('2023-01-01'::date, '2023-01-05'::date, '1 day')::date;

-- upsert: insert, or update if the key already exists
INSERT INTO departments (dept_id, dept_name, location, budget)
VALUES (1, 'Engineering', 'Bangalore', 6000000)
ON CONFLICT (dept_id) DO UPDATE SET budget = EXCLUDED.budget;
```

`ON CONFLICT` (upsert) is very commonly used in backend code, and being able to name it is a small but real signal in an SDE interview.

14.8 Independent Practice

Practice

D12-P1 ● Show every employee with their email domain, using `SPLIT_PART`.

D12-P2 ● Show each employee's contact, substituting `'N/A'` where the email is NULL.

D12-P3 ● Show each employee's tenure in whole years.

D12-P4 ● Group orders by month using `DATE_TRUNC` and count them.

D12-P5 ● Compute the percentage of employees in each department, correctly (no integer division).

D12-P6 ● For each department, produce a single comma-separated string of employee names ordered alphabetically.

D12-P7 ● Find orders placed in the last 400 days relative to `'2023-12-31'`.

D12-P8 ● Show each customer's name in title case with their signup month formatted as `'Mon YYYY'`.

D12-P9 ● Compute the average number of days between a customer's first and last order.

D12-P10 ● List all months of 2023 alongside the order count, including months with zero orders. *Hint:* `generate_series` *plus a* `LEFT JOIN`.

Daily Quiz

1. Why use `NUMERIC` rather than `FLOAT` for money?
2. What does `7/2` return, and how do you get 3.5?
3. What does `NULLIF(x, 0)` protect against?
4. Which function rounds a timestamp down to the start of its month?
5. What is the difference between `||` and `CONCAT` with NULLs?
6. What does `d2 - d1` return for two `DATE` values?
7. Name the PostgreSQL equivalent of MySQL's `GROUP_CONCAT`.

Daily Revision Checklist

☐ Types: `NUMERIC` for money, `TEXT` vs `VARCHAR`, `SERIAL`
☐ `COALESCE` and `NULLIF`
☐ Casting with `::` and the integer-division trap
☐ `EXTRACT`, `DATE_TRUNC`, `AGE`, `INTERVAL`
☐ `SPLIT_PART`, `STRING_AGG`, `generate_series`

Chapter 15

Day 13 — DBMS Interview Theory

Learning Objectives

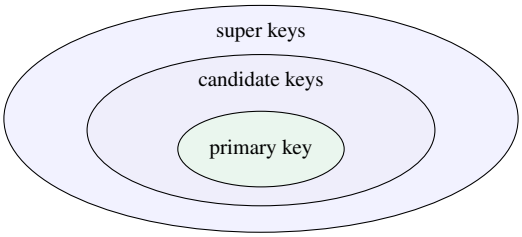
Today is spoken, not typed. By the end you can answer, out loud and in under 60 seconds each: keys, constraints, normalization to 3NF, indexes, transactions and ACID, and isolation levels. Aim for *correct and concise*, not exhaustive.

Interview Insight

How deep should a fresher go? Interviewers want to know that you understand *why* these mechanisms exist, not that you memorised definitions. A good answer is: one-sentence definition, one concrete example, one sentence on why it matters. Then stop talking. Rambling past the answer is the most common failure mode.

15.1 Keys

Super key	Any set of columns that uniquely identifies a row. `(emp_id)`, `(emp_id, name)`, `(emp_id, name, salary)` are all super keys.
Candidate key	A *minimal* super key — remove any column and it stops being unique. In `employees`: `emp_id` and `email` are both candidate keys.
Primary key	The candidate key you *chose* as the row's official identity. Exactly one per table; implies `NOT NULL` + `UNIQUE`.
Alternate key	The candidate keys you did not choose (`email` here).
Composite key	A key made of more than one column. `employee_projects` uses `(emp_id, project_id)`.
Foreign key	A column referencing another table's primary key. Enforces *referential integrity*.



Interview Insight

"PRIMARY KEY vs UNIQUE?" — extremely common. Answer: *"Both enforce uniqueness. A primary key also forbids NULLs and there can be only one per table; a `UNIQUE` constraint allows NULLs and you can have several. The primary key is also the default target for foreign keys."*
"PRIMARY KEY vs FOREIGN KEY?" *"A primary key identifies a row within its own table. A foreign key points at another table's primary key and enforces that the referenced row exists."*

15.2 Constraints

Constraint	Guarantees	From our schema
`NOT NULL`	the column always has a value	`name VARCHAR(80) NOT NULL`
`UNIQUE`	no two rows share the value (NULLs allowed, and multiple NULLs are permitted in PostgreSQL)	`email ... UNIQUE`
`PRIMARY KEY`	unique + not null	`emp_id SERIAL PRIMARY KEY`
`FOREIGN KEY`	the referenced row exists	`dept_id INT REFERENCES departments`
`CHECK`	an arbitrary boolean rule	`salary NUMERIC CHECK (salary > 0)`
`DEFAULT`	a value when none is supplied	`status VARCHAR DEFAULT 'PLACED'`

Referential integrity and what happens on delete

If you delete a department that employees point at, the database must do something. You choose what:

`ON DELETE RESTRICT / NO ACTION`	refuse the delete (the default)
`ON DELETE CASCADE`	delete the children too
`ON DELETE SET NULL`	keep the children, blank the reference

Interview Insight

A good line: *"`CASCADE` is convenient but dangerous — deleting one parent row can silently remove thousands of children. In production I would prefer `RESTRICT` and handle the cleanup explicitly, or use a soft-delete flag."*

15.3 Normalization

Concept

Normalization is organising columns into tables so that each fact is stored exactly once. The goal is to eliminate three anomalies:

- **Update anomaly** — the department is renamed, and you must change 7 rows; miss one and the data contradicts itself.
- **Insert anomaly** — you cannot record a new department until you hire someone into it.
- **Delete anomaly** — the last employee in a department leaves, and the department's budget and location vanish with them.

Unnormalized

```
emp_id | name   | dept_name   | dept_location | skills
-------+--------+-------------+---------------+------------------
     1 | Asha   | Engineering | Bangalore     | Java, SQL, Python
     2 | Rohit  | Engineering | Bangalore     | Java, Go
```

1NF — atomic values, no repeating groups

Every cell holds a single value. The `skills` column above breaks this.

```
employees                          employee_skills
emp_id | name   | dept_name |...    emp_id | skill
-------+--------+-----------        -------+--------
     1 | Asha   | Engineering            1 | Java
     2 | Rohit  | Engineering            1 | SQL
                                         1 | Python
```

2NF — 1NF, plus no partial dependency on part of a composite key

Only relevant when the primary key is composite. If `employee_projects` had a column `project_budget`, that depends only on `project_id` — half of the key `(emp_id, project_id)` — so it is a partial dependency and belongs in `projects`.

3NF — 2NF, plus no transitive dependency

A non-key column must not depend on another non-key column. In the unnormalized table, `dept_location` depends on `dept_name`, which depends on `emp_id`. That is transitive: move `dept_name` and `dept_location` into a `departments` table and leave a `dept_id` foreign key behind.

Mental Model

The one-line summary interviewers want:
"Every non-key attribute must depend on the key, the whole key, and nothing but the key."

- "the key" → 1NF
- "the *whole* key" → 2NF (no partial dependencies)
- "*nothing but* the key" → 3NF (no transitive dependencies)

Our practice schema is in 3NF. Being able to point at `departments` and say "location lives here because it depends on the department, not on the employee" is a complete answer.

Denormalization

Deliberately re-introducing redundancy for read performance — storing an `order_total` column instead of summing `order_items` every time, for example.

Normalized	Denormalized
No duplicated facts; writes are cheap and safe	Facts repeated; must be kept in sync
Reads need joins	Reads are fast, fewer joins
Typical for OLTP (transactional apps)	Typical for OLAP (reporting, data warehouses)

15.4 Indexes

Concept

An index is a separate data structure (in PostgreSQL, usually a B-tree) that lets the database find rows by a column value without scanning the whole table — exactly like the index at the back of a book. Without an index, finding `WHERE email = 'x'` is a **sequential scan**: $O(n)$. With one, it is a B-tree descent: roughly $O(\log n)$.

```
CREATE INDEX idx_employees_dept ON employees(dept_id);
CREATE UNIQUE INDEX idx_employees_email ON employees(email);
CREATE INDEX idx_emp_dept_salary ON employees(dept_id, salary);    -- composite
EXPLAIN ANALYZE SELECT * FROM employees WHERE dept_id = 1;         -- see the plan
```

Indexes help	Indexes cost
`WHERE` lookups, `JOIN` keys, `ORDER BY`, uniqueness enforcement	Extra disk space
Range scans on the indexed column	Every `INSERT`/`UPDATE`/`DELETE` must also update the index

Common Mistake

When an index is not used:

- the column is wrapped in a function — `WHERE LOWER(email) = 'x'` cannot use an index on `email` (you would need an index on `LOWER(email)`);
- a leading wildcard — `LIKE '%abc'`;
- the query returns most of the table anyway (a sequential scan is genuinely faster);
- a composite index on `(a, b)` does not help a query filtering only on `b` — leftmost-prefix rule.

Clustered vs non-clustered — conceptually

Clustered	The index *is* the table — rows are physically stored in index order. Only one per table, since rows can only be in one physical order.
Non-clustered	A separate structure holding the key plus a pointer to the row. Many per table.

Interview Insight

PostgreSQL nuance worth stating: PostgreSQL does not have clustered indexes in the SQL Server sense. All its indexes are secondary; the one-off `CLUSTER` command physically reorders a table once, but the order is not maintained. If asked, give the general concept and then add this — it shows you know the difference between the concept and your specific engine, which is exactly the depth expected of a fresher.

15.5 Transactions and ACID

Concept

A **transaction** is a group of statements treated as one indivisible unit: either all of it happens or none of it does.

```
BEGIN;
UPDATE accounts SET balance = balance - 1000 WHERE id = 1;
UPDATE accounts SET balance = balance + 1000 WHERE id = 2;
COMMIT;       -- or ROLLBACK to undo everything since BEGIN
```

Atomicity	All or nothing. If the second `UPDATE` fails, the first is undone. Money never disappears mid-transfer.
Consistency	The database moves from one valid state to another; all constraints hold at commit time.
Isolation	Concurrent transactions do not see each other's uncommitted work. The degree of protection is the *isolation level*.
Durability	Once committed, the change survives a crash — guaranteed by the write-ahead log (WAL).

15.4.1 Reading an EXPLAIN Plan, and the N+1 Problem

Addendum to Day 13 §15.4 (Indexes) — the workbook tells you to run EXPLAIN ANALYZE; this shows you how to read what comes back.

Plan node types worth recognizing

Node	What it means
Seq Scan	Reads the whole table row by row. Fine for small tables or filters that match most rows; a red flag on a large table with a selective filter.
Index Scan	Walks the index to find matching entries, then fetches each matching row from the table heap.
Index Only Scan	The index alone already contains every column the query needs — no heap fetch at all. The fastest option when it applies.
Bitmap Heap Scan	Builds a bitmap of matching pages from the index first, then reads those pages in physical order. Used for medium selectivity, where a plain Index Scan would jump around the disk too much.

In EXPLAIN ANALYZE output, compare the planner's **estimated** row count against the **actual** row count shown alongside it — a large gap between the two is the standard signal that table statistics are stale (fix with ANALYZE table_name;), not that the query itself is wrong.

The N+1 Query Problem

A performance bug that lives in application code, not in any single query, so it never shows up in an EXPLAIN of one statement: fetching a list of N parent rows, then looping in the application and issuing one further query per row to fetch related data — N+1 round trips instead of one JOIN or one batched WHERE id IN (...) query.

```
-- N+1: one query per order (bad)
SELECT id FROM orders WHERE customer_id = 42;
-- then, in a loop over each returned id:
SELECT * FROM order_items WHERE order_id = ?;

-- fixed: one batched query
SELECT * FROM order_items
WHERE order_id IN (SELECT id FROM orders WHERE customer_id = 42);
```

COMMON TRAP

This is invisible in EXPLAIN output precisely because each individual query looks perfectly fine in isolation — it only shows up as unexpectedly high total request latency or a suspiciously high query count in application-level logging/APM tooling. Common in ORMs, and in any pipeline that fetches a list then enriches each item with a per-item query (e.g. a document list followed by a metadata lookup per document).

Interview Insight

Use the bank-transfer example for every ACID question. It illustrates all four letters in one story, it takes 20 seconds, and it is the example the interviewer already has in their head.

15.6 Concurrency problems and isolation levels

Dirty read	You read a row another transaction has changed but not committed. If it rolls back, you acted on data that never existed.
Non-repeatable read	You read a row twice in one transaction and get different values, because someone committed an `UPDATE` in between.
Phantom read	You run the same `WHERE` twice and get a different *set of rows*, because someone committed an `INSERT`.

Isolation level	Dirty read	Non-repeatable	Phantom
READ UNCOMMITTED	possible	possible	possible
READ COMMITTED *(PostgreSQL default)*	prevented	possible	possible
REPEATABLE READ	prevented	prevented	possible (in the standard)
SERIALIZABLE	prevented	prevented	prevented

Interview Insight

Two PostgreSQL specifics that make a strong answer: (1) PostgreSQL's default is `READ COMMITTED`; (2) PostgreSQL does not actually implement `READ UNCOMMITTED` — asking for it gives you `READ COMMITTED`, because its MVCC design makes dirty reads impossible. Also, PostgreSQL's `REPEATABLE READ` prevents phantoms too, which is stricter than the standard requires.
The trade-off sentence to finish on: *"Higher isolation means fewer anomalies but more blocking or more serialization failures, so you pick the weakest level that is correct for the workload."*

Locking, briefly

Shared lock (read)	Several readers can hold it at once.
Exclusive lock (write)	Only one holder; blocks other writers.
Deadlock	Two transactions each hold a lock the other wants. PostgreSQL detects this and kills one with an error. Avoid it by always acquiring locks in a consistent order.
MVCC	PostgreSQL's approach: writers keep old row versions so that *readers never block writers and writers never block readers*. This is the single most quotable fact about PostgreSQL concurrency.

15.7 DDL / DML / DCL / TCL

DDL	`CREATE`, `ALTER`, `DROP`, `TRUNCATE`	defines structure
DML	`SELECT`, `INSERT`, `UPDATE`, `DELETE`	manipulates data
DCL	`GRANT`, `REVOKE`	permissions
TCL	`COMMIT`, `ROLLBACK`, `SAVEPOINT`	transaction control

	DELETE	TRUNCATE	DROP
Type	DML	DDL	DDL
Removes	selected rows	all rows	the whole table
`WHERE`	yes	no	n/a
Speed	slow (row by row, logged)	fast	fast
Rollback	yes	yes in PostgreSQL	yes in PostgreSQL
Structure kept	yes	yes	no

Interview Insight

Note the PostgreSQL detail: `TRUNCATE` and even `DROP TABLE` are transactional in PostgreSQL, so `BEGIN; DROP TABLE t; ROLLBACK;` restores the table. In Oracle and MySQL they are not. Most candidates recite "TRUNCATE cannot be rolled back" — being precise about your engine is a differentiator.

15.8 Views, procedures and functions

View	A stored query that behaves like a table. Stores no data; runs the underlying query each time. Used to simplify complex joins and to restrict column access.
Materialized view	A view whose result *is* stored on disk. Fast to read, must be refreshed (`REFRESH MATERIALIZED VIEW`). PostgreSQL-specific and worth naming.
Function	Returns a value; can be used inside a query. PostgreSQL's main mechanism.
Stored procedure	A named block of SQL invoked with `CALL`; can manage transactions internally. Added in PostgreSQL 11.

```sql
CREATE VIEW v_employee_details AS
SELECT e.emp_id, e.name, e.salary, d.dept_name, m.name AS manager
FROM   employees e
LEFT   JOIN departments d ON d.dept_id = e.dept_id
LEFT   JOIN employees   m ON m.emp_id  = e.manager_id;

SELECT * FROM v_employee_details WHERE dept_name = 'Engineering';
```

15.9 Self-test — say these out loud

Practice

Set a timer for 60 seconds each. Speak the answer; do not write it.

1. Primary key vs unique key vs candidate key.
2. What is referential integrity, and what are the `ON DELETE` options?
3. Explain 1NF, 2NF and 3NF using one example table.
4. When would you deliberately denormalize?
5. What is an index? Give two situations where it will not be used.
6. Clustered vs non-clustered — and what does PostgreSQL actually do?
7. Explain ACID with the bank-transfer example.
8. What is a dirty read? Which isolation level prevents it?
9. What is PostgreSQL's default isolation level?
10. What is a deadlock and how do you avoid one?
11. `DELETE` vs `TRUNCATE` vs `DROP`.
12. What is a view? What is a materialized view?

Daily Revision Checklist

- ☐ I can define all six key types with an example from our schema
- ☐ I can recite the "key, whole key, nothing but the key" summary
- ☐ I can explain indexes *and* their costs
- ☐ I can tell the ACID bank story in 20 seconds
- ☐ I know the four isolation levels and which anomalies each prevents
- ☐ I know two PostgreSQL-specific nuances to mention (MVCC, transactional DDL)

Chapter 16

Day 14 — LeetCode SQL 50 Intensive

Learning Objectives

Today is a problem-solving day, not a reading day. Work through the list below in order. Attempt each problem for **eight minutes** before reading the hint, and only look at a solution after you have submitted something — even something wrong.

Interview Insight

How to use the eight-minute rule. Set a timer. If you are stuck at eight minutes, read the "What to notice" line. If you are still stuck four minutes later, read the hint. If still stuck, skip it and come back tomorrow — do not read the answer key. A problem you solve after a night's sleep teaches you far more than a problem you read the solution to.

16.1 The complete SQL 50 map

Problems marked **(opt)** are optional if you are short on time — they repeat a concept already covered. Everything else is core.

16.1.1 Block 1 — SELECT and basic filtering (Days 1–2)

#	Name	Diff.	Concept
1757	Recyclable and Low Fat Products	Easy	`WHERE` + `AND`
584	Find Customer Referee	Easy	NULL trap
595	Big Countries	Easy	`WHERE` + `OR`
1148	Article Views I	Easy	`DISTINCT` + `ORDER BY`
1683	Invalid Tweets	Easy	`LENGTH`

16.1.2 Block 2 — JOINs (Days 5–6)

#	Name	Diff.	Concept
1378	Replace Employee ID With The Unique Identifier	Easy	`LEFT JOIN`
1068	Product Sales Analysis I	Easy	`INNER JOIN`
1581	Customer Who Visited but Did Not Make Any Transactions	Easy	anti-join
197	Rising Temperature	Easy	self join on dates
1661	Average Time of Process per Machine	Easy	self join + conditional agg.
577	Employee Bonus	Easy	`LEFT JOIN` + `IS NULL`
1280	Students and Examinations	Easy	`CROSS JOIN` + `LEFT JOIN`
570	Managers with at Least 5 Direct Reports	Medium	self join + `HAVING`
1934	Confirmation Rate	Medium	`LEFT JOIN` + `AVG(CASE)`

Interview Insight

1280 is the sleeper problem here. It needs a `CROSS JOIN` of students and subjects to manufacture every possible pair, then a `LEFT JOIN` onto the exam records. It is the only SQL 50 problem where you must *create* rows that do not exist. If you have never done this, spend the full time on it — "report zero for combinations with no data" is a very common real-world request.

16.1.3 Block 3 — Aggregation (Days 3–4)

#	Name	Diff.	Concept
620	Not Boring Movies	Easy	`WHERE` + modulo
1251	Average Selling Price	Easy	weighted average across a date range join
1075	Project Employees I	Easy	`AVG` + `ROUND`
1633	Percentage of Users Attended a Contest	Easy	ratio + cast
1211	Queries Quality and Percentage	Easy	conditional aggregation
1193	Monthly Transactions I	Medium	`DATE_TRUNC` + conditional agg.
1174	Immediate Food Delivery II	Medium	group-wise minimum
550	Game Play Analysis IV	Medium	first-date + next-day retention
1729	Find Followers Count	Easy	`GROUP BY` + `COUNT` **(opt)**
619	Biggest Single Number	Easy	`HAVING COUNT = 1`
1045	Customers Who Bought All Products	Medium	relational division

16.1.4 Block 4 — Subqueries and CTEs (Days 7–8)

#	Name	Diff.	Concept
1978	Employees Whose Manager Left the Company	Easy	`NOT IN` + NULL care
626	Exchange Seats	Medium	`CASE` + `LAG`/`LEAD`
1341	Movie Rating	Medium	two aggregations + `UNION ALL`
1321	Restaurant Growth	Medium	moving window
602	Friend Requests II	Medium	`UNION ALL` + ranking
585	Investments in 2016	Medium	multiple correlated conditions
185	Department Top Three Salaries	Hard	`DENSE_RANK` + partition

16.1.5 Block 5 — Window functions (Days 9–10)

#	Name	Diff.	Concept
1907	Count Salary Categories	Medium	`UNION ALL` for empty groups
1204	Last Person to Fit in the Bus	Medium	running total
180	Consecutive Numbers	Medium	`LAG` twice
184	Department Highest Salary	Medium	top-1 per group
178	Rank Scores	Medium	`DENSE_RANK`
177	Nth Highest Salary	Medium	`DENSE_RANK` / `OFFSET`
176	Second Highest Salary	Medium	N=2 special case

16.1.6 Block 6 — String, date and miscellaneous (Days 2, 11–12)

#	Name	Diff.	Concept
1667	Fix Names in a Table	Easy	`UPPER` + `SUBSTRING`
1527	Patients With a Condition	Easy	`LIKE` with a word-boundary trap
196	Delete Duplicate Emails	Easy	`DELETE` + self join
1873	Calculate Special Bonus	Easy	`CASE` + `LEFT`
627	Swap Salary	Easy	`CASE` in `UPDATE`
1517	Find Users With Valid E-Mails	Easy	regular expressions
182	Duplicate Emails	Easy	`HAVING` **(opt)**
181	Employees Earning More Than Their Managers	Easy	self join **(opt)**
183	Customers Who Never Order	Easy	anti-join **(opt)**
196	—	—	(listed above)
1084	Sales Analysis III	Easy	"all rows satisfy" via `HAVING`
1070	Product Sales Analysis III	Medium	group-wise minimum
1050	Actors and Directors Who Cooperated	Easy	`HAVING` **(opt)**
596	Classes More Than 5 Students	Easy	`HAVING` **(opt)**
1741	Find Total Time Spent by Each Employee	Easy	`GROUP BY` **(opt)**
175	Combine Two Tables	Easy	`LEFT JOIN` **(opt)**
1113	Reported Posts	Easy	simple CTE **(opt)**
601	Human Traffic of Stadium	Hard	consecutive runs

16.2 Today's target list

You will not finish all fifty today, and you should not try. If you have followed Days 1–12 you have already solved roughly 30 of them. Today, do these fifteen — they are the highest information-per-minute problems in the set.

	#	Name	Why this one
1	1280	Students and Examinations	the only "manufacture missing rows" problem
2	1934	Confirmation Rate	`LEFT JOIN` + `AVG(CASE)` in one step
3	570	Managers with 5 Direct Reports	self join + `HAVING`
4	1174	Immediate Food Delivery II	group-wise minimum, a very common shape
5	550	Game Play Analysis IV	the retention pattern, asked at product companies
6	1045	Customers Who Bought All Products	relational division — genuinely hard
7	626	Exchange Seats	`LAG`/`LEAD` with a parity `CASE`
8	1341	Movie Rating	two answers, `UNION ALL`, alphabetical tie-break
9	185	Department Top Three Salaries	the flagship window problem
10	1204	Last Person to Fit in the Bus	running total + filter
11	180	Consecutive Numbers	double `LAG`
12	177	Nth Highest Salary	write it two ways
13	1321	Restaurant Growth	frames in anger
14	1527	Patients With a Condition	the `LIKE` boundary trap
15	601	Human Traffic of Stadium	gaps and islands

16.3 What to notice, and hints

Read the "notice" line before you start. Read the "hint" only after eight minutes.

Practice

1280 Students and Examinations *Notice:* the answer must contain a row for every student–subject pair, even those with zero exams. No join of existing rows can produce a row that does not exist.

Hint: `students CROSS JOIN subjects`, then `LEFT JOIN` the exam table, then `COUNT` the exam id (not `*`).

1934 Confirmation Rate *Notice:* users with no confirmation messages must show 0.00, not NULL.
Hint: `LEFT JOIN`, then `ROUND(AVG(CASE WHEN action = 'confirmed' THEN 1 ELSE 0 END), 2)`. `AVG` of a 0/1 column *is* the rate.

570 Managers with at Least 5 Direct Reports *Notice:* you need the manager's *name*, which lives in the same table.
Hint: `GROUP BY managerId HAVING COUNT(*) >= 5`, then join back to `Employee` to get the name.

1174 Immediate Food Delivery II *Notice:* "first order" per customer, then a percentage over customers.
Hint: find each customer's minimum `order_date` first (subquery or `ROW_NUMBER`), then compute the ratio with a cast.

550 Game Play Analysis IV *Notice:* the denominator is *all* players, not just the retained ones.
Hint: get each player's first login date, then check whether a login exists at first date + 1 day. Divide by `COUNT(DISTINCT player_id)` over the whole table.

1045 Customers Who Bought All Products *Notice:* "all" is the giveaway — this is relational division.
Hint: `GROUP BY customer_id HAVING COUNT(DISTINCT product_key) = (SELECT COUNT(*) FROM Product)`.

626 Exchange Seats *Notice:* odd ids swap with the next, even ids with the previous, and a final odd id stays put.
Hint: `CASE` on `id % 2`, using `LEAD(student)` and `LAG(student)`; `COALESCE` handles the last row.

1341 Movie Rating *Notice:* two unrelated answers in one output, and both tie-break alphabetically.
Hint: two CTEs, `ORDER BY count DESC, name ASC LIMIT 1` each, then `UNION ALL`.

185 Department Top Three Salaries *Notice:* "top three salaries", not "top three employees" — ties count as one level.
Hint: `DENSE_RANK() OVER (PARTITION BY departmentId ORDER BY salary DESC)`, then filter `<= 3` in an outer query.

1204 Last Person to Fit in the Bus *Notice:* you need the last person whose *cumulative* weight stays under the limit.
Hint: running `SUM(weight) OVER (ORDER BY turn)`, filter `<= 1000`, then `ORDER BY turn DESC LIMIT 1`.

180 Consecutive Numbers *Notice:* ids may have gaps in some variants — read the constraints.
Hint: `LAG(num,1)` and `LAG(num,2)` over `ORDER BY id`; keep rows where all three are equal; `DISTINCT` the result.

177 Nth Highest Salary *Notice:* must return NULL when there is no Nth salary — an empty result set is not the same as NULL.
Hint: wrap the query so it always produces one row, or use `(SELECT DISTINCT salary ... OFFSET N-1 LIMIT 1)` as a scalar subquery.

1321 Restaurant Growth *Notice:* the window is 7 *days*, and the first six days are excluded from the output.
Hint: aggregate to one row per day first, then `ROWS BETWEEN 6 PRECEDING AND CURRENT ROW`, then drop the first six rows.

1527 Patients With a Condition *Notice:* `LIKE '%DIAB1%'` wrongly matches `'ABDIAB100'`.
Hint: match either the start of the string or a space before it: two conditions joined with `OR`.

601 Human Traffic of Stadium *Notice:* runs of three or more consecutive ids, all with people ≥ 100.
Hint: filter first, then `id - ROW_NUMBER() OVER (ORDER BY id)` is constant within a run; group by that value and keep groups of size ≥ 3.

16.4 Review protocol

Spend the last 30 minutes here. For every problem you got wrong or looked up:

1. Write one sentence naming the *pattern* (anti-join, top-N-per-group, running total, relational division…).
2. Write one sentence on what you missed — usually it is one of: a NULL case, a tie case, an empty-group case, or integer division.
3. Add the problem number to a "redo" list and re-solve it tomorrow morning *before* the mock.

Interview Insight

The pattern name is the thing you carry into the interview, not the query. If you can say "that is a top-N-per-group, so I'll rank in a CTE and filter outside" before writing anything, you will look — and be — far more competent than someone who starts typing immediately.

Daily Revision Checklist

☐ I solved at least 12 of today's 15
☐ I named the pattern for each one
☐ I have a redo list for tomorrow
☐ I got 1280 and 1045 (the two conceptually unusual ones)
☐ I wrote 177 two different ways

Chapter 17

Day 15 — Mock SQL Placement Interview

Learning Objectives

A full, timed assessment. Work through Sections A–E without notes, then grade yourself with the rubric at the end and read the weak-area diagnosis.

Interview Insight

Rules. Two hours, no notes, no internet, no LeetCode. Write every query in a text file first, then run it against your practice database to check. You are allowed to look up *function names* (nobody memorises `DATE_TRUNC`'s argument order) but not *patterns*. Grading is in Section F; solutions are in the Answer Key, Chapter 21.

Section A — SQL Coding

10 problems, 50 marks, 60 minutes

Practice

A1 ● (3 marks) List the names and salaries of all employees in the Engineering department, highest paid first.

A2 ● (3 marks) Count how many customers signed up in each country. Show only countries with at least two customers.

A3 ● (4 marks) List every product together with the total quantity ever ordered. Products never ordered must appear with 0.

A4 ● (5 marks) For each department, show the department name, headcount, and average salary rounded to two decimals. Exclude employees with no department.

A5 ● (5 marks) Find all employees who earn more than the average salary of their own department. Show name, department name, salary and the department average.

A6 ● (5 marks) Find the second highest distinct salary in the company. Your query must return NULL rather than an empty result if it does not exist.

A7 ● (5 marks) For each customer, show total revenue and the number of orders, including customers who have never ordered (revenue 0).

A8 ● (6 marks) Find the top two highest-paid employees in each department. Ties at the same salary should both be included.

A9 ● (7 marks) Produce a monthly report for 2023: month, number of orders, revenue, and the percentage change in revenue from the previous month.

A10 ● (7 marks) Find every pair of employees who work on the same project, showing project name and both names. Each pair must appear only once, and an employee must not be paired with themselves.

Section B — SQL Concept Questions

20 questions, 20 marks, 20 minutes

Practice

One mark each. Answer in one or two sentences — brevity is being tested too.

1. What is the logical processing order of a `SELECT` statement?
2. `WHERE` vs `HAVING`.
3. `COUNT(*)` vs `COUNT(column)`.
4. Why does `WHERE col = NULL` return nothing?
5. `INNER JOIN` vs `LEFT JOIN`.
6. What is a Cartesian product and how do you get one accidentally?
7. `UNION` vs `UNION ALL`.
8. `RANK` vs `DENSE_RANK` vs `ROW_NUMBER`.
9. What does `PARTITION BY` do?
10. Why can't you filter a window function in `WHERE`?
11. `IN` vs `EXISTS` — give one semantic difference.
12. Why is `NOT IN` dangerous with subqueries?
13. What is a correlated subquery?
14. What is a CTE, and how does it differ from a view?
15. What does `DISTINCT` deduplicate?
16. Why can you use an alias in `ORDER BY` but not `WHERE`?
17. Is `BETWEEN` inclusive?
18. What does `COALESCE` do?
19. Why does `7/2` return 3 in PostgreSQL, and how do you fix it?
20. How do you turn `SUM` into a running total?

Section C — DBMS Questions

20 questions, 20 marks, 20 minutes

Practice

1. Primary key vs unique key.
2. What is a candidate key?
3. What is a composite key? Give an example from our schema.
4. What is referential integrity?
5. Name three `ON DELETE` behaviours.
6. What problem does normalization solve? Name the three anomalies.
7. State 1NF, 2NF, 3NF in one line each.
8. When would you denormalize?
9. What is an index and what does it cost?
10. Give two situations where an index will not be used.
11. Clustered vs non-clustered index. What does PostgreSQL actually do?
12. What is a transaction?
13. Explain ACID with one example.
14. `COMMIT` vs `ROLLBACK`.
15. What is a dirty read?
16. What is a phantom read?
17. Name the four isolation levels in order of strictness.
18. What is PostgreSQL's default isolation level?
19. What is a deadlock and how do you avoid one?
20. `DELETE` vs `TRUNCATE` vs `DROP`.

Section D — Query Debugging

5 queries, 15 marks, 15 minutes

Practice

Each query below is wrong. State *what* is wrong, *why* it produces the wrong result (or fails), and write the corrected query. 3 marks each.

D1

```
SELECT dept_id, name, AVG(salary) AS avg_sal
FROM employees
GROUP BY dept_id;
```

D2

```
SELECT name, salary
FROM employees
WHERE COUNT(*) > 1
GROUP BY name, salary;
```

D3

```
SELECT c.customer_name, o.order_id
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
WHERE o.status = 'DELIVERED';
-- intended: every customer, with their delivered orders if any
```

D4

```
SELECT name FROM employees
WHERE emp_id NOT IN (SELECT manager_id FROM employees);
-- intended: employees who manage nobody
```

D5

```
SELECT name, dept_id,
       RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS r
FROM employees
WHERE r <= 3;
```

Section E — Schema Design

15 marks, 25 minutes

Practice

Requirement. Design the database for a small online course platform.

- Students register with a name, email (unique) and signup date.
- Instructors have a name and a specialisation.
- A course has a title, a description, a price, and exactly one instructor. An instructor may teach many courses.
- A student can enrol in many courses; a course has many students. Record the enrolment date and the student's progress percentage.
- Each course is made of many lessons, each with a title and a duration in minutes.
- A student can leave at most one review per course, with a rating from 1 to 5 and optional text.

Deliver:

1. (8 marks) `CREATE TABLE` statements with primary keys, foreign keys and at least four meaningful constraints.
2. (3 marks) Identify every relationship as one-to-many or many-to-many, and say which tables are

junction tables.
3. (2 marks) Name two indexes you would add and justify each.
4. (2 marks) Write one query: the top 3 courses by average rating, showing course title, instructor name, average rating and review count — only counting courses with at least 2 reviews.

Section F — Scoring and Diagnosis

Score sheet

Section	Content	Marks	Your score
A	SQL coding (10 problems)	50	_____
B	SQL concepts (20 questions)	20	_____
C	DBMS theory (20 questions)	20	_____
D	Query debugging (5 queries)	15	_____
E	Schema design	15	_____
	Total	**120**	_____

Marking guidance for Section A

Award partial credit honestly — you are trying to find weak spots, not to feel good.

Full marks	Correct result, correct edge cases (NULLs, ties, empty groups), readable.
Minus 1	Correct result but clumsy: unnecessary subquery, missing alias, no `ORDER BY` where the question implied one.
Minus 2	Correct on the sample data but breaks on an edge case (fails on NULL, drops ties, integer division).
Zero	Wrong result or does not run.

What your score means

Score	Band	Interpretation
100–120	Strong	Comfortably placement ready. You would handle an SQL round at most product companies. Spend remaining time on speed and on explaining your reasoning aloud.
85–99	**Placement ready**	You can pass a typical fresher SQL round. Fix the specific gaps the diagnosis below identifies; do not re-study everything.
70–84	Nearly there	The fundamentals are solid but something specific is missing — usually window functions or edge cases. Two focused days should close it.
55–69	Developing	You can write basic queries but struggle to combine concepts. Redo Days 5–6 and 9–10, then retake this mock.
Below 55	Foundations	Do not push forward. Repeat Days 1–8 with the practice problems, then retake. Rushing past this point wastes the later material.

Interview Insight

The 85 threshold is deliberate. A typical fresher SQL round is 2–3 coding questions plus 5–10 concept questions. Someone scoring 85 here gets the coding questions right and can answer most concepts crisply — which is what passing looks like. Scoring 120 is not required, and chasing it has diminishing returns compared with practising *speaking* your answers.

Weak-area diagnosis

Find your lowest-scoring pattern below and follow the prescription. Do not re-read the whole book.

Symptom	Root cause	Prescription
Lost marks on A3, A7 (missing rows, NULL revenue)	You default to `INNER JOIN` and forget `COALESCE`	Redo Day 5, especially the `ON` vs `WHERE` box. LeetCode 1280, 577, 1934.
Lost marks on A5, A8, A9	Window functions not automatic yet	Redo Days 9–10 completely. LeetCode 185, 184, 1321. This is the highest-value fix.
Lost marks on A6 (empty vs NULL)	Not thinking about edge cases	Practise stating three edge cases aloud before writing any query: NULLs, ties, empty groups.
Lost marks on A9 (percentages wrong)	Integer division	Re-read Day 12 casting. Always write `100.0 *` or `::numeric`.
Lost marks on A10	Self joins	Redo Day 6 pattern 2 and the `a.id < b.id` trick.
Section B below 14	You can write SQL but not explain it	Read Chapter 19 and say every answer out loud once. This is fast to fix and disproportionately valuable.
Section C below 14	DBMS theory thin	Redo Day 13's self-test with a timer. Theory is the cheapest marks in any interview.
Section D below 10	You do not read queries critically	Re-do Section D after a week; then deliberately break your own working queries and diagnose them.
Section E below 10	No design practice	Design two more schemas from scratch (a library, a food-delivery app). Focus on identifying many-to-many relationships.

After the mock

1. Grade honestly today, while it is fresh.
2. Write your three weakest areas on one line each.
3. Spend the next three days on those three only.
4. Retake this mock in a week. A second attempt scoring 15–20 marks higher is normal and means the gaps were knowledge, not ability.
5. In the week before real interviews, re-read only Chapter 18 and Chapter 20.

Daily Revision Checklist

☐ I completed all five sections under timed conditions
☐ I graded myself honestly, including partial credit

☐ I identified my three weakest areas
☐ I have a specific plan for each, not “revise SQL”
☐ I scheduled a retake

Chapter 18

SQL Problem-Solving Patterns

Mental Model

How to use this chapter. Interview SQL is not creative work — it is *recognition*. There are roughly seventeen shapes, and almost every question is one of them, or two of them combined. Your job in the first thirty seconds of a problem is not to write SQL; it is to name the shape.
For each pattern below: *how to recognise it* → *how to think about it* → *the skeleton* → *an example* → *problems to practise*.

18.1 Pattern 1 — Simple filtering

Recognise: "find all X where…", a single table, no aggregation.
Think: which rows qualify? Watch for NULLs in the filtered column.

```
SELECT cols FROM t WHERE condition ORDER BY col;
```

Practice: LC 1757, 595, 584, 1683.

18.2 Pattern 2 — Whole-table aggregation

Recognise: "how many", "what is the total/average", one number expected.
Think: the result is one row. Decide whether NULLs should be counted.

```
SELECT COUNT(*), SUM(x), AVG(x) FROM t WHERE condition;
```

Practice: LC 620, 1741.

18.3 Pattern 3 — Group-wise aggregation

Recognise: the phrase **"for each"** followed by a summary word.
Think: what is one row of the output? That is your `GROUP BY` key. Then: does the filter apply before grouping (`WHERE`) or after (`HAVING`)?

```
SELECT group_col, COUNT(*), AVG(x)
FROM   t
WHERE  row_level_filter
GROUP  BY group_col
HAVING group_level_filter;
```

Example: employees per department with average salary.
Practice: LC 596, 1075, 1729, 1050.

18.4 Pattern 4 — Join

Recognise: the question mentions attributes from two entities ("employee name *and* department name").
Think: find the foreign key — it is your `ON` clause. Then decide inner or outer by asking: *must every row of the main table appear even without a match?*

```
SELECT a.col, b.col
FROM   a
JOIN   b ON a.fk = b.pk;          -- or LEFT JOIN to keep unmatched rows of a
```

Practice: LC 175, 1068, 1378.

18.5 Pattern 5 — Anti-join ("never", "no", "without")

Recognise: "customers who never ordered", "students with no exams", "products not sold".

Think: you want rows in A with *no* partner in B. Three formulations; prefer `NOT EXISTS`.

```
-- preferred
SELECT * FROM a WHERE NOT EXISTS (SELECT 1 FROM b WHERE b.fk = a.pk);

-- equally common in interviews
SELECT a.* FROM a LEFT JOIN b ON b.fk = a.pk WHERE b.pk IS NULL;

-- avoid unless you are sure the subquery has no NULLs
SELECT * FROM a WHERE a.pk NOT IN (SELECT fk FROM b);
```

Practice: LC 183, 1581, 577.

18.6 Pattern 6 — Self join

Recognise: the question compares rows of the *same* table — manager and report, this row and yesterday's row, pairs of employees.

Think: give the table two role names and write the relationship as an English sentence. Use `a.id < b.id` to avoid self-pairs and mirrored duplicates.

```
SELECT e.name AS employee, m.name AS manager
FROM   employees e JOIN employees m ON e.manager_id = m.emp_id;
```

Practice: LC 181, 570, 197, 1661.

18.7 Pattern 7 — Subquery for a benchmark

Recognise: "more than the average", "above the company maximum", "compared with the overall…".

Think: `WHERE` cannot hold an aggregate, but it can hold a subquery that *returns* one.

```
SELECT * FROM t WHERE x > (SELECT AVG(x) FROM t);
```

If the benchmark is *per group*, correlate it — or use a window function, which is usually clearer:

```
SELECT * FROM t WHERE x > (SELECT AVG(x) FROM t t2 WHERE t2.g = t.g);   -- correlated
```

Practice: LC 176, 585.

18.8 Pattern 8 — EXISTS (membership test)

Recognise: "customers who have at least one…", "users who did X". You need no columns from the other table.

Think: a yes/no question per row. `EXISTS` short-circuits and is NULL-safe.

```
SELECT * FROM a WHERE EXISTS (SELECT 1 FROM b WHERE b.fk = a.pk AND b.cond);
```

Practice: LC 1978, 1084.

18.9 Pattern 9 — Top N

Recognise: "the highest", "top 5", "most expensive".

Think: sort and cut — but ask whether ties should all be included. If yes, use `RANK`/`DENSE_RANK` instead of `LIMIT`.

```
SELECT * FROM t ORDER BY x DESC LIMIT 5;                    -- exactly 5 rows
-- ties included:
WITH r AS (SELECT *, RANK() OVER (ORDER BY x DESC) rk FROM t)
SELECT * FROM r WHERE rk <= 5;
```

18.10 Pattern 10 — Nth highest

Recognise: "second highest salary", "third most popular".

Think: distinct values or distinct rows? That decides `DENSE_RANK` vs `OFFSET`. Also: what should happen if there is no Nth value?

```
WITH r AS (SELECT DISTINCT x, DENSE_RANK() OVER (ORDER BY x DESC) rk FROM t)
SELECT x FROM r WHERE rk = N;
```

Practice: LC 176, 177.

18.11 Pattern 11 — Ranking within groups / Top N per group

Recognise: "for each department, the top 3…" — "for each" *plus* a superlative *plus* you need row details.

Think: this is the single most common medium-difficulty interview question. Rank in a CTE, filter outside, because `WHERE` cannot see a window function.

```
WITH ranked AS (
  SELECT t.*, DENSE_RANK() OVER (PARTITION BY g ORDER BY x DESC) AS rk
  FROM t
)
SELECT * FROM ranked WHERE rk <= N;
```

Practice: LC 185, 184, 1070.

18.12 Pattern 12 — Running total / cumulative

Recognise: "running", "cumulative", "to date", "until the bus is full".

Think: an aggregate with `ORDER BY` inside `OVER`.

```
SELECT d, x, SUM(x) OVER (ORDER BY d) AS running_total FROM t;
```

For moving windows, add a frame: `ROWS BETWEEN 6 PRECEDING AND CURRENT ROW`.

Practice: LC 1204, 1321.

18.13 Pattern 13 — Previous / next row

Recognise: "compared with the previous", "change from last month", "consecutive".

```
SELECT d, x,
       LAG(x)  OVER (PARTITION BY g ORDER BY d) AS prev,
       x - LAG(x) OVER (PARTITION BY g ORDER BY d) AS change
FROM t;
```

Practice: LC 180, 626, 197.

18.14 Pattern 14 — Conditional aggregation

Recognise: one output row needs several *differently filtered* numbers — "count total, and how many were cancelled, and the percentage".

Think: you cannot use `WHERE`, because it would filter all the columns. Put the condition inside the aggregate.

```
SELECT g,
       COUNT(*)                                  AS total,
       COUNT(*) FILTER (WHERE cond)              AS matching,
       SUM(CASE WHEN cond THEN 1 ELSE 0 END)     AS matching_portable,
       ROUND(100.0 * COUNT(*) FILTER (WHERE cond) / COUNT(*), 1) AS pct
FROM t GROUP BY g;
```

Practice: LC 1934, 1193, 1211, 1174.

18.15 Pattern 15 — Duplicate detection

Recognise: "find duplicates", "emails appearing more than once".

```
SELECT key_col, COUNT(*) FROM t GROUP BY key_col HAVING COUNT(*) > 1;

-- to see the duplicate ROWS, and to delete all but one:
WITH d AS (SELECT id, ROW_NUMBER() OVER (PARTITION BY key_col ORDER BY id) rn FROM t)
SELECT * FROM d WHERE rn > 1;
```

Practice: LC 182, 196, 619.

18.16 Pattern 16 — Missing records

Recognise: "months with no sales", "students who took no exam in a subject", "gaps in the sequence".

Think: you cannot find a missing row by looking at existing rows. *Manufacture* the complete set first (`generate_series`, or a `CROSS JOIN` of the two dimensions), then `LEFT JOIN` the facts onto it.

```
SELECT cal.d, COUNT(o.order_id) AS orders
FROM   generate_series('2023-01-01'::date, '2023-12-01'::date, '1 month') AS cal(d)
LEFT   JOIN orders o ON DATE_TRUNC('month', o.order_date) = cal.d
GROUP  BY cal.d ORDER BY cal.d;
```

Practice: LC 1280, 1907.

18.17 Pattern 17 — Consecutive records (gaps and islands)

Recognise: "three or more in a row", "consecutive days", "streaks".

Think: for a short fixed length, use `LAG` twice. For an arbitrary-length run, use the subtraction trick: *value minus row number is constant within a run*.

```
WITH f AS (SELECT id, ROW_NUMBER() OVER (ORDER BY id) rn FROM t WHERE cond)
SELECT MIN(id), MAX(id), COUNT(*)
FROM   (SELECT id, id - rn AS grp FROM f) g
GROUP  BY grp
HAVING COUNT(*) >= 3;
```

Practice: LC 180, 601, 1225.

18.18 The recognition table — one page

If the question says…	Pattern	Key construct
"find all X where…"	filtering	`WHERE`
"how many / total / average" (one number)	aggregation	`COUNT/SUM/AVG`
"for each X, how many…"	group-wise aggregation	`GROUP BY`
"…having more than N"	group filter	`HAVING`
"name from table A and name from table B"	join	`JOIN ... ON`
"who never / with no / without"	anti-join	`NOT EXISTS`
"at least one"	membership	`EXISTS`
"more than the average"	benchmark subquery	scalar subquery
"employees and their managers"	self join	two aliases
"the highest / top 5"	top N	`ORDER BY ... LIMIT`
"second / Nth highest"	Nth highest	`DENSE_RANK`
"for each group, the top N"	rank in group	`PARTITION BY` + CTE
"running / cumulative / to date"	running total	`SUM() OVER (ORDER BY)`
"compared with previous / consecutive"	neighbour	`LAG` / `LEAD`
"count of X and count of Y in one row"	conditional aggregation	`FILTER` / `CASE`
"duplicates"	duplicate detection	`HAVING COUNT(*) > 1`
"months with no sales"	missing records	`generate_series` + `LEFT JOIN`
"three in a row / streak"	gaps and islands	`id - ROW_NUMBER()`
"bought all products"	relational division	`HAVING COUNT(DISTINCT) = total`

Interview Insight

The thirty-second routine. Before writing anything:

1. What is *one row* of the answer? (This gives you `GROUP BY` or the grain.)
2. Which tables, and what connects them?
3. Which pattern from the table above is this?
4. What are the edge cases — NULLs, ties, empty groups, integer division?

Say these out loud in an interview. Interviewers score communication, and this routine also prevents the three most common mistakes before you have typed a character.

Chapter 19

Top 50 SQL & DBMS Interview Questions

Interview Insight

How to use this chapter. Read the question, answer it *out loud*, then read the model answer. Answering silently in your head feels like knowing; it is not the same skill as speaking under pressure.
Every answer below is deliberately short. A crisp two-sentence answer followed by silence scores better than a rambling one — and if the interviewer wants more, they will ask.

19.1 SQL language questions (1–25)

1. What is the logical processing order of a SELECT? `FROM` → `WHERE` → `GROUP BY` → `HAVING` → `SELECT` → `DISTINCT` → `ORDER BY` → `LIMIT`. Window functions run between `SELECT` and `ORDER BY`. This order explains why aliases work in `ORDER BY` but not `WHERE`.

2. WHERE vs HAVING? `WHERE` filters individual rows *before* grouping; `HAVING` filters groups *after* aggregation. `WHERE` cannot contain an aggregate because aggregates do not exist yet at that stage.

3. COUNT(*) vs COUNT(column) vs COUNT(DISTINCT column)? `COUNT(*)` counts rows; `COUNT(col)` counts non-NULL values in that column; `COUNT(DISTINCT col)` counts distinct non-NULL values. They differ exactly when NULLs or duplicates are present.

4. What is NULL? An unknown or missing value — not zero, not an empty string. Any comparison with NULL yields UNKNOWN, so you must test with `IS NULL` / `IS NOT NULL`.

5. Why does `WHERE col = NULL` return nothing? Because `col = NULL` evaluates to UNKNOWN, and `WHERE` keeps only rows where the condition is TRUE.

6. How do aggregates treat NULLs? They ignore them. `AVG` divides by the count of non-NULL values, and `SUM` over an all-NULL column returns NULL, not 0. `COUNT(*)` is the exception — it counts rows.

7. INNER JOIN vs LEFT JOIN? `INNER` keeps only matching pairs. `LEFT` keeps every left row, filling the right columns with NULL when there is no match. Use `LEFT` whenever "show all X, even those with no Y".

8. What is a FULL OUTER JOIN? Keeps unmatched rows from both sides, NULL-padding whichever side is missing. Used for reconciling two datasets.

9. What is a CROSS JOIN? Every combination of rows from both tables, with no `ON` clause. Deliberate uses include generating all date–product pairs; accidental ones are the classic Cartesian-product bug.

10. What is a self join? A table joined to itself using two aliases, for comparing rows within one table — employee and manager, or pairs of rows. Use `a.id < b.id` to avoid self-pairs and mirrored duplicates.

11. Condition in ON vs in WHERE for an outer join? A condition on the optional table belongs in `ON`. Putting it in `WHERE` discards the NULL-padded rows, silently turning a `LEFT JOIN` into an `INNER JOIN`.

12. UNION vs UNION ALL? Both stack result sets vertically and require the same column count and compatible types. `UNION` removes duplicates (and therefore sorts, which costs time); `UNION ALL` keeps everything and is faster. Default to `UNION ALL` unless you need deduplication.

13. JOIN vs UNION? `JOIN` combines tables *horizontally* (adds columns); `UNION` combines them *vertically* (adds rows).

14. What is a subquery? What is a correlated subquery? A subquery is a `SELECT` inside another statement. It is *correlated* when it references a column from the outer query, which means it conceptually re-runs for each outer row.

15. IN vs EXISTS? `IN` compares against a list of values; `EXISTS` tests whether any matching row exists and can stop at the first one. PostgreSQL often plans both identically, so the real difference is semantic: `NOT`

`IN` breaks when the subquery returns NULL, while `NOT EXISTS` does not.

16. Why is NOT IN dangerous? If the value list contains a NULL, `x NOT IN (\ldots )` is never TRUE, so the query returns zero rows. Use `NOT EXISTS`, or filter NULLs out inside the subquery.

17. JOIN vs subquery — when to use which? Use a join when you need columns from both tables. Use a subquery when you only need a test (`EXISTS`) or a single benchmark value, or when a join would duplicate rows.

18. What is a CTE? A named temporary result set defined with `WITH`, valid for one statement. It improves readability by letting you name each step, and it lets you reference the same intermediate result more than once.

19. CTE vs subquery vs view? A subquery is inline and anonymous; a CTE is inline and named; a view is stored in the database and reusable across statements. None of them store data (except a materialized view).

20. What is a recursive CTE? A CTE that references itself, written as an anchor query `UNION ALL` a recursive query. Used for hierarchies (org charts, category trees) and sequence generation. It needs a condition that eventually stops producing new rows.

21. What is a window function? A function that computes a value across a set of related rows *without collapsing them*. Unlike `GROUP BY`, every input row survives in the output.

22. RANK vs DENSE_RANK vs ROW_NUMBER? With values 100, 90, 90, 80: `ROW_NUMBER` gives 1, 2, 3, 4; `RANK` gives 1, 2, 2, 4 (a gap); `DENSE_RANK` gives 1, 2, 2, 3 (no gap). Use `ROW_NUMBER` to pick exactly one row per group, `DENSE_RANK` for "Nth distinct value".

23. What does PARTITION BY do? It restarts the window calculation for each distinct value, like `GROUP BY` but without merging rows.

24. Why can't you filter a window function in WHERE? Window functions are evaluated after `WHERE` and `GROUP BY`. Wrap the query in a CTE or subquery and filter in the outer query.

25. How do you get a running total? Add `ORDER BY` inside the `OVER` clause: `SUM(x) OVER (ORDER BY d)`. Without `ORDER BY` you get the partition total on every row instead.

19.2 More SQL questions (26–35)

26. What does DISTINCT deduplicate? The entire output row, not one column. `SELECT DISTINCT a, b` returns distinct *pairs*.

27. Is BETWEEN inclusive? Yes, on both ends.

28. Why can you use an alias in ORDER BY but not WHERE? `WHERE` is processed before `SELECT`, so the alias does not exist yet. `ORDER BY` is processed after, so it does.

29. What does COALESCE do? And NULLIF? `COALESCE(a,b,c)` returns the first non-NULL argument — the standard way to supply a default. `NULLIF(a,b)` returns NULL when `a = b`, most often used as `x / NULLIF(y, 0)` to avoid division by zero.

30. Why does 7/2 return 3 in PostgreSQL? Integer divided by integer performs integer division. Cast one side (`7::numeric / 2`) or use a decimal literal (`100.0 * a / b`).

31. DELETE vs TRUNCATE vs DROP? `DELETE` is DML, removes selected rows, supports `WHERE`, is slower and logged. `TRUNCATE` is DDL, removes all rows quickly, no `WHERE`. `DROP` removes the table itself. In PostgreSQL all three can be rolled back inside a transaction, unlike some other engines.

32. What is a view? A materialized view? A view is a stored query that behaves like a table but stores no data. A materialized view stores the result on disk for fast reads and must be refreshed explicitly.

33. What is a stored procedure, and how does it differ from a function? A function returns a value and can be used inside a query. A procedure is invoked with `CALL` and can control transactions internally. PostgreSQL has had procedures since version 11; before that everything was a function.

34. How would you find the second highest salary? `DENSE_RANK` in a CTE filtered to rank 2 — correct with ties. Alternatively `SELECT DISTINCT salary ORDER BY salary DESC LIMIT 1 OFFSET 1`,

which is shorter but counts rows rather than distinct levels. Also decide what should happen when there is no second salary.

35. How do you find duplicate rows? `GROUP BY` the columns that define identity and keep groups with `HAVING COUNT(*) > 1`. To see or delete the extra copies, use `ROW_NUMBER` partitioned by that key and keep only `rn = 1`.

19.3 DBMS questions (36–50)

36. PRIMARY KEY vs UNIQUE? Both enforce uniqueness. A primary key also forbids NULLs, there can be only one per table, and it is the default target of foreign keys. `UNIQUE` permits NULLs and you can have several per table.

37. PRIMARY KEY vs FOREIGN KEY? A primary key identifies a row within its own table. A foreign key points at another table's primary key and enforces that the referenced row exists.

38. What is a candidate key? A composite key? A candidate key is a minimal set of columns that uniquely identifies a row — one of them is chosen as the primary key. A composite key is a key made of more than one column, such as `(order_id, product_id)` in `order_items`.

39. What is referential integrity? The guarantee that every foreign key value refers to an existing row. Enforced by the database, with `ON DELETE RESTRICT`, `CASCADE` or `SET NULL` controlling what happens when the parent is removed.

40. What is normalization and why do it? Organising columns into tables so each fact is stored once, eliminating update, insert and delete anomalies. The practical rule: every non-key attribute must depend on the key, the whole key, and nothing but the key.

41. Explain 1NF, 2NF, 3NF. 1NF: every cell holds a single atomic value, no repeating groups. 2NF: 1NF plus no non-key column depends on only part of a composite key. 3NF: 2NF plus no non-key column depends on another non-key column (no transitive dependency).

42. When would you denormalize? When read performance matters more than write simplicity — reporting systems and data warehouses. You accept duplicated data and the obligation to keep it in sync.

43. What is an index and what does it cost? A separate structure (usually a B-tree) that lets the database find rows by value without scanning the table, turning $O(n)$ lookups into roughly $O(\log n)$. It costs disk space and slows every insert, update and delete, since the index must be maintained too.

44. When will an index not be used? When the column is wrapped in a function, when a `LIKE` pattern starts with a wildcard, when the query returns most of the table anyway, or when the query filters only on a non-leading column of a composite index.

45. Clustered vs non-clustered index? A clustered index determines the physical order of rows, so there can be only one. A non-clustered index is a separate structure pointing at rows, and you can have many. PostgreSQL has no clustered index in the SQL Server sense — its `CLUSTER` command reorders a table once but does not maintain the order.

46. What is a transaction? A group of statements executed as one indivisible unit — either all of them take effect or none do. Delimited by `BEGIN` and `COMMIT` or `ROLLBACK`.

47. Explain ACID. Atomicity: all or nothing. Consistency: constraints hold before and after. Isolation: concurrent transactions do not see each other's uncommitted work. Durability: committed changes survive a crash. The bank-transfer example illustrates all four.

48. What are dirty, non-repeatable and phantom reads? A dirty read sees another transaction's uncommitted change. A non-repeatable read gets different values when re-reading the same row. A phantom read gets a different set of rows when re-running the same `WHERE`.

49. Name the isolation levels and PostgreSQL's default. `READ UNCOMMITTED`, `READ COMMITTED`, `REPEATABLE READ`, `SERIALIZABLE`, in increasing strictness. PostgreSQL defaults to `READ COMMITTED`, never actually permits dirty reads because of MVCC, and its `REPEATABLE READ` also prevents phantoms.

50. What is a deadlock and how do you avoid it? Two transactions each hold a lock the other needs, so neither can proceed. PostgreSQL detects this and aborts one with an error. Avoid it by always acquiring locks in a consistent order and keeping transactions short.

Interview Insight

Five answers to have letter-perfect, because they are asked most often and are easy to fumble: WHERE vs HAVING (2), COUNT(*) vs COUNT(col) (3), INNER vs LEFT JOIN (7), RANK vs DENSE_RANK vs ROW_NUMBER (22), and normalization (40–41). If you can deliver those five crisply, you will sound prepared regardless of what else comes up.

Chapter 20

PostgreSQL Cheat Sheets

20.1 1. SQL syntax cheat sheet

```
-- the full skeleton, in WRITING order
SELECT   [DISTINCT] col, agg(col), window() OVER (...)
FROM     table1
JOIN     table2 ON table1.fk = table2.pk
WHERE    row_condition
GROUP BY col
HAVING   group_condition
ORDER BY col [ASC|DESC] [NULLS FIRST|LAST]
LIMIT    n OFFSET m;

-- logical EXECUTION order
-- FROM -> WHERE -> GROUP BY -> HAVING -> SELECT -> DISTINCT
--      -> window functions -> ORDER BY -> LIMIT
```

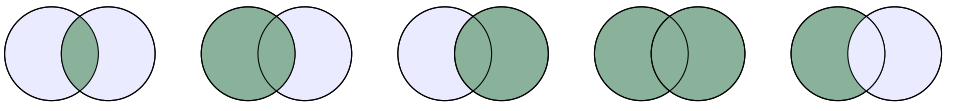
```
-- DDL
CREATE TABLE t (id SERIAL PRIMARY KEY, name TEXT NOT NULL,
                fk INT REFERENCES other(id), amt NUMERIC(10,2) CHECK (amt >= 0));
ALTER TABLE t ADD COLUMN c TEXT;
ALTER TABLE t DROP COLUMN c;
DROP TABLE IF EXISTS t CASCADE;
TRUNCATE t;

-- DML
INSERT INTO t (name, amt) VALUES ('a', 10), ('b', 20);
UPDATE t SET amt = amt * 1.1 WHERE id = 3;
DELETE FROM t WHERE id = 3;
INSERT INTO t (id, name) VALUES (1,'x')
  ON CONFLICT (id) DO UPDATE SET name = EXCLUDED.name;      -- upsert

-- TCL
BEGIN; ... COMMIT;   -- or ROLLBACK;
```

```
-- set operations (same column count and compatible types)
SELECT a FROM t1 UNION     SELECT a FROM t2;   -- deduplicates
SELECT a FROM t1 UNION ALL SELECT a FROM t2;   -- keeps duplicates, faster
SELECT a FROM t1 INTERSECT SELECT a FROM t2;
SELECT a FROM t1 EXCEPT    SELECT a FROM t2;
```

20.2 2. PostgreSQL functions cheat sheet

Aggregates	
`COUNT(*)` / `COUNT(col)`	rows / non-NULL values
`SUM, AVG, MIN, MAX`	NULLs ignored
`COUNT(*) FILTER (WHERE cond)`	conditional count (PostgreSQL)
`STRING_AGG(col, ', ' ORDER BY col)`	concatenate group values
`ARRAY_AGG(col ORDER BY col)`	collect into an array
NULL handling	
`COALESCE(a,b,c)`	first non-NULL
`NULLIF(a,b)`	NULL if equal — guards division by zero
`col IS [NOT] NULL`	the only valid NULL test
Numbers and casting	
`ROUND(x, 2)`	round to 2 decimals (needs numeric)
`CEIL, FLOOR, ABS, MOD(a,b)`	the usual
`x::numeric, CAST(x AS INT)`	casting
`100.0 * a / b`	force real division
Strings	
`LENGTH, UPPER, LOWER, INITCAP`	
`TRIM, LTRIM, RTRIM`	strip whitespace
`SUBSTRING(s FROM 2 FOR 3), LEFT, RIGHT`	slices
`POSITION('a' IN s)`	1-based index, 0 if absent
`s1 || s2`	concat (NULL propagates)
`CONCAT(s1, s2)`	concat (NULL ignored)
`REPLACE(s,'a','b'), SPLIT_PART(s,'@',2)`	
`s LIKE 'a%'` / `s ILIKE 'a%'`	pattern / case-insensitive

20.3 3. JOIN cheat sheet




```sql
FROM a JOIN b ON a.fk = b.pk              -- INNER: matches only
FROM a LEFT JOIN b ON a.fk = b.pk         -- all of a, NULLs where no match
FROM a RIGHT JOIN b ON a.fk = b.pk        -- = b LEFT JOIN a
FROM a FULL OUTER JOIN b ON a.fk = b.pk   -- everything from both
FROM a CROSS JOIN b                       -- every pair, no ON
FROM emp e JOIN emp m ON e.mgr_id = m.id  -- self join

-- anti-join (rows of a with no partner)
FROM a LEFT JOIN b ON a.fk = b.pk WHERE b.pk IS NULL
-- or, preferred:
WHERE NOT EXISTS (SELECT 1 FROM b WHERE b.pk = a.fk)
```

Condition on the *optional* table	put it in `ON`
Condition on the *required* table	put it in `WHERE`
Counts inflated after joining two children	fan-out — use `COUNT(DISTINCT)` or pre-aggregate
Ambiguous column error	qualify with the table alias

20.4 4. GROUP BY / HAVING cheat sheet

```
SELECT g, COUNT(*), AVG(x)
FROM   t
WHERE  row_filter          -- BEFORE grouping; no aggregates allowed here
GROUP  BY g                -- one output row per distinct g
HAVING COUNT(*) > 5        -- AFTER grouping; aggregates allowed
ORDER  BY 2 DESC;
```

Golden rule	every `SELECT` column must be in `GROUP BY` or inside an aggregate
`GROUP BY a, b`	one row per distinct *pair* — finer grained
NULLs	all NULLs form a single group
Conditional count	`SUM(CASE WHEN c THEN 1 ELSE 0 END)` or `COUNT(*) FILTER (WHERE c)`
Never	`COUNT(CASE WHEN c THEN 1 ELSE 0 END)` — counts everything
Percentage	`ROUND(100.0 * COUNT(*) FILTER (WHERE c) / COUNT(*), 1)`

20.5 5. Window function cheat sheet

```
func() OVER (PARTITION BY g ORDER BY x [frame])

-- ranking
ROW_NUMBER()    -- 1,2,3,4  always distinct
RANK()          -- 1,2,2,4  ties share, next rank skips
DENSE_RANK()    -- 1,2,2,3  ties share, no gap
NTILE(4)        -- quartile bucket

-- offset
LAG(x, 1, default)  OVER (ORDER BY d)
LEAD(x, 1, default) OVER (ORDER BY d)
FIRST_VALUE(x)      OVER (PARTITION BY g ORDER BY x)
LAST_VALUE(x)       OVER (PARTITION BY g ORDER BY x
                          ROWS BETWEEN UNBOUNDED PRECEDING
                                   AND UNBOUNDED FOLLOWING)   -- frame REQUIRED

-- aggregates as windows
SUM(x) OVER (PARTITION BY g)              -- group total on every row
SUM(x) OVER (ORDER BY d)                  -- RUNNING total
AVG(x) OVER (ORDER BY d ROWS BETWEEN 2 PRECEDING AND CURRENT ROW)  -- moving avg
```

Filter a window result	impossible in `WHERE` — wrap in a CTE and filter outside
Top N per group	`DENSE_RANK() OVER (PARTITION BY g ORDER BY x DESC)`, then `WHERE rk <= N`
`ROWS` vs `RANGE`	`ROWS` counts physical rows; `RANGE` groups equal `ORDER BY` values
Default frame with `ORDER BY`	`RANGE UNBOUNDED PRECEDING TO CURRENT ROW`

20.6 6. Date/time cheat sheet

```
CURRENT_DATE, CURRENT_TIMESTAMP, NOW()
EXTRACT(YEAR   FROM d)       -- also MONTH, DAY, DOW, QUARTER, HOUR
DATE_TRUNC('month', ts)      -- start of month: the standard grouping tool
AGE(d1, d2)                  -- interval between two dates
d + INTERVAL '7 days'        -- date arithmetic
d2 - d1                      -- integer number of days (for DATE)
TO_CHAR(d, 'YYYY-MM')        -- format as text
TO_DATE('2023-05-01','YYYY-MM-DD')
generate_series('2023-01-01'::date, '2023-12-31'::date, '1 day')
```

Whole year, index-friendly	`d >= '2023-01-01' AND d < '2024-01-01'`
Whole year, index-unfriendly	`EXTRACT(YEAR FROM d) = 2023`
Last 30 days	`d >= CURRENT_DATE - INTERVAL '30 days'`
Group by month	`GROUP BY DATE_TRUNC('month', d)`

20.7 7. Interview patterns cheat sheet

Anti-join (“never”, “no”)	`WHERE NOT EXISTS (SELECT 1 FROM b WHERE b.fk=a.pk)`
Duplicates	`GROUP BY key HAVING COUNT(*) > 1`
Delete duplicates	keep `rn = 1` from `ROW_NUMBER() OVER (PARTITION BY key ORDER BY id)`
Nth highest	`DENSE_RANK() OVER (ORDER BY x DESC)` then `WHERE rk = N`
Top N per group	rank with `PARTITION BY` in a CTE, filter outside
Max per group with details	window `RANK`, correlated subquery, or `DISTINCT ON (g)`
Above own group’s average	correlated subquery, or `AVG(x) OVER (PARTITION BY g)`
More than their manager	self join `e.mgr_id = m.id`, `WHERE e.sal > m.sal`
Running total	`SUM(x) OVER (ORDER BY d)`
Previous row	`LAG(x) OVER (ORDER BY d)`
Consecutive N	`LAG(x,1)` and `LAG(x,2)` compared with `x`
Streaks (any length)	group by `id - ROW_NUMBER() OVER (ORDER BY id)`
Missing periods	`generate_series` + `LEFT JOIN`
All combinations	`CROSS JOIN` + `LEFT JOIN` facts
“bought all products”	`HAVING COUNT(DISTINCT p) = (SELECT COUNT(*) FROM products)`
Pairs without duplicates	self join with `a.id < b.id`

20.8 8. DBMS cheat sheet

Keys	super ⊃ candidate ⊃ primary; alternate = unchosen candidates; composite = multi-column; foreign = points at another PK
PK vs UNIQUE	PK: one per table, no NULLs. UNIQUE: many, NULLs allowed
Constraints	`NOT NULL`, `UNIQUE`, `PRIMARY KEY`, `FOREIGN KEY`, `CHECK`, `DEFAULT`
ON DELETE	`RESTRICT` (default), `CASCADE`, `SET NULL`
Normalization	"the key, the whole key, and nothing but the key" = 1NF, 2NF, 3NF
1NF	atomic values, no repeating groups
2NF	no partial dependency on part of a composite key
3NF	no transitive dependency (non-key → non-key)
Denormalize	for read speed in reporting/OLAP; costs consistency
Index	B-tree; $O(n) \to O(\log n)$ lookups; costs space and write speed
Index not used	function on the column, leading wildcard, low selectivity, non-leading column of a composite
Clustered	physical row order; one per table; PostgreSQL has no maintained clustered index
Transaction	`BEGIN` ... `COMMIT` / `ROLLBACK`
ACID	Atomicity, Consistency, Isolation, Durability
Anomalies	dirty read, non-repeatable read, phantom read
Isolation levels	`READ UNCOMMITTED` < `READ COMMITTED` (PG default) < `REPEATABLE READ` < `SERIALIZABLE`
MVCC	PostgreSQL: readers never block writers, writers never block readers
Deadlock	mutual lock wait; PG detects and aborts one; avoid by consistent lock ordering
DDL/DML/DCL/TCL	structure / data / permissions / transactions
DELETE/TRUNCATE/DROP	rows with `WHERE` / all rows fast / the table itself
View	stored query, no data. Materialized view: stored result, needs refresh

20.9 The night-before page

IF YOU READ ONE PAGE BEFORE THE INTERVIEW

Processing order FROM → WHERE → GROUP BY → HAVING → SELECT → window → ORDER BY → LIMIT

The thirty-second routine before writing any query
1. What is *one row* of the answer? 2. Which tables, what connects them? 3. Which pattern? 4. Edge cases: NULLs, ties, empty groups, integer division.

The five most-asked concepts
WHERE vs HAVING · COUNT(*) vs COUNT(col) · INNER vs LEFT JOIN · RANK vs DENSE_RANK vs ROW_NUMBER · Normalization (key, whole key, nothing but the key)

The four traps that cost most marks
`= NULL` instead of `IS NULL` · `NOT IN` with NULLs · `WHERE` on the outer table of a `LEFT JOIN` · integer division in percentages

The one skeleton to have memorised

```
WITH ranked AS (
  SELECT t.*, DENSE_RANK() OVER (PARTITION BY g ORDER BY x DESC) AS rk
  FROM t
)
SELECT * FROM ranked WHERE rk <= N;
```

It answers top-N-per-group, Nth highest, highest per department, and about a third of all medium interview questions.

Chapter 21

Complete Answer Key

Interview Insight

Before you read any of this: have you written and *run* your own attempt? If not, go back. Reading a correct query teaches you almost nothing; comparing it against your own wrong one teaches you a great deal.
Where your query differs from mine but produces the same result, yours is probably fine — SQL has many correct forms. Check the *Insight* note to see whether you got the concept the problem was testing.

21.1 Day 1 — SQL Fundamentals

D1-P1

```
SELECT * FROM departments;
```

Insight: `SELECT *` is fine for exploring, but name your columns in real code so the result does not change when someone adds a column.

D1-P2

```
SELECT name, job_title FROM employees;
```

D1-P3

```
SELECT name, hire_date FROM employees
WHERE hire_date < '2018-01-01';
```

Insight: dates compare with the ordinary operators. Always write them as `'YYYY-MM-DD'` — unambiguous in every locale.

D1-P4

```
SELECT name, salary FROM employees
WHERE salary >= 60000 AND salary <= 90000;
```

Alternative: `WHERE salary BETWEEN 60000 AND 90000` — identical, since `BETWEEN` is inclusive.

D1-P5

```
SELECT DISTINCT location FROM departments;
```

Returns Bangalore, Mumbai, Delhi, Hyderabad — four rows from five, because Engineering and HR share a location.

D1-P6

```
SELECT name, salary FROM employees
WHERE dept_id = 2 AND salary > 60000;
```

D1-P7

```
SELECT name, dept_id FROM employees
WHERE dept_id <> 1 OR dept_id IS NULL;
```

Explanation: `dept_id <> 1` alone drops Omar Ali, because his `dept_id` is NULL and `NULL <> 1` evaluates to UNKNOWN, not TRUE. You must add the NULL branch explicitly.
Insight: this is LeetCode 584 in miniature, and it is the most commonly failed beginner concept.

D1-P8

```sql
SELECT name FROM employees WHERE manager_id IS NULL;
```

Asha Nair, Vikram Singh, Omar Ali.

D1-P9

```sql
SELECT DISTINCT category FROM products WHERE unit_price > 20000;
```

D1-P10

```sql
SELECT name, salary, hire_date, dept_id
FROM   employees
WHERE  (salary > 90000 OR hire_date < '2016-01-01')
  AND  (dept_id <> 4 OR dept_id IS NULL);
```

Explanation: the parentheses around the `OR` are essential — without them, `AND` binds tighter and the meaning changes completely. The second pair handles the NULL department, as in D1-P7.

21.2 Day 2 — Filtering and Sorting

D2-P1

```sql
SELECT name, salary FROM employees ORDER BY salary DESC;
```

D2-P2

```sql
SELECT product_name, unit_price FROM products
ORDER BY unit_price ASC LIMIT 5;
```

D2-P3

```sql
SELECT name, dept_id FROM employees WHERE dept_id IN (1, 3, 5);
```

D2-P4

```sql
SELECT product_name, unit_price FROM products
WHERE unit_price BETWEEN 2000 AND 30000
ORDER BY unit_price;
```

D2-P5

```sql
SELECT name FROM employees WHERE name LIKE 'A%';
```

D2-P6

```sql
SELECT name, dept_id, salary FROM employees
ORDER BY dept_id ASC, salary DESC;
```

Insight: the second sort key breaks ties in the first. Note that Omar Ali (NULL department) sorts last by default in `ASC`.

D2-P7

```sql
SELECT order_id, order_date FROM orders
ORDER BY order_date ASC
LIMIT 5 OFFSET 5;
```

Insight: for page p with size s, `OFFSET` is $(p-1) \times s$.

D2-P8

```sql
SELECT customer_name, city FROM customers WHERE city ILIKE '%ba%';
```

Alternative (portable): `WHERE LOWER(city) LIKE '%ba%'`. Mention that wrapping the column in `LOWER` prevents index use.

D2-P9

```
SELECT product_name, unit_price,
       CASE WHEN unit_price > 50000 THEN 'Premium'
            WHEN unit_price >= 5000 THEN 'Standard'
            ELSE 'Budget'
       END AS tier
FROM products
ORDER BY unit_price DESC;
```

Insight: the branches must be ordered from most to least restrictive, because `CASE` stops at the first match. Reversing them would label everything 'Budget'.

D2-P10

```
SELECT name, salary,
       CASE WHEN manager_id IS NULL THEN 'No' ELSE 'Yes' END AS has_manager
FROM employees
ORDER BY (manager_id IS NOT NULL), salary DESC;
```

Explanation: `manager_id IS NOT NULL` is a boolean; FALSE sorts before TRUE in PostgreSQL, so the managerless employees come first. `ORDER BY has_manager` also works, since `ORDER BY` can see `SELECT` aliases.

21.3 Day 3 — Aggregates and GROUP BY

D3-P1

```
SELECT COUNT(*) AS total_employees FROM employees;     -- 20
```

D3-P2

```
SELECT MAX(salary) AS highest, MIN(salary) AS lowest FROM employees;
```

D3-P3

```
SELECT COUNT(email) AS with_email, COUNT(*) AS total FROM employees;    -- 19, 20
```

Insight: the whole point of the problem. `COUNT(email)` skips Omar Ali's NULL.

D3-P4

```
SELECT category, ROUND(AVG(unit_price), 2) AS avg_price
FROM products GROUP BY category ORDER BY avg_price DESC;
```

D3-P5

```
SELECT dept_id, COUNT(*) AS headcount
FROM employees GROUP BY dept_id ORDER BY dept_id;
```

Insight: six rows, not five — the NULL department forms its own group.

D3-P6

```
SELECT dept_id, SUM(salary) AS total_salary
FROM employees WHERE dept_id IS NOT NULL
GROUP BY dept_id ORDER BY total_salary DESC;
```

D3-P7

```
SELECT COUNT(DISTINCT job_title) AS distinct_titles FROM employees;
```

D3-P8

```sql
SELECT dept_id, ROUND(AVG(salary), 2) AS avg_salary
FROM   employees
WHERE  dept_id IS NOT NULL
GROUP  BY dept_id
HAVING AVG(salary) > 80000;
```

Insight: the filter is on an aggregate, so it must be `HAVING`. You may repeat the aggregate expression there; you cannot use the `SELECT` alias in standard SQL (PostgreSQL does allow it in `GROUP BY` but not `HAVING`).

D3-P9

```sql
SELECT status, COUNT(*) AS n, MIN(order_date) AS earliest
FROM orders GROUP BY status ORDER BY n DESC;
```

D3-P10

```sql
SELECT category, COUNT(*) AS n, ROUND(AVG(unit_price), 2) AS avg_price
FROM   products
GROUP  BY category
HAVING COUNT(*) >= 2 AND AVG(unit_price) > 10000;
```

21.4 Day 4 — GROUP BY Mastery

D4-P1

```sql
SELECT dept_id, job_title, COUNT(*) AS n
FROM employees WHERE dept_id IS NOT NULL
GROUP BY dept_id, job_title ORDER BY dept_id, n DESC;
```

D4-P2

```sql
SELECT customer_id, COUNT(*) AS orders
FROM orders GROUP BY customer_id ORDER BY orders DESC;
```

Note: this misses customers with zero orders — they have no rows in `orders` at all. Fixing that needs a `LEFT JOIN` (Day 5).

D4-P3

```sql
SELECT EXTRACT(YEAR FROM hire_date) AS yr, COUNT(*) AS hires
FROM employees GROUP BY EXTRACT(YEAR FROM hire_date) ORDER BY yr;
```

D4-P4

```sql
SELECT category, COUNT(*) AS n, MIN(unit_price) AS cheapest, MAX(unit_price) AS
    priciest
FROM products GROUP BY category;
```

D4-P5

```sql
SELECT customer_id,
       COUNT(*) AS total_orders,
       COUNT(*) FILTER (WHERE status = 'DELIVERED') AS delivered
FROM   orders
GROUP  BY customer_id
ORDER  BY customer_id;
```

Alternative (portable): `SUM(CASE WHEN status = 'DELIVERED' THEN 1 ELSE 0 END)`.
Insight: you cannot use `WHERE status = 'DELIVERED'` here — it would filter the total count as well. That is precisely why conditional aggregation exists.

D4-P6

```sql
SELECT dept_id, COUNT(*) AS headcount, SUM(salary) AS total_salary
FROM   employees WHERE dept_id IS NOT NULL
GROUP  BY dept_id
HAVING SUM(salary) > 300000
ORDER  BY total_salary DESC;
```

D4-P7

```sql
SELECT order_id, SUM(quantity * unit_price) AS order_total
FROM order_items GROUP BY order_id ORDER BY order_total DESC;
```

D4-P8

```sql
SELECT dept_id,
       COUNT(*) FILTER (WHERE hire_date <  '2020-01-01') AS before_2020,
       COUNT(*) FILTER (WHERE hire_date >= '2020-01-01') AS from_2020,
       COUNT(*) AS total
FROM   employees WHERE dept_id IS NOT NULL
GROUP  BY dept_id ORDER BY dept_id;
```

D4-P9

```sql
SELECT city, COUNT(*) AS customers
FROM customers GROUP BY city HAVING COUNT(*) > 1;
```

D4-P10

```sql
SELECT dept_id,
       COUNT(*) AS headcount,
       COUNT(*) FILTER (WHERE salary > 90000) AS high_earners,
       ROUND(100.0 * COUNT(*) FILTER (WHERE salary > 90000) / COUNT(*), 1) AS pct
FROM   employees WHERE dept_id IS NOT NULL
GROUP  BY dept_id ORDER BY pct DESC;
```

Insight: `100.0` rather than `100` — with integer division every percentage would come out as 0.

D4-P11

```sql
SELECT project_id, SUM(hours_worked) AS total_hours,
       COUNT(DISTINCT emp_id) AS team_size
FROM employee_projects GROUP BY project_id ORDER BY total_hours DESC;
```

D4-P12

```sql
SELECT EXTRACT(YEAR FROM order_date) AS yr,
       COUNT(*) AS orders,
       COUNT(*) FILTER (WHERE status = 'CANCELLED') AS cancelled
FROM orders GROUP BY EXTRACT(YEAR FROM order_date) ORDER BY yr;
```

D4-P13

```sql
SELECT job_title, COUNT(*) AS n,
       MAX(salary) - MIN(salary) AS salary_range
FROM employees GROUP BY job_title HAVING COUNT(*) > 1;
```

21.5 Day 5 — JOINs

D5-P1

```sql
SELECT e.name, d.dept_name
FROM employees e JOIN departments d ON e.dept_id = d.dept_id;
```

19 rows — Omar Ali is excluded by the inner join.

D5-P2

```
SELECT e.name, d.dept_name
FROM employees e LEFT JOIN departments d ON e.dept_id = d.dept_id;
```

20 rows; Omar Ali appears with a NULL `dept_name`.
Insight: the table you must preserve goes on the left.

D5-P3

```
SELECT o.order_id, o.order_date, c.customer_name
FROM orders o JOIN customers c ON o.customer_id = c.customer_id
ORDER BY o.order_date;
```

D5-P4

```
SELECT d.dept_name, e.name
FROM departments d LEFT JOIN employees e ON d.dept_id = e.dept_id
ORDER BY d.dept_name;
```

D5-P5

```
SELECT e.name AS employee, m.name AS manager
FROM employees e LEFT JOIN employees m ON e.manager_id = m.emp_id
ORDER BY m.name NULLS FIRST, e.name;
```

Insight: `LEFT` keeps the three employees without a manager. With `INNER JOIN` you would silently return 17 rows.

D5-P6

```
SELECT e.name, e.salary, d.dept_name
FROM employees e JOIN departments d ON e.dept_id = d.dept_id
WHERE d.location = 'Bangalore' AND e.salary > 80000
ORDER BY e.salary DESC;
```

D5-P7

```
SELECT o.order_id, o.order_date, c.customer_name, c.country
FROM orders o JOIN customers c ON o.customer_id = c.customer_id
WHERE o.status = 'DELIVERED'
ORDER BY o.order_date;
```

Insight: here `WHERE` is correct — `orders` is the required table, not the optional one.

D5-P8

```
SELECT p.product_name, COALESCE(SUM(oi.quantity), 0) AS total_qty
FROM   products p
LEFT   JOIN order_items oi ON p.product_id = oi.product_id
GROUP  BY p.product_id, p.product_name
ORDER  BY total_qty DESC;
```

Insight: `COALESCE` turns the NULL from the unmatched `LEFT JOIN` into 0 — "never ordered" should read as zero, not blank.

D5-P9

```
SELECT c.customer_name
FROM   customers c
LEFT   JOIN orders o ON c.customer_id = o.customer_id
WHERE  o.order_id IS NULL;
```

Lyra Interiors and Cobalt Motors.
Alternative: `WHERE NOT EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id)` — preferred, because it is NULL-safe.

D5-P10

```
SELECT e.name AS employee, m.name AS manager, d.dept_name
FROM   employees e
LEFT   JOIN employees   m ON e.manager_id = m.emp_id
LEFT   JOIN departments d ON e.dept_id    = d.dept_id
ORDER  BY d.dept_name NULLS LAST, e.name;
```

21.6 Day 6 — Advanced JOINs

D6-P1

```
SELECT e.name, p.project_name, ep.hours_worked, ep.role
FROM   employee_projects ep
JOIN   employees e ON e.emp_id     = ep.emp_id
JOIN   projects  p ON p.project_id = ep.project_id
ORDER  BY p.project_name, ep.hours_worked DESC;
```

D6-P2

```
SELECT o.order_id, c.customer_name, pr.product_name, oi.quantity
FROM   orders o
JOIN   customers   c  ON c.customer_id = o.customer_id
JOIN   order_items oi ON oi.order_id   = o.order_id
JOIN   products    pr ON pr.product_id = oi.product_id
ORDER  BY o.order_id;
```

D6-P3

```
SELECT d.dept_name, COUNT(p.project_id) AS projects
FROM departments d LEFT JOIN projects p ON p.dept_id = d.dept_id
GROUP BY d.dept_id, d.dept_name ORDER BY projects DESC;
```

D6-P4

```
SELECT p.project_name, COUNT(ep.emp_id) AS team_size
FROM projects p LEFT JOIN employee_projects ep ON ep.project_id = p.project_id
GROUP BY p.project_id, p.project_name ORDER BY team_size DESC;
```

D6-P5

```
SELECT e.name FROM employees e
WHERE NOT EXISTS (SELECT 1 FROM employee_projects ep WHERE ep.emp_id = e.emp_id);
```

Asha Nair, Vikram Singh, Sameer Khan, Omar Ali.

D6-P6

```
SELECT c.customer_name,
       COUNT(DISTINCT o.order_id) AS orders,
       COALESCE(SUM(oi.quantity * oi.unit_price), 0) AS revenue
FROM   customers c
LEFT   JOIN orders      o  ON o.customer_id = c.customer_id
LEFT   JOIN order_items oi ON oi.order_id   = o.order_id
GROUP  BY c.customer_id, c.customer_name
ORDER  BY revenue DESC;
```

Insight: `COUNT(DISTINCT o.order_id)` is essential — the item join duplicates each order row once per line item. `SUM` is safe because every duplicated row carries a genuine line amount.

D6-P7

```
SELECT p.product_name FROM products p
WHERE NOT EXISTS (SELECT 1 FROM order_items oi WHERE oi.product_id = p.product_id);
```

D6-P8

```sql
SELECT d.dept_name,
       COUNT(DISTINCT e.emp_id)     AS employees,
       COUNT(DISTINCT p.project_id) AS projects
FROM   departments d
LEFT   JOIN employees e ON e.dept_id = d.dept_id
LEFT   JOIN projects  p ON p.dept_id = d.dept_id
GROUP  BY d.dept_id, d.dept_name;
```

Alternative (cleaner, and what you would write in production): aggregate each child in its own CTE and join the two summaries.
Insight: this is the fan-out problem. Without `DISTINCT`, Engineering reports 14 and 14.

D6-P9

```sql
SELECT a.name AS emp1, b.name AS emp2, a.dept_id, a.job_title
FROM   employees a
JOIN   employees b ON a.dept_id = b.dept_id
                  AND a.job_title = b.job_title
                  AND a.emp_id < b.emp_id;
```

Insight: `a.emp_id < b.emp_id` does two jobs — it prevents a row pairing with itself and prevents each pair appearing twice in mirrored order.

D6-P10

```sql
SELECT p.project_name,
       MAX(e.name) FILTER (WHERE ep.role = 'Lead') AS lead_name,
       SUM(ep.hours_worked) AS total_hours
FROM   projects p
JOIN   employee_projects ep ON ep.project_id = p.project_id
JOIN   employees e ON e.emp_id = ep.emp_id
GROUP  BY p.project_id, p.project_name
ORDER  BY total_hours DESC;
```

Insight: `MAX(...) FILTER (...)` is a neat way to pull one specific row's value into a grouped result when you know there is exactly one match.

D6-P11

```sql
SELECT c.customer_name, c.country,
       SUM(oi.quantity * oi.unit_price) AS revenue
FROM   customers c
JOIN   orders o       ON o.customer_id = c.customer_id
JOIN   order_items oi ON oi.order_id   = o.order_id
GROUP  BY c.customer_id, c.customer_name, c.country
ORDER  BY revenue DESC
LIMIT  3;
```

D6-P12

```sql
SELECT e.name AS employee, e.salary, m.name AS manager, m.salary AS manager_salary
FROM   employees e JOIN employees m ON e.manager_id = m.emp_id
WHERE  e.salary > m.salary;
```

Empty on our data — and saying "the correct answer here is zero rows, which I verified" is a good habit.

D6-P13

```sql
SELECT d.dept_name,
       COUNT(DISTINCT e.emp_id) AS headcount,
       COUNT(DISTINCT e.emp_id) FILTER (
           WHERE EXISTS (SELECT 1 FROM employees r WHERE r.manager_id = e.emp_id)
       ) AS managers
FROM   departments d
LEFT   JOIN employees e ON e.dept_id = d.dept_id
GROUP  BY d.dept_id, d.dept_name;
```

21.7 Day 7 — Subqueries

D7-P1

```
SELECT name, salary FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees)
ORDER BY salary DESC;
```

D7-P2

```
SELECT name FROM employees
WHERE dept_id IN (SELECT dept_id FROM departments WHERE location = 'Bangalore');
```

D7-P3

```
SELECT product_name, unit_price FROM products
WHERE unit_price > (SELECT AVG(unit_price) FROM products);
```

D7-P4

```
SELECT c.customer_name FROM customers c
WHERE EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id);
```

D7-P5

```
SELECT name, salary,
       ROUND((SELECT AVG(salary) FROM employees), 2) AS company_avg,
       ROUND(salary - (SELECT AVG(salary) FROM employees), 2) AS diff
FROM employees ORDER BY diff DESC;
```

D7-P6

```
SELECT e.name, e.dept_id, e.salary
FROM   employees e
WHERE  e.salary > (SELECT AVG(e2.salary) FROM employees e2
                   WHERE e2.dept_id = e.dept_id)
ORDER  BY e.dept_id, e.salary DESC;
```

Alternative (Day 9, better):

```
WITH t AS (SELECT e.*, AVG(salary) OVER (PARTITION BY dept_id) AS da FROM employees e)
SELECT name, dept_id, salary FROM t WHERE salary > da;
```

Insight: the correlated version re-computes the average per row; the window version computes it once per partition. Both are correct; know which you would prefer and why.

D7-P7

```
SELECT c.customer_name FROM customers c
WHERE NOT EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id);
```

D7-P8

```
SELECT dept_id, total FROM (
    SELECT dept_id, SUM(salary) AS total
    FROM employees WHERE dept_id IS NOT NULL
    GROUP BY dept_id
) s
ORDER BY total DESC LIMIT 1;
```

D7-P9

```
SELECT MAX(salary) AS second_highest FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);
```

Insight: "the largest value smaller than the largest value". Elegant, no window functions, and it returns NULL rather than an empty set when there is no second salary — exactly what LeetCode 176 requires.

D7-P10

```
SELECT DISTINCT e.name FROM employees e
WHERE EXISTS (SELECT 1 FROM employee_projects ep
              WHERE ep.emp_id = e.emp_id AND ep.hours_worked > 300);
```

D7-P11

```
SELECT ROUND(AVG(dept_avg), 2) AS avg_of_dept_avgs
FROM (
    SELECT dept_id, AVG(salary) AS dept_avg
    FROM employees WHERE dept_id IS NOT NULL
    GROUP BY dept_id
) s;
```

Insight: `AVG(AVG(x))` is illegal; you must aggregate in a derived table and aggregate again outside. Note this is *not* the same as the overall average, because each department gets equal weight regardless of size.

D7-P12

```
SELECT name FROM employees e
WHERE NOT EXISTS (SELECT 1 FROM employees r WHERE r.manager_id = e.emp_id);
```

Insight: the `NOT IN` version returns zero rows, because `manager_id` contains NULLs. If you must use `NOT IN`, add `WHERE manager_id IS NOT NULL` inside the subquery.

21.8 Day 8 — CTEs

D8-P1

```
WITH avg_sal AS (SELECT AVG(salary) AS a FROM employees)
SELECT e.name, e.salary FROM employees e, avg_sal
WHERE e.salary > avg_sal.a ORDER BY e.salary DESC;
```

D8-P2

```
WITH counts AS (
  SELECT dept_id, COUNT(*) AS n FROM employees
  WHERE dept_id IS NOT NULL GROUP BY dept_id
)
SELECT d.dept_name, c.n FROM counts c
JOIN departments d ON d.dept_id = c.dept_id
WHERE c.n > 2 ORDER BY c.n DESC;
```

D8-P3

```
WITH order_totals AS (
  SELECT order_id, SUM(quantity * unit_price) AS total
  FROM order_items GROUP BY order_id
)
SELECT c.customer_name, o.order_id, o.order_date, t.total
FROM   order_totals t
JOIN   orders o    ON o.order_id    = t.order_id
JOIN   customers c ON c.customer_id = o.customer_id
ORDER  BY t.total DESC LIMIT 5;
```

D8-P4

```
WITH dept_total AS (
  SELECT dept_id, SUM(salary) AS total FROM employees
  WHERE dept_id IS NOT NULL GROUP BY dept_id
```

```
), top_dept AS (
  SELECT dept_id, total FROM dept_total ORDER BY total DESC LIMIT 1
)
SELECT d.dept_name, t.total FROM top_dept t
JOIN departments d ON d.dept_id = t.dept_id;
```

D8-P5

```
WITH qty AS (
  SELECT product_id, SUM(quantity) AS total_qty
  FROM order_items GROUP BY product_id
)
SELECT p.product_name, q.total_qty FROM qty q
JOIN products p ON p.product_id = q.product_id
WHERE q.total_qty > 10 ORDER BY q.total_qty DESC;
```

D8-P6

```
WITH rev AS (
  SELECT o.customer_id, SUM(oi.quantity * oi.unit_price) AS revenue
  FROM orders o JOIN order_items oi ON oi.order_id = o.order_id
  GROUP BY o.customer_id
), total AS (SELECT SUM(revenue) AS grand FROM rev)
SELECT c.customer_name, r.revenue,
       ROUND(100.0 * r.revenue / t.grand, 2) AS pct_of_total
FROM   rev r CROSS JOIN total t
JOIN   customers c ON c.customer_id = r.customer_id
ORDER  BY r.revenue DESC;
```

Alternative: `SUM(revenue) OVER ()` as the denominator — one CTE fewer.

D8-P7

```
WITH dept_avg AS (
  SELECT dept_id, AVG(salary) AS a FROM employees
  WHERE dept_id IS NOT NULL GROUP BY dept_id
), company AS (SELECT AVG(salary) AS c FROM employees)
SELECT d.dept_name, ROUND(da.a, 2) AS dept_avg, ROUND(co.c, 2) AS company_avg
FROM   dept_avg da CROSS JOIN company co
JOIN   departments d ON d.dept_id = da.dept_id
WHERE  da.a > co.c;
```

D8-P8

```
WITH ph AS (
  SELECT project_id, SUM(hours_worked) AS hrs FROM employee_projects GROUP BY
    project_id
), avg_h AS (SELECT AVG(hrs) AS a FROM ph)
SELECT p.project_name, ph.hrs FROM ph CROSS JOIN avg_h
JOIN projects p ON p.project_id = ph.project_id
WHERE ph.hrs > avg_h.a ORDER BY ph.hrs DESC;
```

D8-P9

```
WITH monthly AS (
  SELECT DATE_TRUNC('month', o.order_date)::date AS mth,
         COUNT(DISTINCT o.order_id) AS orders,
         SUM(oi.quantity * oi.unit_price) AS revenue
  FROM orders o JOIN order_items oi ON oi.order_id = o.order_id
  WHERE o.order_date >= '2023-01-01' AND o.order_date < '2024-01-01'
  GROUP BY 1
)
SELECT TO_CHAR(mth, 'Mon YYYY') AS month, orders, revenue
FROM monthly ORDER BY mth;
```

D8-P10

```
WITH RECURSIVE tree AS (
    SELECT emp_id, name, manager_id, 1 AS depth
    FROM   employees WHERE emp_id = 1
    UNION ALL
    SELECT e.emp_id, e.name, e.manager_id, t.depth + 1
    FROM   employees e JOIN tree t ON e.manager_id = t.emp_id
)
SELECT depth, name FROM tree ORDER BY depth, name;
```

21.9 Day 9 — Window Functions I

D9-P1

```
SELECT name, salary, ROUND(AVG(salary) OVER (), 2) AS company_avg FROM employees;
```

D9-P2

```
SELECT name, dept_id, salary,
       ROUND(AVG(salary) OVER (PARTITION BY dept_id), 2) AS dept_avg
FROM employees;
```

D9-P3

```
SELECT name, salary,
       ROW_NUMBER() OVER (ORDER BY salary DESC) AS rn,
       RANK()       OVER (ORDER BY salary DESC) AS rnk,
       DENSE_RANK() OVER (ORDER BY salary DESC) AS drnk
FROM employees ORDER BY salary DESC;
```

Look at the two employees on 67000: `rn` gives them 12 and 13, `rnk` gives both 12 and then skips to 14, `drnk` gives both 12 and then 13.

D9-P4

```
SELECT name, dept_id, salary,
       RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS dept_rank
FROM employees WHERE dept_id IS NOT NULL;
```

D9-P5

```
SELECT name, dept_id, salary,
       ROUND(salary - AVG(salary) OVER (PARTITION BY dept_id), 2) AS
   diff_from_dept_avg
FROM employees WHERE dept_id IS NOT NULL ORDER BY dept_id, diff_from_dept_avg DESC;
```

D9-P6

```
WITH r AS (
  SELECT e.name, e.salary, d.dept_name,
         ROW_NUMBER() OVER (PARTITION BY e.dept_id ORDER BY e.salary DESC, e.emp_id)
    AS rn
  FROM employees e JOIN departments d ON d.dept_id = e.dept_id
)
SELECT dept_name, name, salary FROM r WHERE rn = 1 ORDER BY dept_name;
```

Insight: `ROW_NUMBER` guarantees exactly one row per department; the `e.emp_id` tie-breaker makes the choice deterministic.

D9-P7

```
WITH r AS (
  SELECT e.name, e.salary, d.dept_name,
         DENSE_RANK() OVER (PARTITION BY e.dept_id ORDER BY e.salary DESC) AS rk
  FROM employees e JOIN departments d ON d.dept_id = e.dept_id
)
SELECT dept_name, name, salary, rk FROM r WHERE rk <= 2 ORDER BY dept_name, rk;
```

Insight: `DENSE_RANK` because the question says "including ties".

D9-P8

```
WITH r AS (
  SELECT DISTINCT salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS rk FROM
    employees
)
SELECT salary FROM r WHERE rk = 3;
```

D9-P9

```
WITH ot AS (
  SELECT o.order_id, o.customer_id, SUM(oi.quantity * oi.unit_price) AS total
  FROM orders o JOIN order_items oi ON oi.order_id = o.order_id
  GROUP BY o.order_id, o.customer_id
), r AS (
  SELECT ot.*, ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY total DESC) AS rn
  FROM ot
)
SELECT c.customer_name, r.order_id, r.total
FROM r JOIN customers c ON c.customer_id = r.customer_id
WHERE r.rn = 1 ORDER BY r.total DESC;
```

D9-P10

```
WITH r AS (
  SELECT p.project_name, e.name, ep.role, ep.hours_worked,
         RANK() OVER (PARTITION BY ep.project_id ORDER BY ep.hours_worked DESC) AS rk
  FROM employee_projects ep
  JOIN employees e ON e.emp_id = ep.emp_id
  JOIN projects  p ON p.project_id = ep.project_id
)
SELECT project_name, name, role, hours_worked FROM r WHERE rk = 1 ORDER BY
    project_name;
```

D9-P11

```
SELECT name, dept_id, salary,
       ROUND(AVG(salary) OVER (PARTITION BY dept_id), 2) AS dept_avg,
       ROUND(100.0 * RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC)
             / COUNT(*) OVER (PARTITION BY dept_id), 1) AS pct_position
FROM employees WHERE dept_id IS NOT NULL ORDER BY dept_id, salary DESC;
```

Insight: three window functions over the same partition in one `SELECT` — each is computed independently, and none of them collapses a row.

21.10 Day 10 — Window Functions II

D10-P1

```
SELECT name, hire_date, salary,
       LAG(salary) OVER (ORDER BY hire_date) AS prev_hire_salary
FROM employees ORDER BY hire_date;
```

D10-P2

```
SELECT customer_id, order_date,
       LEAD(order_date) OVER (PARTITION BY customer_id ORDER BY order_date) AS
    next_order
FROM orders ORDER BY customer_id, order_date;
```

D10-P3

```
WITH daily AS (
  SELECT o.order_date, SUM(oi.quantity * oi.unit_price) AS value
  FROM orders o JOIN order_items oi ON oi.order_id = o.order_id
  GROUP BY o.order_date
)
SELECT order_date, value, SUM(value) OVER (ORDER BY order_date) AS running_total
FROM daily ORDER BY order_date;
```

D10-P4

```
SELECT name, hire_date, salary,
       salary - LAG(salary) OVER (ORDER BY hire_date) AS diff_from_prev
FROM employees ORDER BY hire_date;
```

D10-P5

```
SELECT name, dept_id, salary,
       FIRST_VALUE(name) OVER (PARTITION BY dept_id ORDER BY salary DESC) AS
    top_earner
FROM employees WHERE dept_id IS NOT NULL;
```

D10-P6

```
SELECT name, dept_id, hire_date, salary,
       SUM(salary) OVER (PARTITION BY dept_id ORDER BY hire_date) AS
    running_dept_salary
FROM employees WHERE dept_id IS NOT NULL ORDER BY dept_id, hire_date;
```

D10-P7

```
WITH ot AS (
  SELECT o.order_id, o.order_date, SUM(oi.quantity * oi.unit_price) AS total
  FROM orders o JOIN order_items oi ON oi.order_id = o.order_id
  GROUP BY o.order_id, o.order_date
)
SELECT order_date, total,
       ROUND(AVG(total) OVER (ORDER BY order_date
                              ROWS BETWEEN 2 PRECEDING AND CURRENT ROW), 2) AS
    moving_avg_3
FROM ot ORDER BY order_date;
```

Insight: `ROWS`, not `RANGE` — with `RANGE`, orders sharing a date would all be lumped into the current row's frame.

D10-P8

```
SELECT customer_id, order_date,
       order_date - LAG(order_date) OVER (PARTITION BY customer_id ORDER BY order_date
    )
         AS days_since_prev
FROM orders ORDER BY customer_id, order_date;
```

D10-P9

```
WITH ot AS (
  SELECT o.customer_id, o.order_date, SUM(oi.quantity * oi.unit_price) AS total
  FROM orders o JOIN order_items oi ON oi.order_id = o.order_id
```

```
  GROUP BY o.order_id, o.customer_id, o.order_date
), cmp AS (
  SELECT customer_id, order_date, total,
         LAG(total)      OVER (PARTITION BY customer_id ORDER BY order_date) AS
    prev_total,
         ROW_NUMBER()    OVER (PARTITION BY customer_id ORDER BY order_date DESC) AS
    rn
  FROM ot
)
SELECT c.customer_name, cmp.total AS latest, cmp.prev_total
FROM cmp JOIN customers c ON c.customer_id = cmp.customer_id
WHERE cmp.rn = 1 AND cmp.prev_total IS NOT NULL AND cmp.total < cmp.prev_total;
```

D10-P10

```
SELECT name, dept_id, salary,
       FIRST_VALUE(name) OVER (PARTITION BY dept_id ORDER BY salary DESC) AS
    top_earner,
       LAST_VALUE(name)  OVER (PARTITION BY dept_id ORDER BY salary DESC
                               ROWS BETWEEN UNBOUNDED PRECEDING
                                        AND UNBOUNDED FOLLOWING) AS lowest_earner
FROM employees WHERE dept_id IS NOT NULL;
```

Insight: the explicit frame on `LAST_VALUE` is mandatory. Without it you get the current row's name.

D10-P11

```
WITH monthly AS (
  SELECT DATE_TRUNC('month', o.order_date)::date AS mth,
         SUM(oi.quantity * oi.unit_price) AS revenue
  FROM orders o JOIN order_items oi ON oi.order_id = o.order_id
  WHERE o.order_date >= '2023-01-01' AND o.order_date < '2024-01-01'
  GROUP BY 1
)
SELECT TO_CHAR(mth,'Mon YYYY') AS month, revenue,
       LAG(revenue) OVER (ORDER BY mth) AS prev_revenue,
       ROUND(100.0 * (revenue - LAG(revenue) OVER (ORDER BY mth))
             / NULLIF(LAG(revenue) OVER (ORDER BY mth), 0), 1) AS pct_change
FROM monthly ORDER BY mth;
```

Insight: `NULLIF(..., 0)` guards against division by zero, and `100.0` avoids integer division. Both traps in one query.

21.11 Day 11 — Interview Patterns

D11-P1

```
SELECT salary, COUNT(*) AS n, STRING_AGG(name, ', ') AS who
FROM employees GROUP BY salary HAVING COUNT(*) > 1;
```

D11-P2

```
WITH r AS (
  SELECT e.name, e.salary, d.dept_name,
         DENSE_RANK() OVER (PARTITION BY e.dept_id ORDER BY e.salary DESC) AS rk
  FROM employees e JOIN departments d ON d.dept_id = e.dept_id
)
SELECT dept_name, name, salary FROM r WHERE rk = 2;
```

D11-P3 See D6-P12.

D11-P4 See D6-P7.

D11-P5 See D6-P11.

D11-P6

```sql
WITH m AS (
  SELECT DISTINCT customer_id, DATE_TRUNC('month', order_date)::date AS mth
  FROM orders WHERE order_date >= '2023-01-01' AND order_date < '2024-01-01'
), g AS (
  SELECT customer_id, mth,
         LAG(mth) OVER (PARTITION BY customer_id ORDER BY mth) AS prev_mth
  FROM m
)
SELECT DISTINCT c.customer_name
FROM g JOIN customers c ON c.customer_id = g.customer_id
WHERE g.mth = g.prev_mth + INTERVAL '1 month';
```

D11-P7 See D9-P6.

D11-P8 See D10-P11.

D11-P9

```sql
SELECT TO_CHAR(m.mth,'Mon YYYY') AS month, COUNT(o.order_id) AS orders
FROM   generate_series('2023-01-01'::date,'2023-12-01'::date,'1 month') AS m(mth)
LEFT   JOIN orders o ON DATE_TRUNC('month', o.order_date) = m.mth
GROUP  BY m.mth
HAVING COUNT(o.order_id) = 0
ORDER  BY m.mth;
```

Insight: you cannot find a missing month by querying `orders` — the month has no rows there. You must generate the calendar first.

D11-P10

```sql
SELECT d.dept_name,
       ROUND(100.0 * COUNT(*) FILTER (WHERE e.hire_date >= '2021-01-01')
             / COUNT(*), 1) AS pct_recent
FROM employees e JOIN departments d ON d.dept_id = e.dept_id
GROUP BY d.dept_name ORDER BY pct_recent DESC;
```

D11-P11

```sql
WITH r AS (
  SELECT name, dept_id, salary,
         PERCENT_RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS pr
  FROM employees WHERE dept_id IS NOT NULL
)
SELECT * FROM r WHERE pr <= 0.25;
```

Alternative: `NTILE(4) OVER (PARTITION BY dept_id ORDER BY salary DESC) = 1`.

D11-P12

```sql
WITH ot AS (
  SELECT o.order_id, o.customer_id, o.order_date,
         SUM(oi.quantity * oi.unit_price) AS total
  FROM orders o JOIN order_items oi ON oi.order_id = o.order_id
  GROUP BY o.order_id, o.customer_id, o.order_date
)
SELECT c.customer_name, ot.order_date, ot.total
FROM ot JOIN customers c ON c.customer_id = ot.customer_id
ORDER BY ot.total DESC LIMIT 1;
```

D11-P13

```sql
SELECT p.project_name, d.dept_name, p.budget,
       ROUND(AVG(p.budget) OVER (PARTITION BY p.dept_id), 2) AS dept_avg_budget,
       CASE WHEN p.budget > AVG(p.budget) OVER (PARTITION BY p.dept_id)
            THEN 'Above' ELSE 'At or below' END AS position
FROM projects p JOIN departments d ON d.dept_id = p.dept_id;
```

D11-P14

```sql
WITH pc AS (
  SELECT emp_id, COUNT(*) AS n FROM employee_projects GROUP BY emp_id
), avg_pc AS (SELECT AVG(n) AS a FROM pc)
SELECT e.name, pc.n FROM pc CROSS JOIN avg_pc
JOIN employees e ON e.emp_id = pc.emp_id
WHERE pc.n > avg_pc.a ORDER BY pc.n DESC;
```

21.12 Day 12 — PostgreSQL Specifics

D12-P1

```sql
SELECT name, SPLIT_PART(email, '@', 2) AS domain
FROM employees WHERE email IS NOT NULL;
```

D12-P2

```sql
SELECT name, COALESCE(email, 'N/A') AS contact FROM employees;
```

D12-P3

```sql
SELECT name, hire_date,
       EXTRACT(YEAR FROM AGE(CURRENT_DATE, hire_date))::int AS years_of_service
FROM employees ORDER BY years_of_service DESC;
```

D12-P4

```sql
SELECT DATE_TRUNC('month', order_date)::date AS mth, COUNT(*) AS orders
FROM orders GROUP BY 1 ORDER BY 1;
```

D12-P5

```sql
SELECT dept_id,
       COUNT(*) AS n,
       ROUND(100.0 * COUNT(*) / SUM(COUNT(*)) OVER (), 2) AS pct_of_company
FROM employees WHERE dept_id IS NOT NULL GROUP BY dept_id ORDER BY pct_of_company DESC
    ;
```

Insight: `SUM(COUNT(*)) OVER ()` is the aggregate-then-window pattern from Day 10 — the window runs over the grouped rows.

D12-P6

```sql
SELECT d.dept_name, STRING_AGG(e.name, ', ' ORDER BY e.name) AS members
FROM employees e JOIN departments d ON d.dept_id = e.dept_id
GROUP BY d.dept_name ORDER BY d.dept_name;
```

D12-P7

```sql
SELECT order_id, order_date FROM orders
WHERE order_date >= DATE '2023-12-31' - INTERVAL '400 days'
ORDER BY order_date;
```

D12-P8

```sql
SELECT INITCAP(customer_name) AS name,
       TO_CHAR(signup_date, 'Mon YYYY') AS signed_up
FROM customers ORDER BY signup_date;
```

D12-P9

```sql
WITH span AS (
  SELECT customer_id, MAX(order_date) - MIN(order_date) AS days_span
  FROM orders GROUP BY customer_id HAVING COUNT(*) > 1
)
SELECT ROUND(AVG(days_span), 1) AS avg_days_between_first_and_last FROM span;
```

D12-P10

```sql
SELECT TO_CHAR(m.mth, 'Mon YYYY') AS month, COUNT(o.order_id) AS orders
FROM   generate_series('2023-01-01'::date, '2023-12-01'::date, '1 month') AS m(mth)
LEFT   JOIN orders o ON DATE_TRUNC('month', o.order_date) = m.mth
GROUP  BY m.mth ORDER BY m.mth;
```

21.13 Day 15 — Mock Interview, Section A

A1

```sql
SELECT e.name, e.salary
FROM employees e JOIN departments d ON d.dept_id = e.dept_id
WHERE d.dept_name = 'Engineering'
ORDER BY e.salary DESC;
```

A2

```sql
SELECT country, COUNT(*) AS customers
FROM customers GROUP BY country HAVING COUNT(*) >= 2 ORDER BY customers DESC;
```

A3 See D5-P8. The marks are for `LEFT JOIN` *and* `COALESCE`.

A4

```sql
SELECT d.dept_name, COUNT(*) AS headcount, ROUND(AVG(e.salary), 2) AS avg_salary
FROM employees e JOIN departments d ON d.dept_id = e.dept_id
GROUP BY d.dept_id, d.dept_name ORDER BY avg_salary DESC;
```

Note: the inner join already excludes the NULL-department employee, so no extra `WHERE` is needed — say that out loud rather than adding a redundant filter.

A5

```sql
WITH t AS (
  SELECT e.name, e.salary, e.dept_id,
         AVG(e.salary) OVER (PARTITION BY e.dept_id) AS dept_avg
  FROM employees e WHERE e.dept_id IS NOT NULL
)
SELECT t.name, d.dept_name, t.salary, ROUND(t.dept_avg, 2) AS dept_avg
FROM t JOIN departments d ON d.dept_id = t.dept_id
WHERE t.salary > t.dept_avg
ORDER BY d.dept_name, t.salary DESC;
```

A6

```sql
SELECT MAX(salary) AS second_highest
FROM employees WHERE salary < (SELECT MAX(salary) FROM employees);
```

Insight: full marks require the NULL-not-empty behaviour. `MAX` over an empty set returns NULL, which is exactly what is wanted — an `OFFSET 1` solution returns no rows instead and loses a mark.

A7 See D6-P6.

A8 See D9-P7 (`DENSE_RANK`, because ties must be included).

A9 See D10-P11.

A10

```sql
SELECT p.project_name, e1.name AS employee_1, e2.name AS employee_2
FROM   employee_projects a
JOIN   employee_projects b ON a.project_id = b.project_id AND a.emp_id < b.emp_id
JOIN   employees e1 ON e1.emp_id = a.emp_id
JOIN   employees e2 ON e2.emp_id = b.emp_id
JOIN   projects  p  ON p.project_id = a.project_id
ORDER  BY p.project_name, e1.name;
```

Insight: `a.emp_id < b.emp_id` is the whole problem. With <> you get every pair twice; with no condition you also get everyone paired with themselves.

21.14 Day 15 — Section D, Query Debugging

D1 **Problem:** `name` is neither in `GROUP BY` nor aggregated, so PostgreSQL raises *column "employees.name" must appear in the GROUP BY clause*. The deeper issue: a department has seven employees, so there is no single name to show.

Fix — depends on intent. If you want the department average only, drop `name`. If you want each employee *and* the department average, use a window function:

```sql
SELECT dept_id, name, AVG(salary) OVER (PARTITION BY dept_id) AS avg_sal
FROM employees;
```

D2 **Problem:** an aggregate in `WHERE`. `WHERE` runs before grouping, so `COUNT(*)` does not exist yet.

Fix:

```sql
SELECT name, salary FROM employees
GROUP BY name, salary HAVING COUNT(*) > 1;
```

D3 **Problem:** the `WHERE` on the right-hand table converts the `LEFT JOIN` into an `INNER JOIN`. For a customer with no orders, `o.status` is NULL, `NULL = 'DELIVERED'` is UNKNOWN, and the row is discarded.

Fix: move the condition into `ON`:

```sql
SELECT c.customer_name, o.order_id
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id AND o.status = 'DELIVERED';
```

D4 **Problem:** `manager_id` contains NULLs, so `emp_id NOT IN (\ldots , NULL)` is never TRUE and the query returns zero rows.

Fix:

```sql
SELECT name FROM employees e
WHERE NOT EXISTS (SELECT 1 FROM employees r WHERE r.manager_id = e.emp_id);
-- or: ... NOT IN (SELECT manager_id FROM employees WHERE manager_id IS NOT NULL)
```

D5 **Problem:** you cannot reference a window-function alias in `WHERE` — window functions are evaluated after `WHERE`.

Fix:

```sql
WITH r AS (
  SELECT name, dept_id,
         RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS r
  FROM employees
)
SELECT * FROM r WHERE r <= 3;
```

21.15 Day 15 — Section E, Schema Design

1. Tables

```sql
CREATE TABLE instructors (
    instructor_id  SERIAL PRIMARY KEY,
    name           VARCHAR(80) NOT NULL,
    specialisation VARCHAR(60)
);

CREATE TABLE students (
    student_id  SERIAL PRIMARY KEY,
    name        VARCHAR(80) NOT NULL,
    email       VARCHAR(120) NOT NULL UNIQUE,
    signup_date DATE NOT NULL DEFAULT CURRENT_DATE
);

CREATE TABLE courses (
    course_id     SERIAL PRIMARY KEY,
    title         VARCHAR(120) NOT NULL,
    description   TEXT,
    price         NUMERIC(10,2) NOT NULL CHECK (price >= 0),
    instructor_id INT NOT NULL REFERENCES instructors(instructor_id)
);

CREATE TABLE lessons (
    lesson_id        SERIAL PRIMARY KEY,
    course_id        INT NOT NULL REFERENCES courses(course_id) ON DELETE CASCADE,
    title            VARCHAR(120) NOT NULL,
    duration_minutes INT NOT NULL CHECK (duration_minutes > 0)
);

CREATE TABLE enrollments (                       -- junction table
    student_id      INT REFERENCES students(student_id),
    course_id       INT REFERENCES courses(course_id),
    enrolled_on     DATE NOT NULL DEFAULT CURRENT_DATE,
    progress_pct    NUMERIC(5,2) DEFAULT 0 CHECK (progress_pct BETWEEN 0 AND 100),
    PRIMARY KEY (student_id, course_id)
);

CREATE TABLE reviews (
    student_id  INT REFERENCES students(student_id),
    course_id   INT REFERENCES courses(course_id),
    rating      INT NOT NULL CHECK (rating BETWEEN 1 AND 5),
    review_text TEXT,
    created_on  DATE NOT NULL DEFAULT CURRENT_DATE,
    PRIMARY KEY (student_id, course_id)          -- enforces "at most one review"
);
```

2. Relationships

Relationship	Type	Implemented by
instructor → courses	one-to-many	FK on `courses`
course → lessons	one-to-many	FK on `lessons`
students ↔ courses (enrolment)	many-to-many	**junction** `enrollments`
students ↔ courses (review)	many-to-many, at most one	**junction** `reviews` with a composite PK

The composite primary key on `reviews` is what enforces "at most one review per student per course" —

a constraint, not application logic. Pointing that out is worth a mark.

3. Indexes

```
CREATE INDEX idx_courses_instructor ON courses(instructor_id);
CREATE INDEX idx_enrollments_course ON enrollments(course_id);
```

The first supports "all courses by this instructor"; the second supports "all students on this course", which the composite PK `(student_id, course_id)` does *not* help with, because `course_id` is not the leading column.

4. The query

```
SELECT c.title,
       i.name AS instructor,
       ROUND(AVG(r.rating), 2) AS avg_rating,
       COUNT(*) AS review_count
FROM   reviews r
JOIN   courses c     ON c.course_id     = r.course_id
JOIN   instructors i ON i.instructor_id = c.instructor_id
GROUP  BY c.course_id, c.title, i.name
HAVING COUNT(*) >= 2
ORDER  BY avg_rating DESC, review_count DESC
LIMIT  3;
```

Interview Insight

Marking your own schema. Award the 8 design marks for: all six tables present (2), correct PKs including the two composite ones (2), all FKs present (2), and at least four meaningful constraints beyond keys — `NOT NULL`, `UNIQUE` on email, `CHECK` on rating, `CHECK` on price or progress (2).
The most common omissions are the composite primary key on `reviews` (people add a surrogate `review_id` and then cannot enforce "one review per student per course") and the `CHECK` on the rating range.

21.16 Connection Lab (Chapter 1)

L1 `whoami` asks the **operating system** which Linux user is running the command. It has nothing to do with PostgreSQL. Whatever it printed is the name that peer authentication will compare against a role name.

L2 `which psql` shows where the client executable lives (typically `/usr/bin/psql`), proving that `psql` is an ordinary Linux program. `psql -version` reports the **client** version. The *server* version is not shown — for that you need `SELECT version();` after connecting, and the two can legitimately differ.

L3

`SELECT current_user;`	SQL, answered by the server: which *PostgreSQL role* you authenticated as. Here, `postgres`.
`SELECT current_database();`	SQL: which database you are connected to. With no `-d`, it defaults to one named after the role, so `postgres`.
`SELECT version();`	SQL: the *server* version string.
`\du`	a psql meta-command listing roles and their attributes. Not SQL — psql runs a query on your behalf and formats the result.
`\l`	lists databases.
`\conninfo`	tells you the database, user, socket or host, and port for this session — the fastest way to answer "where am I actually connected?"
`\q`	quits the client. The server keeps running.

L4 The eight-step trace:

1. The shell runs the command as your Linux user (say `saiyam-jain`).
2. `psql` launches under that same identity — no `sudo` was used.
3. `-U postgres` requests the *PostgreSQL role* `postgres`.
4. No `-h` was given, so this is a local **Unix socket** connection, matching `local` rules.
5. PostgreSQL scans `pg_hba.conf` top to bottom; the first matching rule is `local all postgres peer`.
6. Peer asks the OS which user opened the socket: `saiyam-jain`.
7. `saiyam-jain` $\neq$ `postgres`.
8. The connection is rejected with *Peer authentication failed for user "postgres"*.

Insight: nothing is broken. The server is running, the role exists, and your password (if any) is irrelevant — peer never asks for one. Only the OS identity failed the test.

L5 Because peer authentication now *succeeds*: your OS user and the requested role are the same string. With no `-U`, psql defaults the role to your OS username, so the comparison is trivially satisfied. No password is requested because peer does not use passwords at all — the one you set with `CREATE USER` is only used for TCP connections.

L6 Adding `-h localhost` switches the connection from the Unix socket to **TCP**. That changes which `pg_hba.conf` line matches: instead of a `local` rule you now match `host all all 127.0.0.1/32 scram-sha-256` (or the `::1/128` line if localhost resolves to IPv6), so you are prompted for a password.

Insight: same machine, same person, same role — different authentication, purely because of one flag. This is the clearest possible demonstration of why the socket/TCP distinction matters.

L7 All three are server settings, so your machine's real answers beat anything quoted in a book. `SHOW hba_file;` gives the actual path (version- and distro-dependent); `SHOW port;` is normally 5432; `SHOW unix_socket_directories;` is usually `/var/run/postgresql` on Debian/Ubuntu. Always prefer asking the server over trusting a remembered path.

21.17 Connection & Authentication Interview Answers (Chapter 1)

1. What is PostgreSQL? An open-source relational database management system — the *server* process that stores data, executes SQL, and manages roles, connections and transactions.

2. What is psql? PostgreSQL's command-line *client*: a program that connects to the server, sends SQL, and displays results.

3. Is psql the PostgreSQL server? No. They are separate programs. psql is one of many possible clients — pgAdmin, DBeaver, JDBC and psycopg2 all talk to the same server.

4. What happens when you type `psql`? The shell locates the executable on `PATH` and runs it; psql decides how to connect (Unix socket unless `-h` is given) and as which role (your OS username unless `-U` is given); the server consults `pg_hba.conf`, authenticates the connection, and psql presents an interactive prompt.

5–8. `-U`, `-h`, `-p`, `-d`? `-U` the PostgreSQL role to authenticate as; `-h` the host (its presence also forces TCP instead of a socket); `-p` the port, default 5432; `-d` the database to connect to.

9. What is a PostgreSQL role? An identity inside the database server that can own objects and be granted privileges. A role with the LOGIN attribute can connect, which is what older docs call a "user".

10. Is a role the same as a Linux user? No — two independent identity systems. They share names only when someone deliberately creates matching ones, which is exactly what peer authentication relies on.

11. What does `sudo -u postgres psql` do? It is a *Linux* command: `sudo` runs `psql` as the Linux user `postgres`. psql then defaults its role to that OS username, so under peer authentication the names match and the connection succeeds.

12. Difference between the two commands? `sudo -u` changes your **operating system** identity before psql starts; `-U` leaves your OS identity alone and changes which **PostgreSQL role** psql requests. Under peer, the first works and the second usually fails.

13. What is peer authentication? For local socket connections, PostgreSQL asks the OS which user opened the connection and requires that name to equal the requested role. No password is used.

14. Why can `psql -U postgres` fail? Because your OS user is not `postgres`, and peer compares the two names literally.

15. What is SCRAM? `scram-sha-256`, PostgreSQL's modern password authentication. The password is never sent across the connection and is stored salted, so intercepting the exchange does not reveal it. Default since PostgreSQL 14.

16. What is MD5 authentication? The legacy password method that SCRAM replaced. Still found in older configurations; weaker, and worth migrating away from.

17. What is trust authentication? PostgreSQL performs no authentication at all and accepts any matching connection as whatever role it claims.

18. Why is trust dangerous? It removes the check entirely. A rule such as `host all all 0.0.0.0/0 trust` grants anyone who can reach the port full superuser access with no password. It is a common cause of compromised databases and should never be used to silence an authentication error.

19. What is certificate authentication? `cert`: the client proves its identity with a TLS client certificate and the matching private key, rather than a password. The server verifies the certificate was signed by a trusted CA and maps its identity to a role. Used for machine-to-machine authentication where no human types a password.

20–21. What is `pg_hba.conf`? What does HBA stand for? Host-Based Authentication — the configuration file listing which authentication method applies to each kind of incoming connection. Columns: TYPE, DATABASE, USER, ADDRESS, METHOD.

22. How does PostgreSQL choose a rule? It reads the file top to bottom and applies the **first** rule matching the connection type, database, role and address — then stops. Later rules are never consulted, which is why rule order is logic rather than cosmetics.

23. Unix socket vs TCP? A socket is a local filesystem endpoint used when no `-h` is given; it never touches the network and supports peer authentication. TCP is a network connection (used with `-h`, even for localhost) and matches `host` rules, which typically require a password. The same role can be treated differently by each.

24. Authentication vs authorization? Authentication establishes *who you are* and happens at connection time (governed by `pg_hba.conf`). Authorization establishes *what you may do* and happens per statement (governed by `GRANT` and ownership). A *permission denied* error means authentication already succeeded.

25. Default port? 5432, though it is configurable — a second instance on the same machine commonly uses 5433.

Interview Insight

The two most likely of these to be asked in a general SDE interview are 24 (authentication vs authorization — a security fundamental, not really a PostgreSQL question) and 22 (first matching rule — because it shows you have actually configured something rather than only read about it).

Chapter 22

Relational Algebra & Tuple Relational Calculus (Bonus Chapter)

Relational Algebra & Tuple Relational Calculus

Why This Matters: SQL is just one way to express queries. Underneath, databases use Relational Algebra (RA) for query planning and Tuple Relational Calculus (TRC) as a theoretical foundation. Understanding both helps you understand why certain queries are fast or slow (query optimization), think about data transformation at a higher level than SQL syntax, and handle GATE-style or academic theory questions.

The progression: Relational Model → RA → TRC → SQL

22.1 Relational Algebra (Procedural)

What It Is: A procedural (step-by-step) way to express queries. You specify how to get the result.

Key Operators:

Operator	Symbol	Meaning	Example
Selection	σ	Filter rows (WHERE)	σ age>25(Customers)
Projection	π	Select columns (SELECT)	π name,email(Customers)
Union	∪	Combine two relations	R ∪ S (same schema)
Intersection	∩	Common rows	R ∩ S
Difference	−	Rows in R but not S	R − S
Cartesian Product	×	All combinations	R × S
Rename	ρ	Rename relation/attributes	ρ P(R)
Theta Join	⋈θ	Join with condition	R ⋈ R.x=S.y S
Natural Join	⋈	Join on common columns	R ⋈ S (auto-match columns)
Outer Join	L/R/F ⋈	Left/Right/Full outer	keep unmatched rows from one or both sides
Division	÷	Rows matching all of S	R ÷ S (advanced)

22.2 Examples: English → Relational Algebra

Example 1: Find names of all customers over 30

```
English: SELECT name FROM Customers WHERE age > 30
```

```
RA: π name(σ age>30(Customers))
```

Example 2: Find all orders (order_id, customer_name) where status is 'shipped'

```
English: SELECT o.order_id, c.name FROM Orders o JOIN Customers c ON o.cid = c.id WHERE
o.status = 'shipped'
```

```
RA: π order_id,name(σ status='shipped'(Orders ⋈ Customers))
```

Example 3: Find customers who have placed at least one order (semi-join)

```
RA: π customer_id(Orders) — projects the unique customers who have orders
```

22.3 Query Evaluation & Optimization

The optimizer converts SQL into RA and then optimizes the expression tree:

Naive Plan: (Orders × Customers) → Filter(status='shipped') → Project(order_id, name) — cost: a Cartesian product of huge tables (very slow).

Optimized Plan: Filter(Orders, status='shipped') → Join(Customers) → Project(order_id, name) — cost: filter first to reduce rows, then join (much faster).

> *Key Principle: Push selections (σ) down the tree (filter early); avoid Cartesian products by using joins instead.*

22.4 Tuple Relational Calculus (Declarative)

What It Is: A declarative way to express queries. You specify what you want, not how to get it.

```
Basic structure: { t | Condition(t) } — “find all tuples t that satisfy Condition”
```

Example 1: Find all customer names

```
TRC: { c.name | c ∈ Customers }
```

Meaning: find all names c.name where c is in the Customers table. SQL equivalent: SELECT name FROM Customers

Example 2: Find all order IDs where the customer's age > 25

```
TRC: { o.order_id | o ∈ Orders ∧ ∃c (c ∈ Customers ∧ c.id = o.cid ∧ c.age > 25) }
```

Meaning: find all order IDs belonging to orders where a matching customer exists with age over 25.

Logical connectives used in TRC:

- ∧ = AND
- ∨ = OR
- ¬ = NOT
- ∃ = EXISTS (there exists at least one)
- ∀ = FOR ALL (for every)

Free vs Bound Variables: In { t | c ∈ Customers ∧ c.age > 30 }, t is free (it appears in the result), while c is bound (it is quantified by ∈).

22.5 Safety & Equivalence

TRC Safety: A query is safe if it always returns a finite result (it doesn't return infinite tuples drawn from the universe).

```
Example unsafe query: { t | ¬(t ∈ Customers) } — this returns everything not in
Customers, which is infinite.
```

Equivalence: RA and TRC are equivalent in expressive power — any RA expression can be written as TRC, and vice versa. However, SQL is strictly more powerful than both, because it allows aggregates (COUNT, SUM) and GROUP BY, which RA/TRC cannot express directly.

> *Interview Insight: If asked “why do we still teach relational algebra if nobody writes it by hand?” — the honest answer is that it's the internal language query optimizers actually think in. When you read an EXPLAIN plan, you are reading a physical realization of an RA expression tree.*

Chapter 23
Distributed & Scalable Databases (Bonus Chapter)

Distributed & Scalable Databases

Why This Matters: Single-node databases hit limits: disk space, CPU, network bandwidth. Distributed databases scale horizontally by spreading data across multiple servers. This chapter covers the architectural patterns (replication, sharding), theoretical foundations (CAP theorem), and the practical trade-offs you'll face in real systems.

23.1 Partitioning vs Sharding (Quick Distinction)

Partitioning (single database): data split across multiple tables or disks on one server. Example: table `orders` split into `orders_2023`, `orders_2024` — physically separate, logically one database.

- Single server, single coordination point.
- Useful for maintenance (archive old partitions) and query performance (scan only the relevant partition).

Sharding (distributed): data split across multiple servers. Each shard is a complete, separate database. Example: customers 1–1000 on shard 1, customers 1001–2000 on shard 2.

- Multiple independent servers; a genuinely distributed architecture.
- Buys scalability, but adds complexity: cross-shard queries, consistent hashing, rebalancing.

23.2 Sharding Strategies

Range Sharding: split by range — customer_id 1–1M on shard 1, 1M–2M on shard 2.

- Pro: easy to understand; range queries stay within a shard.
- Con: hot spots if data isn't evenly distributed (e.g. all new customers routed to the latest shard).

Hash Sharding: shard_id = hash(customer_id) % num_shards.

Consistent Hashing: map both keys and shards onto a hash ring. Adding or removing a shard only affects nearby keys.

- Pro: minimal data movement when scaling up or down; used in real systems (Memcached, Cassandra).
- Con: slightly more complex; requires ring management.

Directory-Based Sharding: keep a lookup table, customer_id → shard_id, with a coordinator service handling routing.

- Pro: extremely flexible; supports any sharding logic; easy to rebalance.
- Con: the coordinator is a bottleneck; adds an extra hop on every query; single point of failure if not replicated.

23.3 Replication: Consistency vs Availability

Primary-Replica (Master-Slave): all writes go to the primary; reads typically go to replicas (though they can go to the primary too). Replicas are exact, usually read-only copies that provide redundancy and read scaling.

Synchronous Replication: the coordinator waits for a replica to acknowledge before confirming the write to the client.

- Pro: strong consistency — the write reaches all replicas before responding.

- Con: slow (network latency multiplied by number of replicas); any replica lag blocks writes.

Asynchronous Replication: the coordinator confirms the write to the client immediately; replicas sync in the background.

- Pro: fast writes — no waiting on replicas.
- Con: temporary inconsistency (replica lag); if the primary crashes before replicating, writes can be lost.

> *Practical Trade-Off: most production systems use a hybrid — wait for a quorum (e.g. 2 out of 3 replicas) before confirming a write, balancing durability against latency.*

23.4 The CAP Theorem

In the presence of a network Partition, a distributed system must choose between Consistency and Availability — it cannot guarantee both.

CP (Consistency + Partition tolerance): ACID relational databases, HBase. When a partition occurs, nodes that can't confirm they're consistent go offline. Trade-off: availability is lost during partitions.

AP (Availability + Partition tolerance): DynamoDB, Cassandra, and many NoSQL systems. When a partition occurs, all nodes stay online and accept writes independently, causing temporary inconsistency. Trade-off: consistency becomes eventual — data converges once the partition heals.

CA (Consistency + Availability): only possible on a single node, where there is no partition risk. Not applicable to genuinely distributed systems, since network partitions are inevitable at scale.

ACID vs BASE: ACID = Atomicity, Consistency, Isolation, Durability (the strong guarantees typical of CP systems). BASE = Basically Available, Soft state, Eventually consistent (the looser but more available model typical of AP systems).

23.5 Practical Challenges of Sharded Systems

Cross-Shard Queries: “Find all orders by customers over age 30” is a problem if age is not the shard key (data is sharded by customer_id), so every shard must be scanned. The coordinator fans the query out to all shards, aggregates the results, and returns them to the client — much slower than a single-shard query, and worth avoiding in performance-critical paths.

Joins Across Shards: if customers live on shard 1 and orders on shard 2 (different sharding schemes), the join has to fetch data from both shards and combine it at the coordinator — expensive in both memory and network traffic.

Resharding: consistent hashing minimizes data movement, since only keys near the changed part of the ring are affected. A common technique is a double-write strategy: write to both the old and new shard during the transition, then verify consistency before cutting over.

23.6 Interview Scenarios

Scenario 1: “Design a system to handle 1 billion users.”

- A single database becomes a bottleneck — you need sharding.
- Shard by user_id using consistent hashing.
- Each shard has a primary plus replicas, for redundancy.
- A coordinator routes requests to the correct shard.
- Cross-shard analytics queries are handled separately (MapReduce-style aggregation).

Scenario 2: “A customer reports missing data; I suspect replication lag.”

- Likely cause: asynchronous replication — the customer read from a replica before the write fully propagated.

- Solutions: read-after-write consistency (route a user's own reads to the primary), wait for quorum replication, or version vectors to track causality.

Scenario 3: “We had a network partition; part of the cluster went offline.”

- The response depends on the system's CP vs AP choice.
- CP system: goes read-only (or fully unavailable) during the partition.
- AP system: accepts conflicting writes on both sides; needs conflict resolution on merge (last-write-wins, version vectors, or application-level logic).

Chapter 24

pgvector: Vector Similarity Search in PostgreSQL

(Bonus Chapter)

pgvector: Vector Similarity Search in PostgreSQL

Why This Matters: Modern applications — semantic search, recommendation engines, RAG (retrieval-augmented generation) pipelines for LLMs — need to store embeddings (arrays of floats representing meaning) and find the "closest" ones to a query. pgvector adds a native vector type and nearest-neighbor search directly to PostgreSQL, so you don't need a separate vector database for most workloads. It is now a regular fixture in PostgreSQL interviews for roles touching search or AI features.

24.1 Installing the Extension

pgvector ships as a PostgreSQL extension, not a core feature, so it must be installed on the server and then enabled per-database:

```
-- one-time, on the OS (e.g. via apt or building from source):
-- apt install postgresql-16-pgvector

-- then, inside the target database:
CREATE EXTENSION IF NOT EXISTS vector;
```

> *Interview Insight: "Why not just store embeddings as an array of floats (float4[])?" — because a plain array has no distance operators or index support. pgvector adds a dedicated type plus operators (<->, <#>, <=>) that the planner and index machinery understand.*

24.2 The vector Type

A column is declared with a fixed dimensionality matching your embedding model's output size (for example, 1536 for OpenAI's text-embedding-3-small, or 384 for many small sentence-transformer models):

```
CREATE TABLE documents (
id BIGSERIAL PRIMARY KEY,
title TEXT NOT NULL,
body TEXT NOT NULL,
embedding VECTOR(1536)
);

INSERT INTO documents (title, body, embedding)
VALUES ('Intro to CTEs', 'A CTE is a named subquery...', '[0.012, -0.034, ...]');
```

The literal form is a bracketed list of floats passed as text, which PostgreSQL casts to `vector`. In practice the application layer (Python, Node, etc.) generates the array from an embedding model call and passes it as a parameter.

24.3 Distance Operators

Operator	Distance	Typical use
<->	Euclidean (L2)	General-purpose; default choice if unsure
<#>	Negative inner product	Fastest; use only with normalized vectors
<=>	Cosine distance	Most common for text embeddings

Nearest-neighbor query — find the 5 documents closest to a query embedding:

```
SELECT id, title, embedding <=> '[0.01, -0.02, ...]' AS distance
FROM documents
ORDER BY embedding <=> '[0.01, -0.02, ...]'
LIMIT 5;
```

Common Mistake: mixing distance operators between the ORDER BY clause and an index built for a different operator class. An index built with `vector_cosine_ops` *only accelerates queries ordered by* `<=>`*; ordering by* `<->` *against that same index falls back to a full sequential scan.*

24.4 Indexing: IVFFlat vs HNSW

Without an index, a nearest-neighbor query is an exact but linear scan of every row — fine for a few thousand rows, too slow at scale. Both index types below are approximate nearest neighbor (ANN) indexes: they trade a small amount of recall for large speedups.

IVFFlat (Inverted File with Flat compression):

```
CREATE INDEX ON documents
USING ivfflat (embedding vector_cosine_ops)
WITH (lists = 100);
```

- Clusters vectors into `lists` partitions (rule of thumb: rows / 1000, capped and tuned by testing).
- Must be built after the table has representative data — an index built on an empty table clusters poorly.
- Faster to build and smaller than HNSW, but generally lower recall at the same speed.

HNSW (Hierarchical Navigable Small World):

```
CREATE INDEX ON documents
USING hnsw (embedding vector_cosine_ops)
WITH (m = 16, ef_construction = 64);
```

- Builds a multi-layer graph of neighbors; no dependency on existing data distribution, so it can be built before data is loaded.
- Better query-time recall/speed trade-off than IVFFlat in most benchmarks, at the cost of slower, more memory-hungry index builds.
- Query-time recall is tunable via `SET hnsw.ef_search = 100;` — higher values mean better recall, more CPU per query.

Interview Insight: “Which index would you pick?” — HNSW is the default recommendation for new projects on recent pgvector versions: better recall/latency trade-off, and it doesn't need to be rebuilt as the data distribution shifts. IVFFlat is still reasonable when build time or memory footprint dominates the decision.

24.5 Hybrid Search: Combining Vectors with SQL

The real advantage over a dedicated vector database is that ordinary WHERE clauses, joins, and transactions still work alongside the vector search:

```
SELECT d.id, d.title, d.embedding <=> :query_vec AS distance
FROM documents d
JOIN authors a ON a.id = d.author_id
WHERE d.published_at > now() - interval '1 year'
AND a.is_active = true
ORDER BY d.embedding <=> :query_vec
LIMIT 10;
```

This is often called hybrid search: a metadata filter narrows the candidate set, and the vector operator ranks what's left. Filtering before the ANN index is consulted (rather than filtering the top-K results after the fact) is what keeps recall high when the filter is selective; pgvector supports this via standard query planning once an appropriate index exists on the filtered

columns too.

24.6 Self-Test — Say These Out Loud

- Why does cosine distance need normalized vectors to match inner product ranking? — because inner product alone is sensitive to vector magnitude, not just direction; cosine distance divides that out.
- What happens if you query with <-> against an index built with vector_cosine_ops? — the index is not used for that operator; PostgreSQL falls back to a sequential scan.
- Why build HNSW before bulk-loading data, but IVFFlat after? — IVFFlat's clusters are computed from a sample of existing rows, so an empty table produces poor clusters; HNSW's graph structure doesn't depend on the data's initial distribution.
- How do you trade recall for speed at query time? — raise hnsw.ef_search (HNSW) or ivfflat.probes (IVFFlat); both default low for speed and can be raised per-session.